



fit@hcmus

DATA STRUCTURES & ALGORITHMS

Lecture 2: Sorting (part 1)

Lecturer: Dr. Nguyen Hai Minh



CONTENT

- Introduction
- Analysis of sorting algorithms using different algorithm design methods
 - Brute force: Selection Sort & Bubble Sort
 - Decrease-and-conquer: Insertion sort
 - Divide-and-conquer: Mergesort & Quicksort
 - Transform-and-conquer: Heapsort

INTRODUCTION

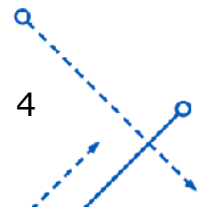
5/20/24

nhminh@FIT

3

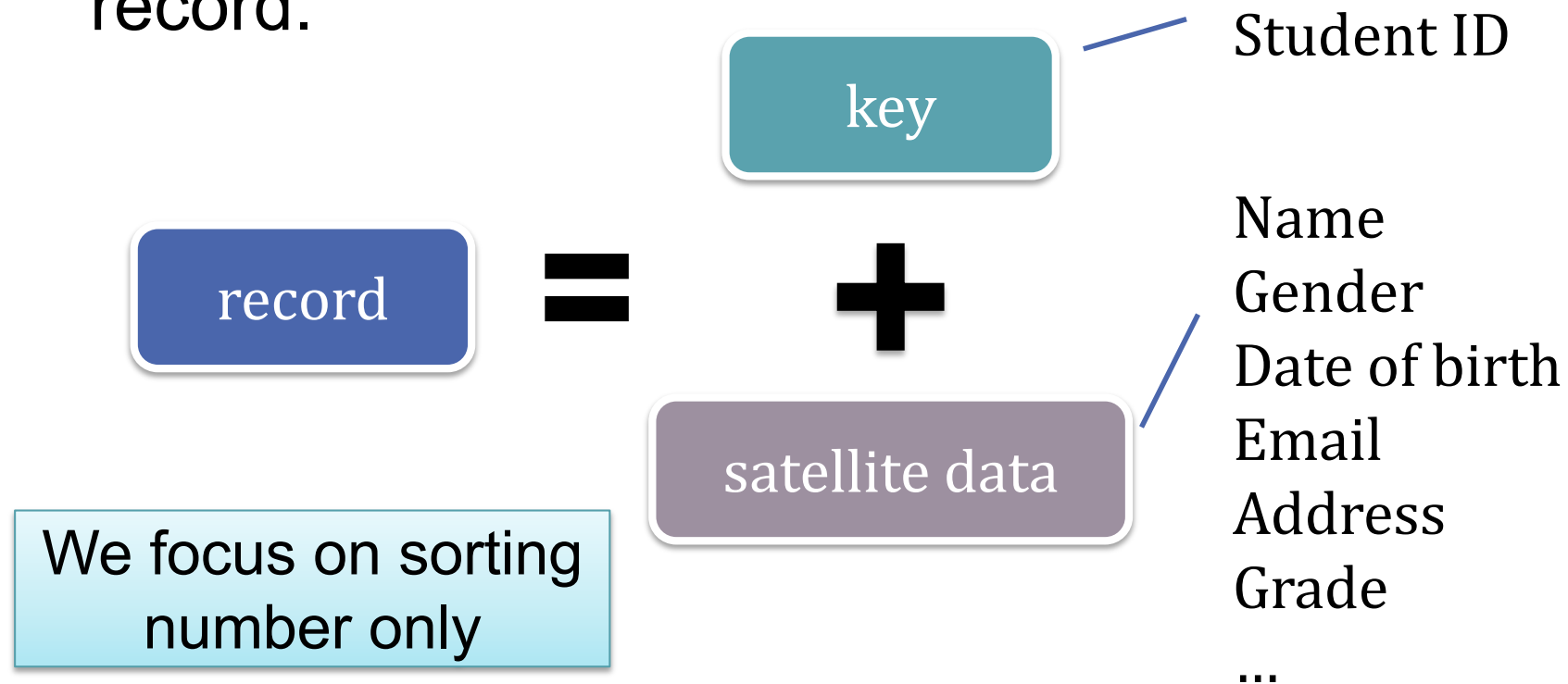
What is Sorting?

- Sorting is a process that organizes a collection of data into either **ascending** or **descending** order.
- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A **permutation** (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a_1 \leq a_2 \leq \dots \leq a_n$
- The input sequence is usually an n -element array, although it may be represented in some other fashion, such as a linked list.



The Structure of Data

- The numbers to be sorted are rarely isolate values.
- Each is usually part of a collection of data called record.



Why Sorting?

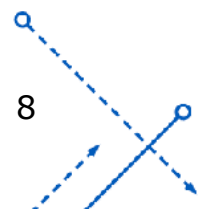
- Sorting is considered the **most fundamental problem** in the study of algorithms
 1. The need of an application
 2. Sorting is used as a subroutine: *searching, min, max, k^{th} smallest, ...*
 3. Many important techniques are used throughout algorithm design appear in sorting algorithms:
 - Iterative
 - Recursive
 - Divide-and-conquer
 - Randomized algorithms
 - ...

Why Sorting?

- By analyzing sorting algorithms, we can prove a nontrivial **lower bound for sorting** to see that some sorting algorithms are *asymptotically optimal*.
- *This lower bound can be used to prove lower bounds for various other problems.*

Classification of sorting algorithms

1. **By computational complexity** of element comparisons in terms of the size of the array (n)
 - Good: $O(n \log_2 n)$
 - Bad: $O(n^2)$
 - Ideal: $O(n)$
2. **Memory usage**: in-place sort / not in-place sort
3. **By stability**: maintain the relative order of the records with equal keys.
4. **Comparison** compare or not?
5. **Adaptability**: whether or not the presortedness of the input affects the running time
6. **Algorithm strategies**



Classified by Complexity

❑ Bad: $O(n^2)$

- Selection Sort
- Bubble Sort
- Insertion Sort

❑ Ideal: $O(n)$

- Counting Sort
- Bucket Sort
- Radix Sort

❑ Good: $O(n \log_2 n)$

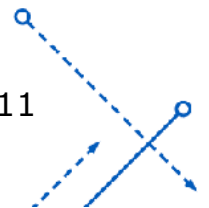
- Heapsort
- Mergesort
- Quicksort

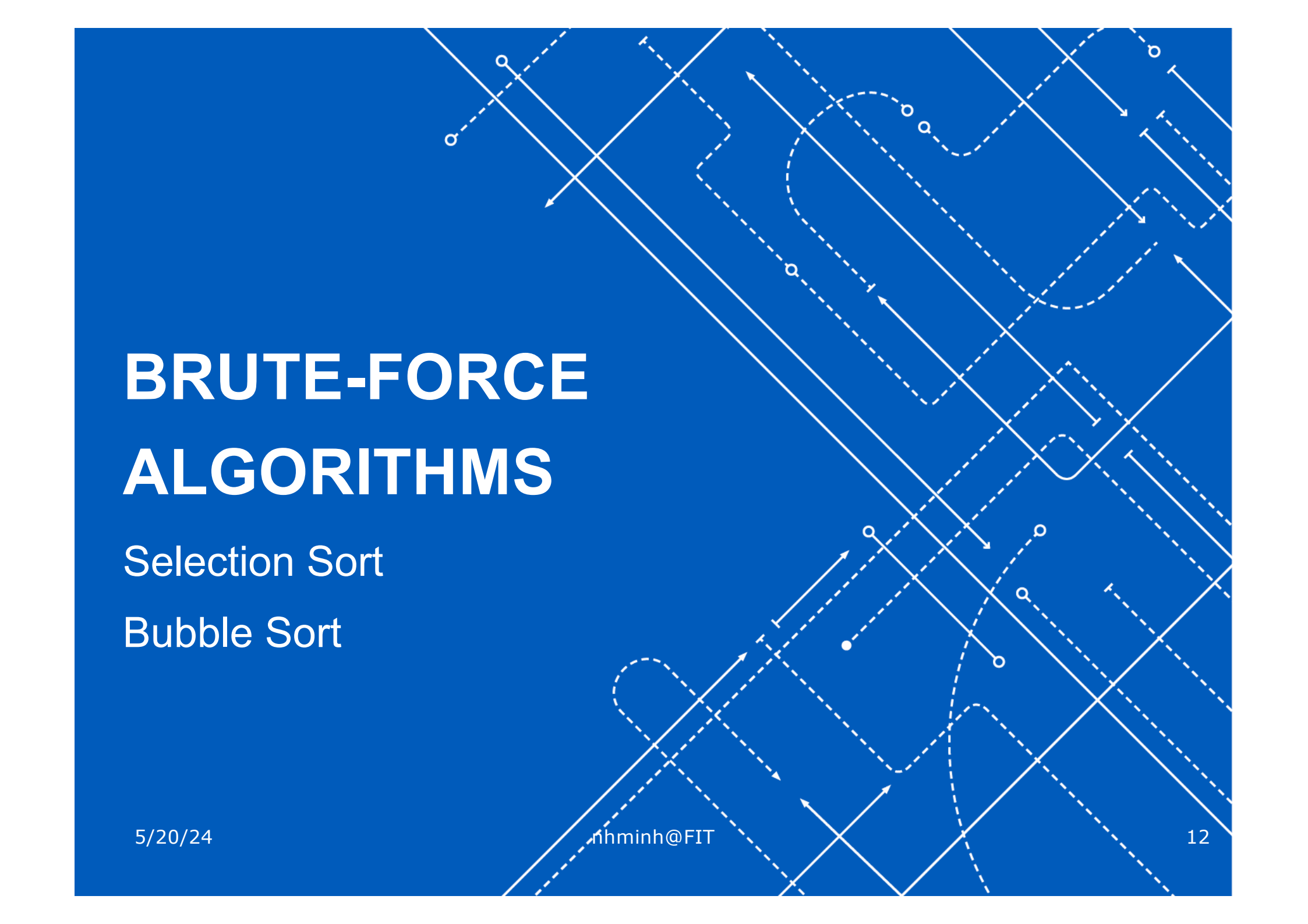
Classified by Comparison or not

- ❑ **Comparison Sorts:** Selection Sort, Bubble Sort, Insertion Sort, Heapsort, Mergesort, Quicksort, etc.
- ❑ **Non-comparison Sorts:** Counting Sort, Bucket Sort, Radix Sort, etc.

Classified by Algorithm Design

- ❑ Brute-force: Selection Sort, Bubble Sort
- ❑ Decrease-and-conquer: Insertion Sort
- ❑ Divide-and-conquer: Mergesort, Quicksort
- ❑ Transform-and-conquer: Heapsort
- ❑ Space-and-time tradeoffs: Counting Sort, Radix Sort



The background of the slide is a solid blue color. Overlaid on this background is a complex, abstract pattern of white lines and dots. The pattern consists of several intersecting straight lines, some solid and some dashed, creating a grid-like structure. Scattered throughout this grid are numerous small white circles, some of which are connected by dashed lines, suggesting a network or a path. The overall effect is a technical, geometric aesthetic.

BRUTE-FORCE ALGORITHMS

Selection Sort

Bubble Sort

Brute Force Algorithms



- **Brute force** is a **straightforward** approach to solve a problem, directly based on the problem statement and definitions of the concepts involved.

- The “**force**” implied “**Just do it!**”
- Brute force strategy is the easiest to apply
- Brute-force algorithms consider all possibilities to find the appropriate one.

- For example:

- Calculate a^n
→ Multiplying 1 by a n times.

Selection Sort - Idea

- Given an array of n items, start with index i
 1. Find the smallest item x in the array
 2. Swap x with the i^{th} item
 3. Increase i by 1 and go to Step 1
- We can also find the largest and put it in the end instead.

	8
	5
	2
	6
	9
	3
	1
	4
	0
	7

[Click here to view animation](#)



Selection Sort - Illustration

□ Example

-  Unsorted items
-  1st unsorted items
-  Sorted items
-  Min item for current iteration

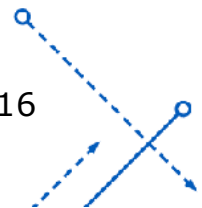
0	1	2	3	4	5	6	7
9	12	8	15	3	6	4	10
3	12	8	15	9	6	4	10
3	4	8	15	9	6	12	10
3	4	6	15	9	8	12	10
3	4	6	8	9	15	12	10
3	4	6	8	9	15	12	10
3	4	6	8	9	10	12	15
3	4	6	8	9	10	12	15

15



Selection Sort Algorithm

SELECTION-SORT(A,n) //Input: An array $A[0..n - 1]$ of orderable elements //Output: Array $A[0..n - 1]$ sorted in nondecreasing order	Cost times
<pre> 1 for $i \leftarrow 0$ to $n - 2$ do 2 $\text{min} \leftarrow i$ 3 for $j \leftarrow i + 1$ to $n - 1$ do 4 if $A[j] < A[\text{min}]$ 5 $\text{min} \leftarrow j$ 6 swap $A[i]$ and $A[\text{min}]$ </pre> Basic operation	n $n - 1$ $C(n) + 1$ $C(n)$ $C(n) - \alpha$ $n - 1$



Selection Sort Analysis

1. Input size: n
2. Basic operation: key comparison $A[j] < A[\min]$
3. The number of key comparisons ***depends only on the array size***
4. Sum of number of times the basic operation is:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i).$$

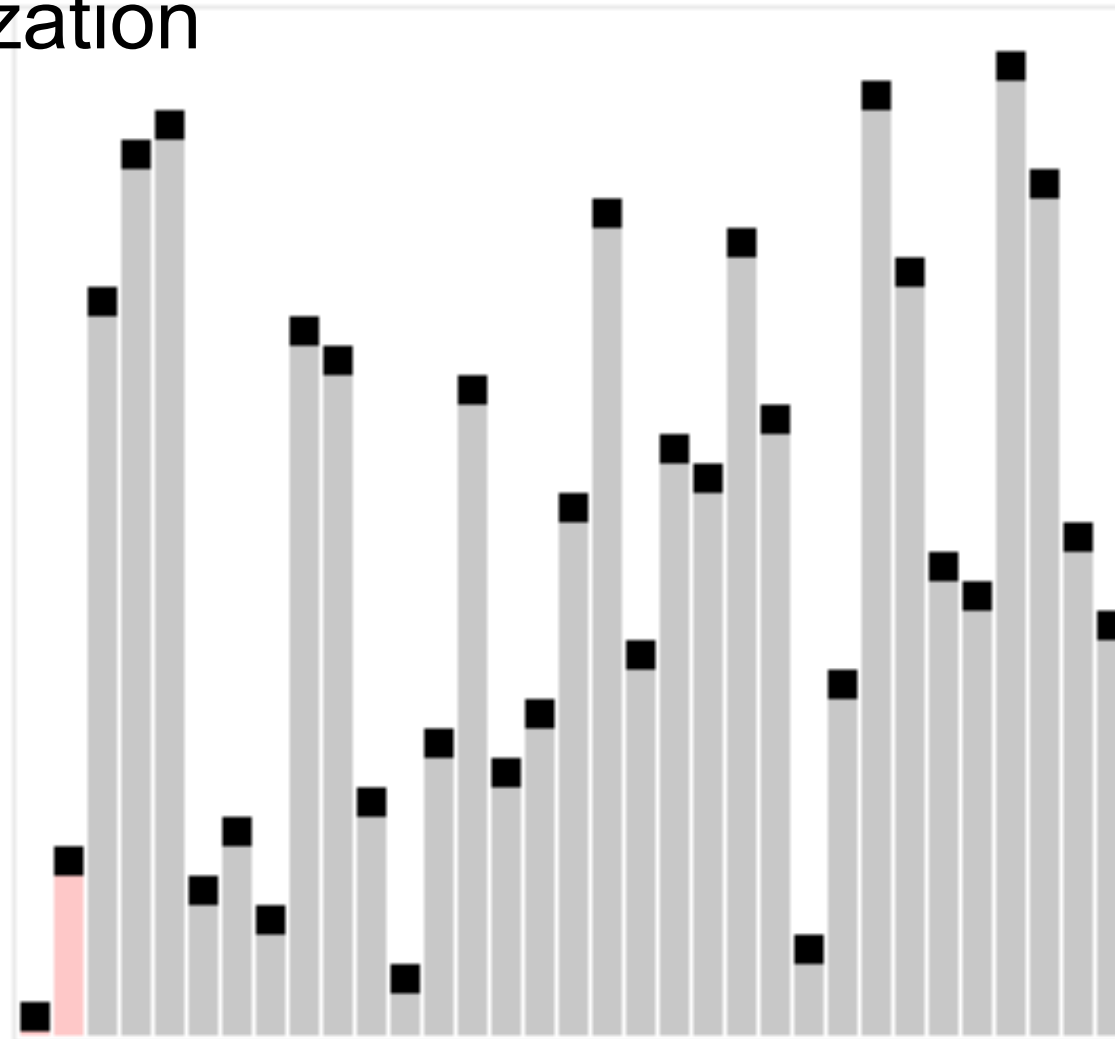
$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}.$$

5. Order of growth: **$O(n^2)$**



Selection Sort - Illustration

□ Visualization



[Click here to view animation](#)

nhminh@FIT

5/20/24

Bubble Sort - Idea

- Given an array of n items
 1. Compare pair of adjacent items
 2. Swap if the items are out of order
 3. Repeat until the end of array
 4. Reduce n by 1 and go to Step 1

8 5 3 1 4 7 9

[Click here to view animation](#)

Large item is like “bubble” that floats to the end of the array

Bubble Sort - Illustration

□ Example

■ 1st pass

✕ Unsorted items

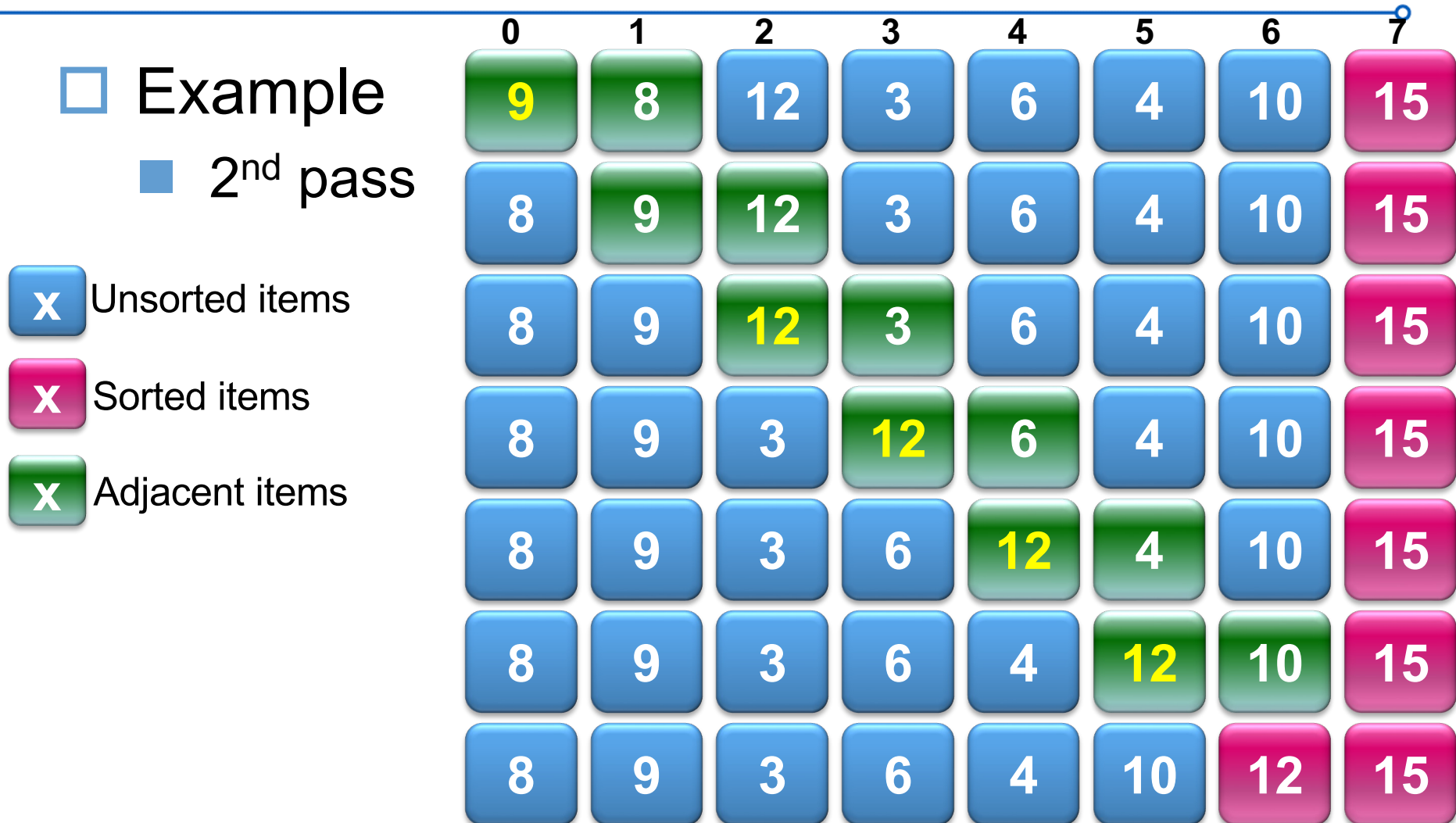
✕ Sorted items

✕ Adjacent items

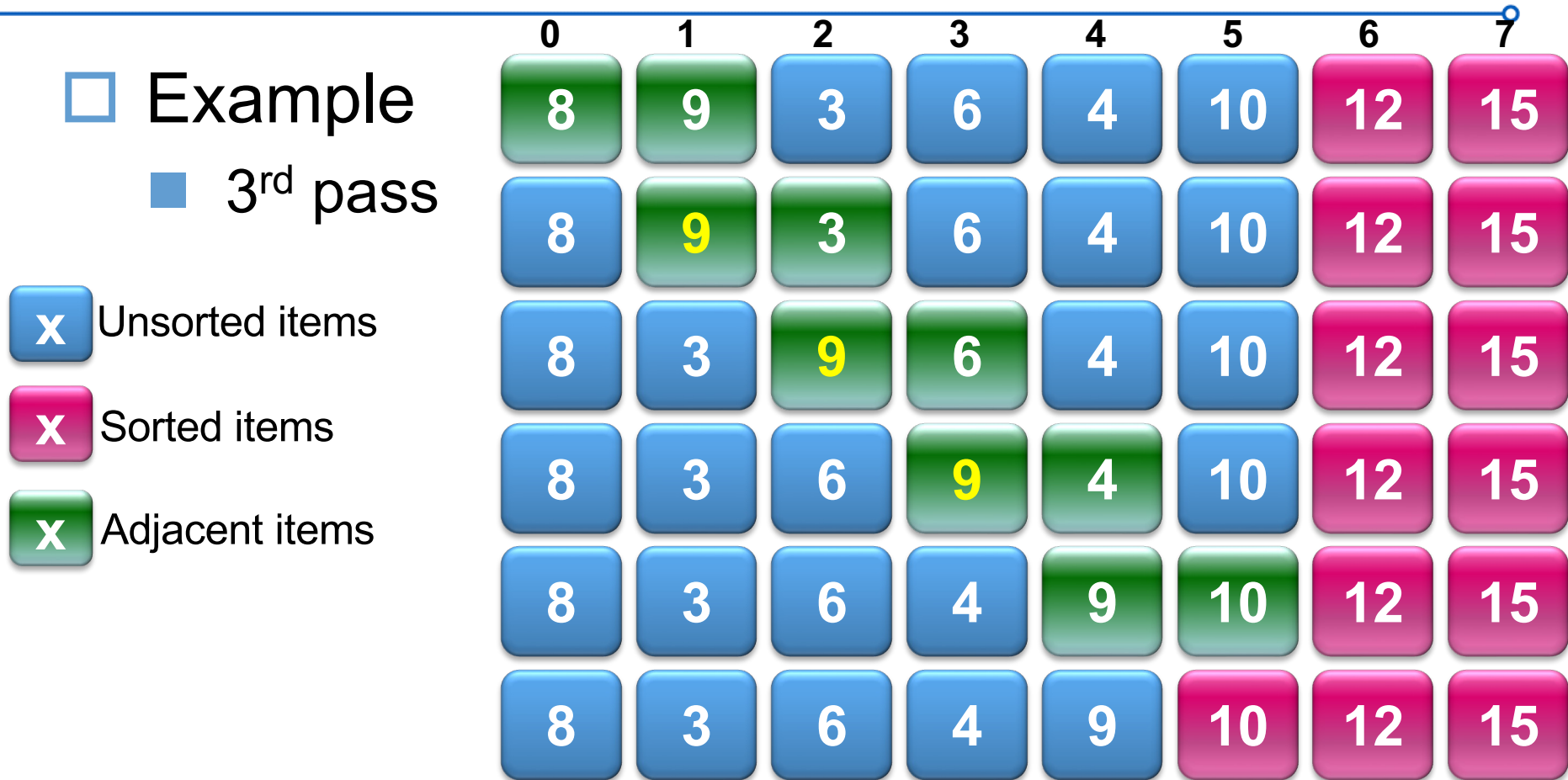
0	1	2	3	4	5	6	7
9	12	8	15	3	6	4	10
9	12	8	15	3	6	4	10
9	8	12	15	3	6	4	10
9	8	12	15	3	6	4	10
9	8	12	3	15	6	4	10
9	8	12	3	6	15	4	10
9	8	12	3	6	4	15	10
9	8	12	3	6	4	10	15



Bubble Sort - Illustration



Bubble Sort - Illustration



Bubble Sort - Illustration

□ Example

■ 4th pass

✕ Unsorted items

✕ Sorted items

✕ Adjacent items

0	1	2	3	4	5	6	7
8	3	6	4	9	10	12	15
3	8	6	4	9	10	12	15
3	6	8	4	9	10	12	15
3	6	4	8	9	10	12	15
3	6	4	8	9	10	12	15

Bubble Sort - Illustration

□ Example

■ 5th pass

✕ Unsorted items

✕ Sorted items

✕ Adjacent items

0	1	2	3	4	5	6	7
3	6	4	8	9	10	12	15
3	6	4	8	9	10	12	15
3	4	6	8	9	10	12	15
3	4	6	8	9	10	12	15

Bubble Sort - Illustration

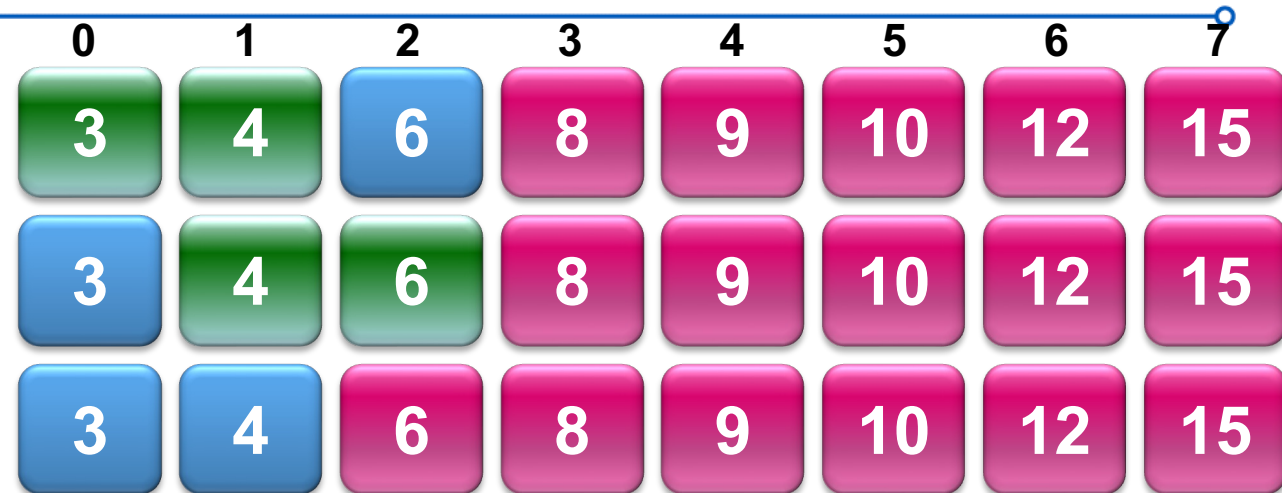
□ Example

■ 6th pass

✕ Unsorted items

✕ Sorted items

✕ Adjacent items



■ 7th pass

→ Done



Bubble Sort

BUBBLE-SORT(A,n) //Input: An array $A[0..n - 1]$ of orderable elements //Output: Array $A[0..n - 1]$ sorted in nondecreasing order	Cost times
1 for $i \leftarrow 0$ to $n - 2$ do 2 for $j \leftarrow 0$ to $n - 2 - i$ do 3 if $A[j+1] < A[j]$ 4 swap $A[j+1]$ and $A[j]$	n $n - 1$ $C(n)$ $C(n) - \alpha$

Basic operation



Bubble Sort Analysis

1. Input size: n
2. Basic operation: key comparison $A[j+1] < A[j]$
3. The number of key comparisons ***depends only on the array size***
4. Sum of number of times the basic operation is:

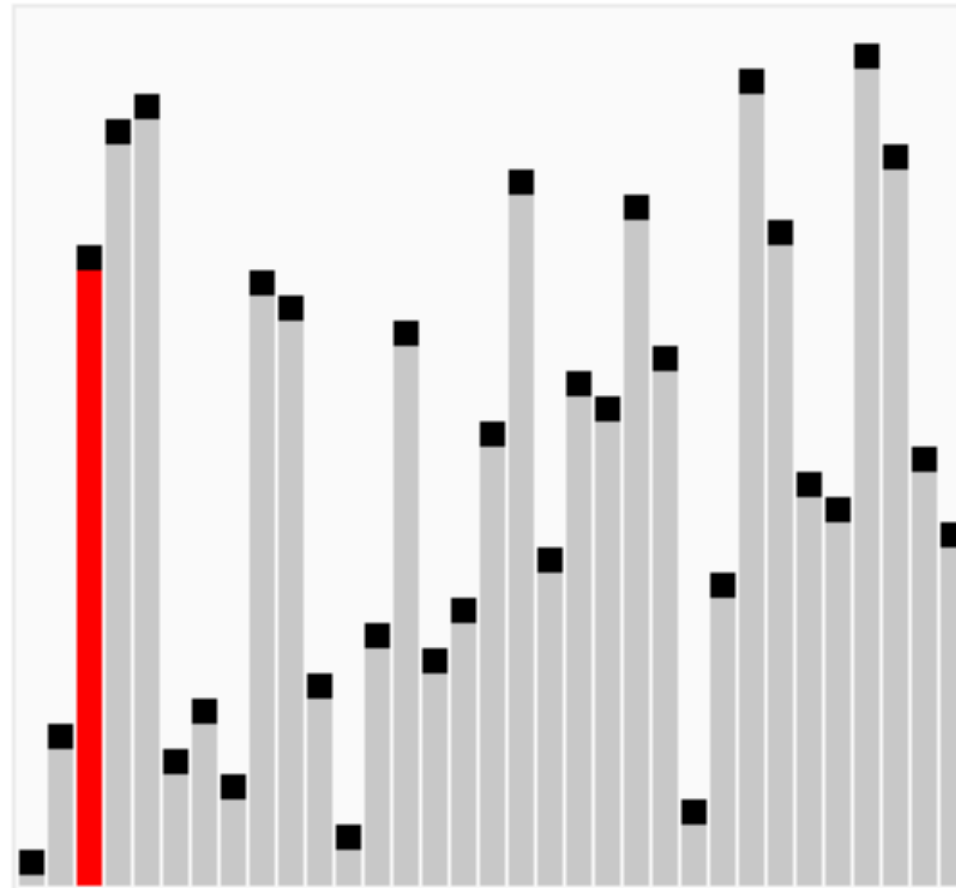
$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1] \\ &= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \end{aligned}$$

5. Order of growth: **$O(n^2)$**



Bubble Sort - Illustration

□ Visualization



[Click here to view animation](#)

Quiz

- Analyze the number of key swap operation of Selection Sort and Bubble Sort in term of Big-O

DECREASE-AND- CONQUER

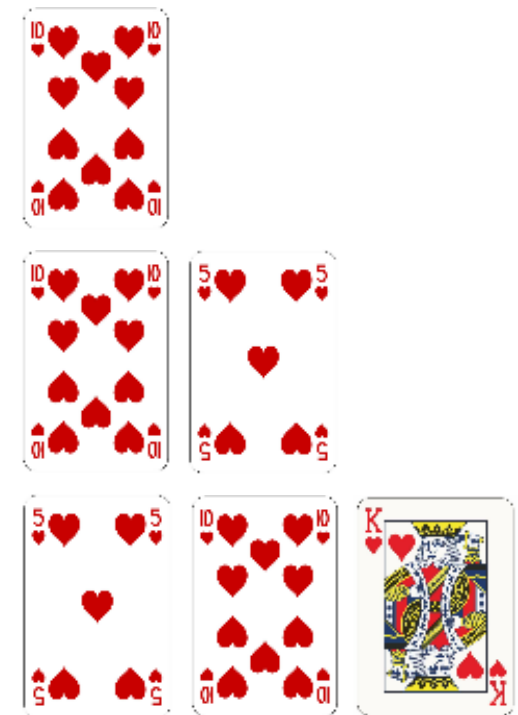
Insertion Sort

Decrease-and-Conquer Algorithms

- **Decrease-and-Conquer** technique is based on exploiting the relationship between a solution to a given instance of a problem and a solution to its **smaller instance**.
 - It can be exploited either top down or bottom up
 - **Top down** → *recursive implementation*
 - **Bottom-up** → *iterative implementation (incremental approach)*
- For example: Calculate a^n
 - Top down: $a^n = a^{n-1} \cdot a$ if $n > 0$; $a^n = 1$ if $n = 0$
 - Bottom up: Multiplying 1 by a n times (same as brute-force algorithm, but different thought process)

Insertion Sort - Idea

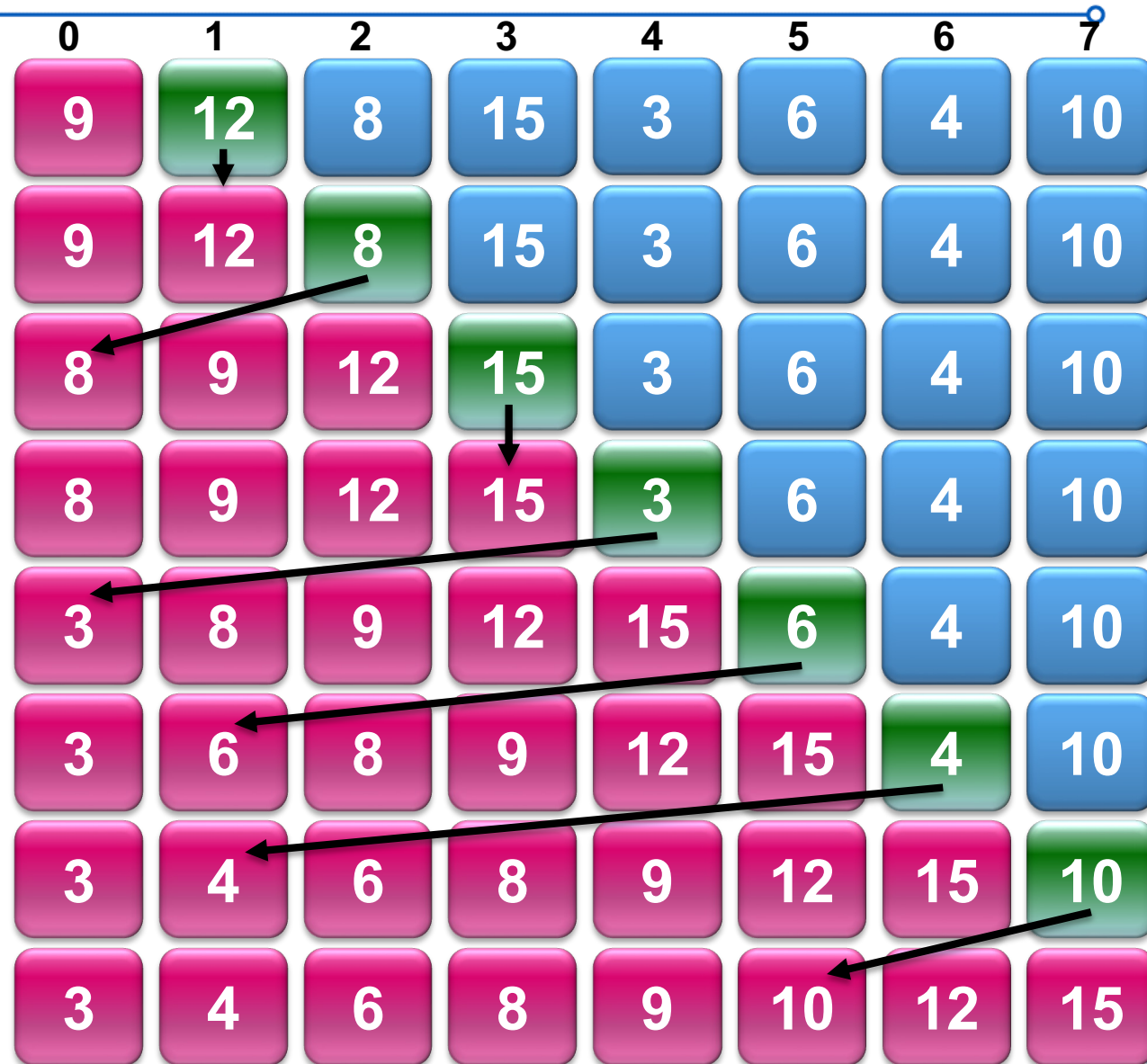
- Similar to how most people arrange a hand of poker cards
 - Start with one card in your hand
 - Pick the next card and insert it into its proper position to keep the sorted order.
 - Repeat the previous step for all cards.



Insertion Sort - Illustration

□ Example

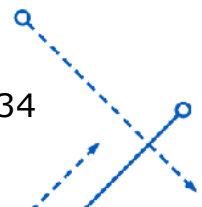
-  Unsorted items
-  Sorted items
-  Unsorted item to be inserted



Insertion Sort

INSERTION-SORT(A,n) //Input: An array $A[0..n - 1]$ of orderable elements //Output: Array $A[0..n - 1]$ sorted in nondecreasing order	Cost times
<pre> 1 for $i \leftarrow 1$ to $n - 1$ 2 $key \leftarrow A[i]$ 3 $j \leftarrow i - 1$ 4 while $j \geq 0$ and $A[j] > key$ do 5 $A[j+1] \leftarrow A[j]$ 6 $j \leftarrow j - 1$ 7 $A[j+1] \leftarrow key$ </pre>	n $n - 1$ $n - 1$ $C(n)$ $C(n) - \alpha$ $C(n) - \alpha$ $n - 1$

Basic operation

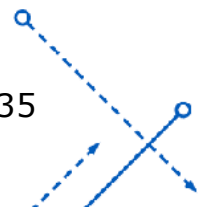


Insertion Sort Analysis

1. Input size: n
2. Basic operation: key comparison $A[j] > \text{key}$
3. The number of key comparisons ***depends on the nature of the input.***
 - In the **worst case**, $A[j] > \text{key}$ is executed the largest number of times, i.e., for every $j = i-1, \dots, 0$
 - This happens if and only if $A[j] > A[i]$ for $j=i-1, \dots, 0$
4. Sum of number of times the basic operation for the worst case is:

$$C_{\text{worst}}(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2}$$

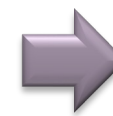
5. Order of growth: **$O(n^2)$**



Insertion Sort Analysis

- The number of key comparisons *depends on the nature of the input.*
 - In the **best case**, $A[j] > \text{key}$ is executed once for every $j = i-1, \dots, 0$
 - This happens if and only if $A[i-1] \leq A[i]$ for $i=1, \dots, n-1$
- Sum of number of times the basic operation for the best case is:

$$C_{best}(n) = \sum_{i=1}^{n-1} 1 = n - 1$$



Excellent efficiency
on sorted array

- Order of growth: $\mathbf{O(n)}$



Insertion Sort Analysis

- The number of key comparisons *depends on the nature of the input.*
 - In the **average case**, half of the elements are less than key, and half the elements are greater.
 - Therefore, key is compared with just half of the subarray $A[0, \dots, i-1]$
- Sum of number of times the basic operation for the average case is:

$$C_{avg}(n) \approx \frac{n^2}{4}$$

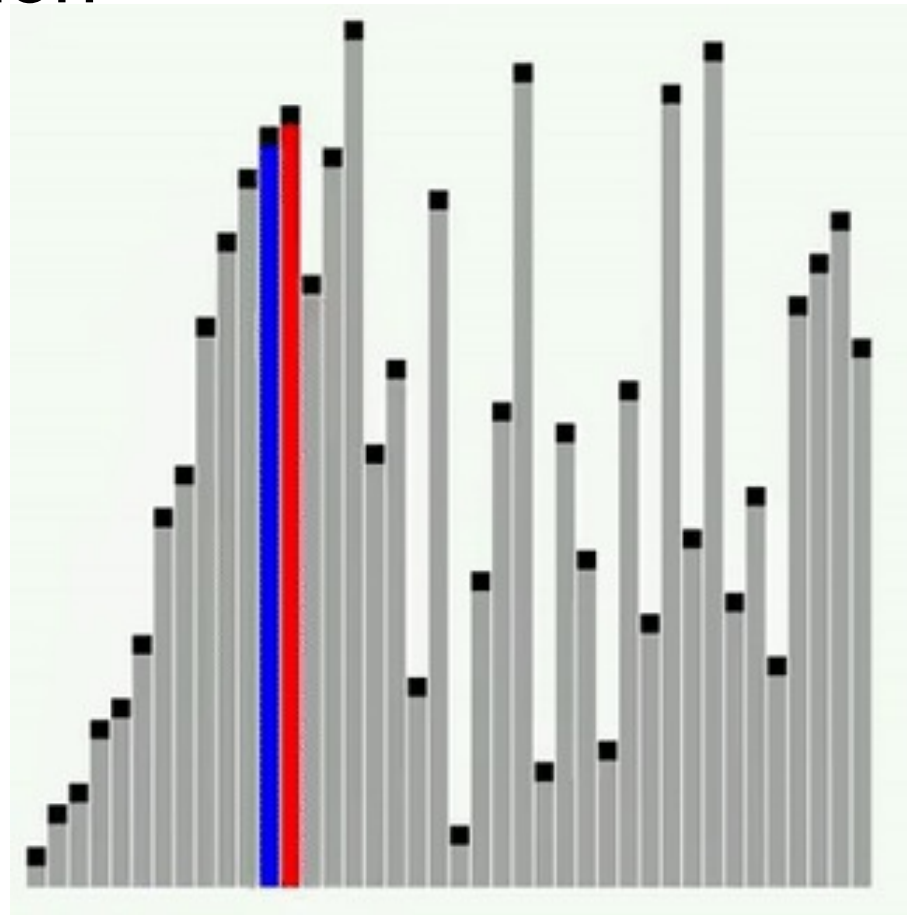
Twice-as-fast performance compared with worst-case

- Order of growth: **$O(n^2)$**



Insertion Sort - Illustration

□ Visualization



[Click here to view animation](#)

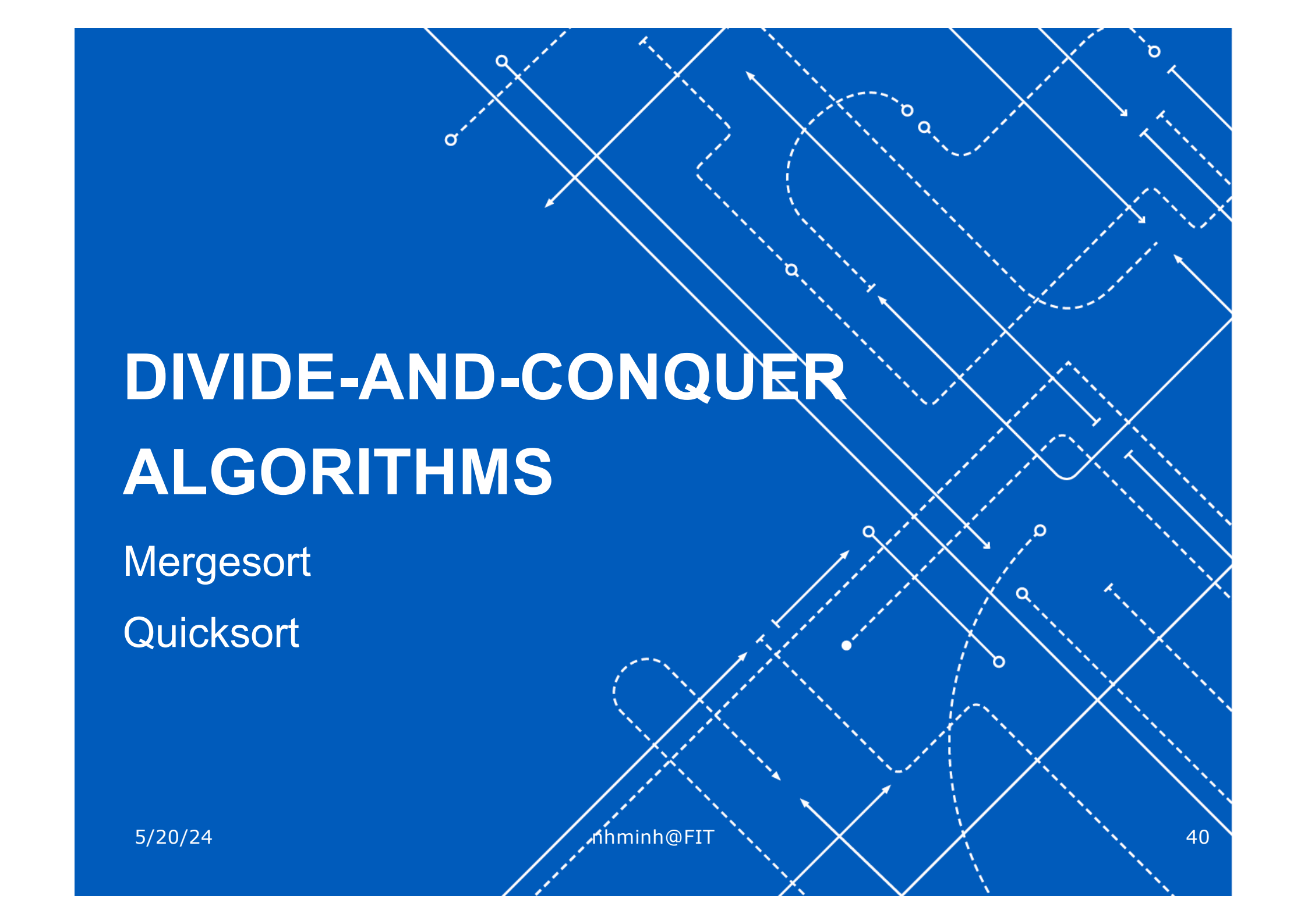
Quiz

1. Show that the average-case number of key comparisons in insertion sort is given by the formula

$$C_{avg}(n) \approx \frac{n^2}{4}$$

2. Apply insertion sort to sort the list E, X, A, M, P, L, E in alphabetical order



The background of the slide is a solid blue color. Overlaid on this background is a complex, abstract pattern of white lines and arrows. The pattern consists of numerous straight lines of varying lengths and orientations, some of which are dashed. Interspersed among these lines are several curved, wavy lines, also in white. Small white circles and dots are scattered throughout the pattern, some appearing at the ends of lines or as isolated points. The overall effect is a dynamic, geometric design that suggests movement and structure, fitting the theme of algorithms.

DIVIDE-AND-CONQUER ALGORITHMS

Mergesort

Quicksort

Divide-and-Conquer Algorithms

□ **Divide-and-Conquer** technique is the best-known general design technique:

1. A problem is divided into several **subproblems** of the same type, ideally of about **equal size**.
2. The subproblems are solved **recursively**.
3. If necessary, the solutions to the subproblems are **combined** to get a solution to the original problem.

→ *Ideally suited for parallel computations, in which each subproblem can be solved simultaneously by its own processor.*

□ For example: Calculate a^n

$$\rightarrow a^n = \begin{cases} a^{n/2} * a^{n/2} & \text{if } n \text{ is even} \\ a^{(n-1)/2} * a^{(n-1)/2} * a & \text{if } n \text{ is odd} \end{cases}$$

Divide-and-Conquer Analysis

- In the most typical case of divide-and-conquer a problem's instance of size n is divided into two instances of size $n/2$.
- More generally, an instance of size n can be divided into b instances of size n/b , with a of them needing to be solved.
 - a and b are constants $a \geq 1, b \geq 1$
 - To simplify, assume that n is a power of b
 - $f(n)$: the time spent on dividing an instance of size n into instances of size n/b and combining the solutions
- **General divide-and-conquer recurrence:**

$$T(n) = aT(n/b) + f(n)$$



Divide-and-Conquer Analysis

□ Master Theorem:

■ If $f(n) \in O(n^d)$ where $d \geq 0$

$$T(n) \in \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

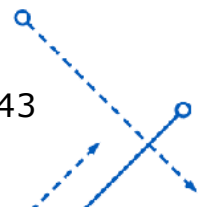
■ This result holds for the Ω and Θ , too.

□ For example, in calculating a^n :

■ Recurrence: $M(n) = M(n/2) + 1$

■ $a = 1$, $b = 2$ and $f(n) = 1$, $d = 0 \rightarrow a = b^d$

$\rightarrow M(n) \in O(n^d \log n) = O(\log n)$



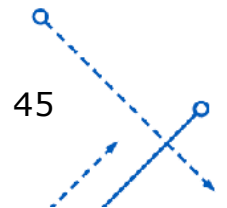
Mergesort - Idea

- Suppose we only know how to merge two sorted sets of items into one
 - Merge $\{1, 5, 7\}$ with $\{2, 9\} \rightarrow \{1, 2, 5, 7, 9\}$
- But where do we get the two sorted sets in the first place?
- Idea:
 - Merge each pair of items into sets of 2
 - Merge each pair of sets of 2 into sets of 4
 - Repeat previous step for sets of 4
 - Final step: merge two sets of $n/2$ items to obtain a fully sorted set



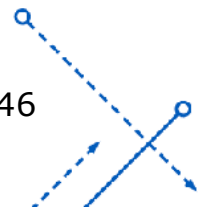
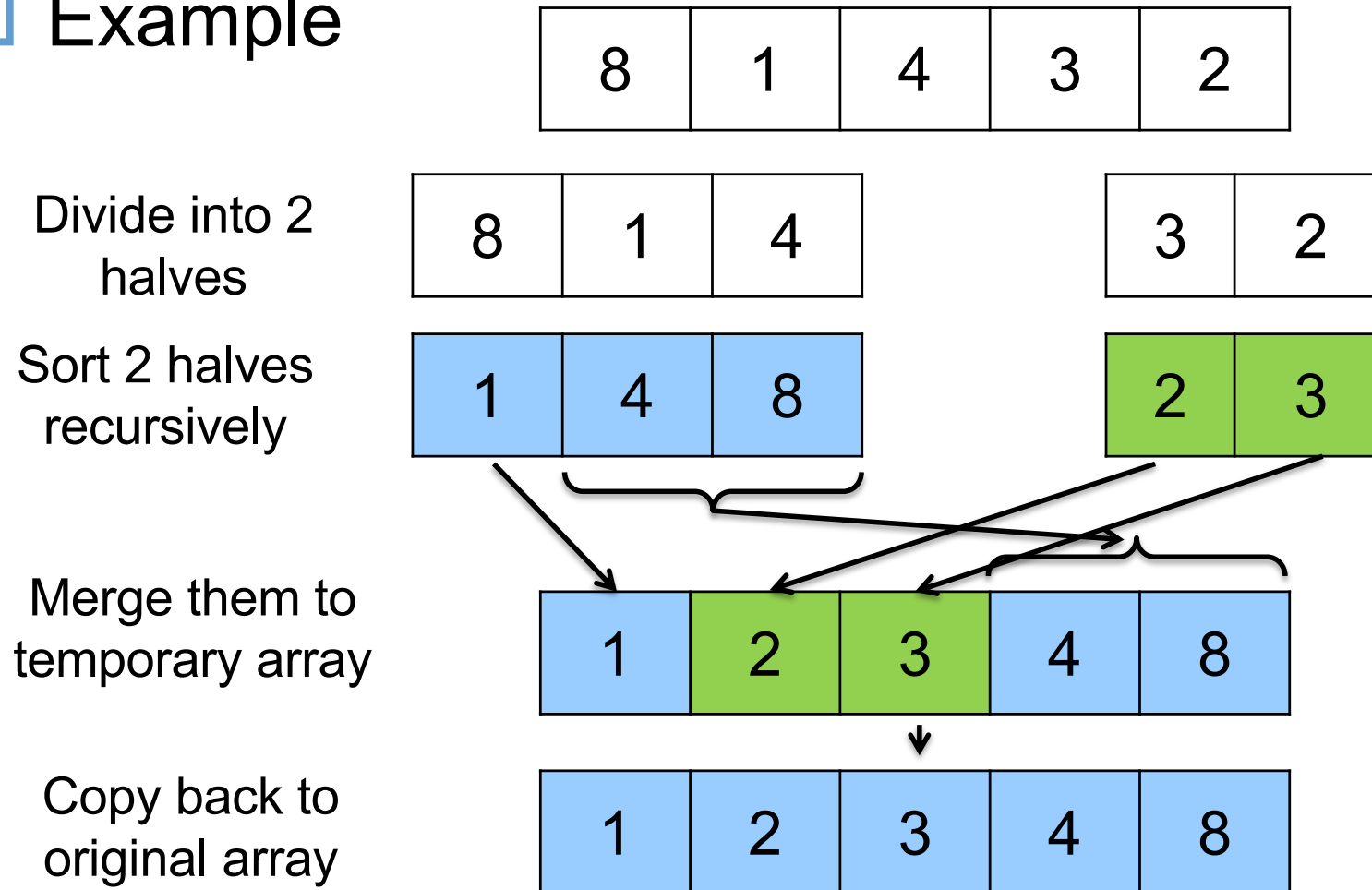
Mergesort – Divide-and-conquer

- Merge sort is a divide and conquer sorting algorithm
- Divide step:
 - Divide the array into two (equal) halves
 - Recursively sort the two halves
- Conquer step:
 - Merge the two halves to form a sorted array

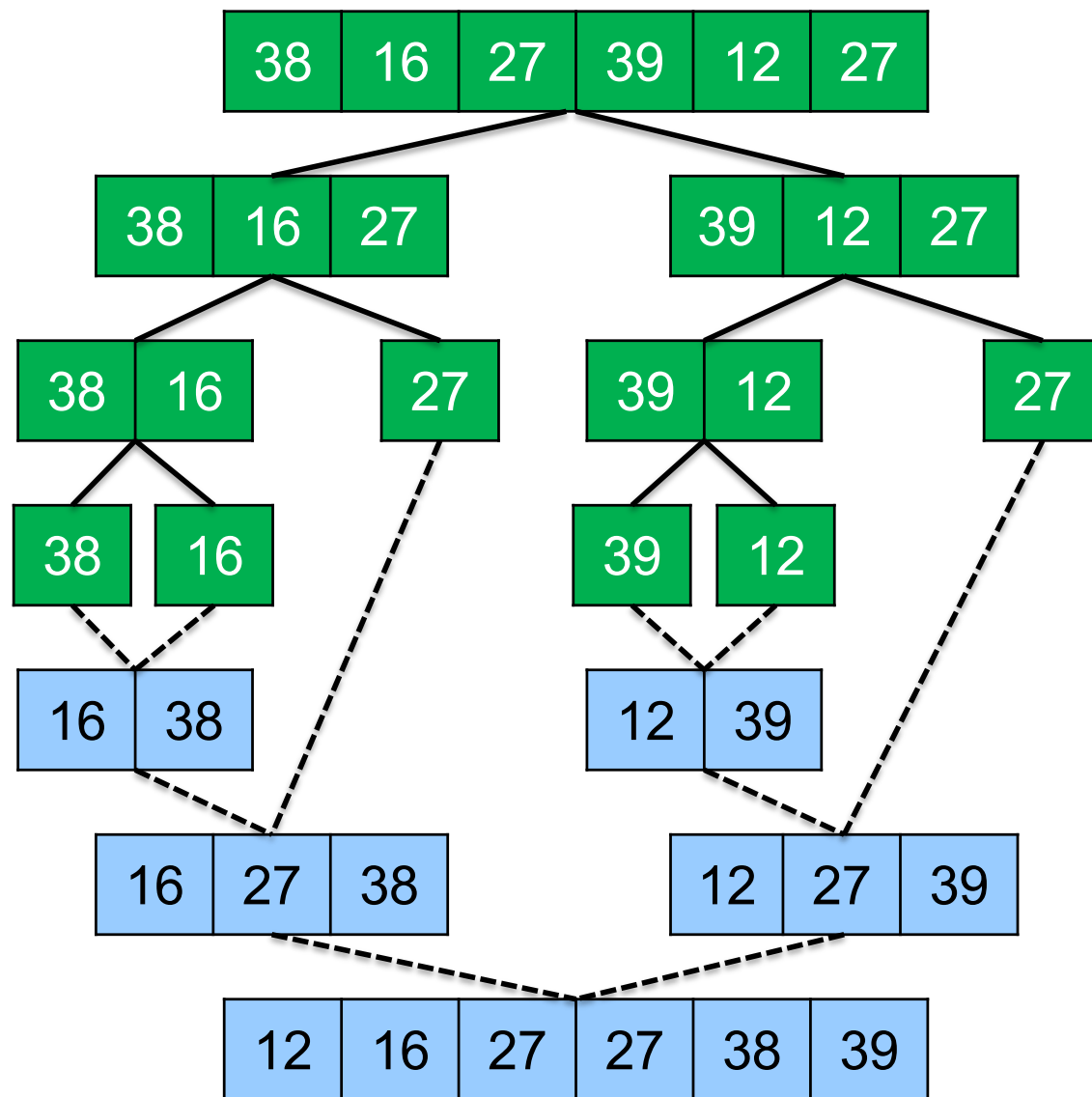


Mergesort - Illustration

□ Example



Mergesort - Illustration

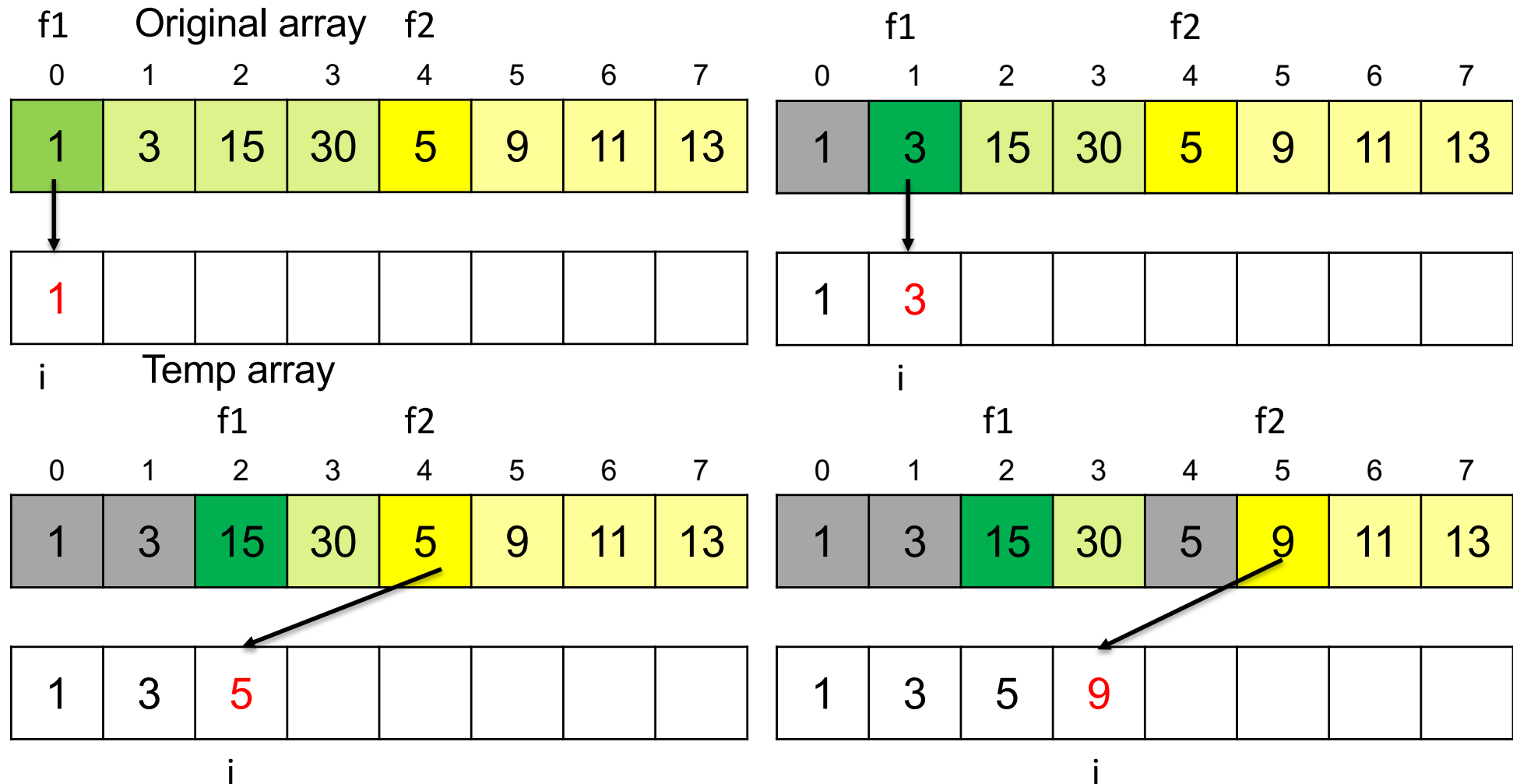


Recursive calls to mergesort

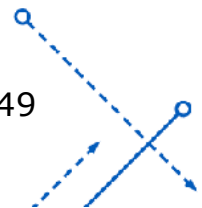
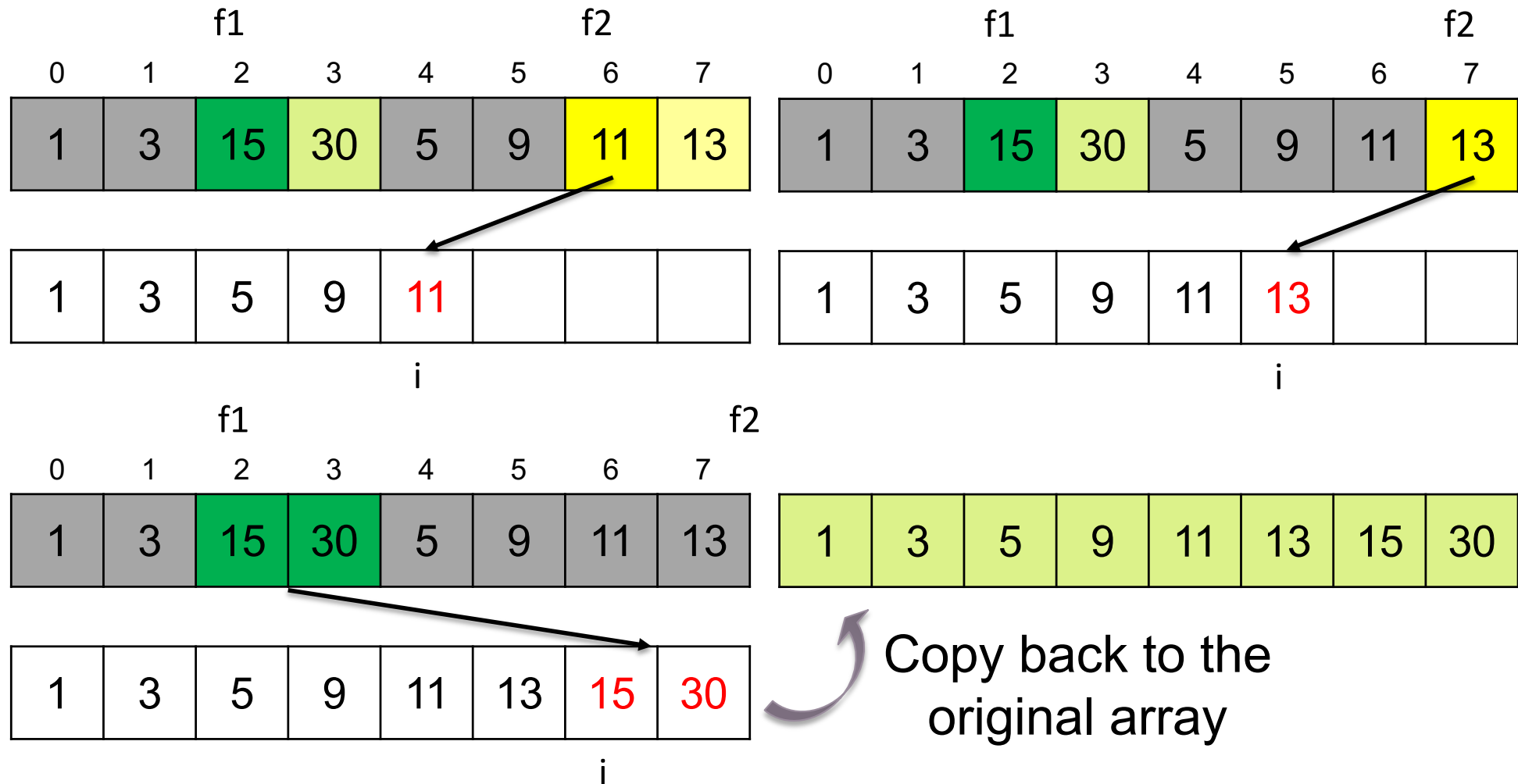
Merge steps



Mergesort – Merge Steps



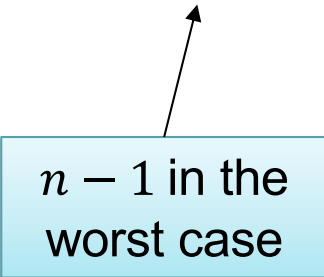
Mergesort – Merge Steps



Mergesort – Algorithm

MERGESORT(A,n) //Input: An array $A[0..n - 1]$ of orderable elements //Output: Array $A[0..n - 1]$ sorted in nondecreasing order	Cost times
1 if $n > 1$ 2 copy $A[0..n/2 - 1]$ to $B[0..n/2 - 1]$ 3 copy $A[n/2..n]$ to $C[0..n/2 - 1]$ 4 Mergesort ($B, n/2$) 5 Mergesort ($C, n/2$) 6 Merge (B, C, A)	 $C(n/2)$ $C(n/2)$ $C_{merge}(n)$

Mergesort – Merging step

MERGE(B,C,A) //Input: Array B[0..p – 1] and C[0..q – 1] both sorted //Output: Array A[0..p + q – 1] sorted in nondecreasing order	Cost times
<pre> 1 i ← 0; j ← 0; k ← 0 2 while i < p and j < q do 3 if B[i] ≤ C[j] 4 A[k] ← B[i]; i ← i + 1 5 else A[k] ← C[j]; j ← j + 1 6 k ← k + 1 7 if i = p 8 copy C[j..q – 1] to A[k..p + q – 1] 9 else copy B[i..p – 1] to A[k..p + q – 1] </pre>	$C_{merge}(n)$ 

Mergesort Analysis

1. Input size: n
2. Basic operations: 2 Mergesort + 1 Merge
3. Sum of number of times the basic operations is:
 - $C(n) = 2C\left(\frac{n}{2}\right) + C_{merge}(n)$ for $n > 1$
 - $C(1) = 0$
4. The number of key comparisons in merging step ***depends on the nature of 2 subarrays***
 - In the worst case: $C_{merge}(n) = n - 1$
 - Thus, $C_{worst}(n) = 2C_{worst}\left(\frac{n}{2}\right) + n - 1$ for $n > 1$, $C_{worst}(1) = 0$
 - According to Master Theorem, $C_{worst}(n) \in O(n \log n)$
 - In fact: $C_{worst}(n) = n \log_2 n - n + 1$ for $n = 2^k$
5. Order of growth: ***$O(n \log_2 n)$***



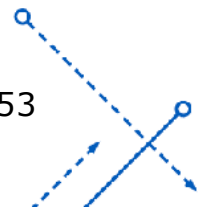
Mergesort – Pros and Cons

□ Pros:

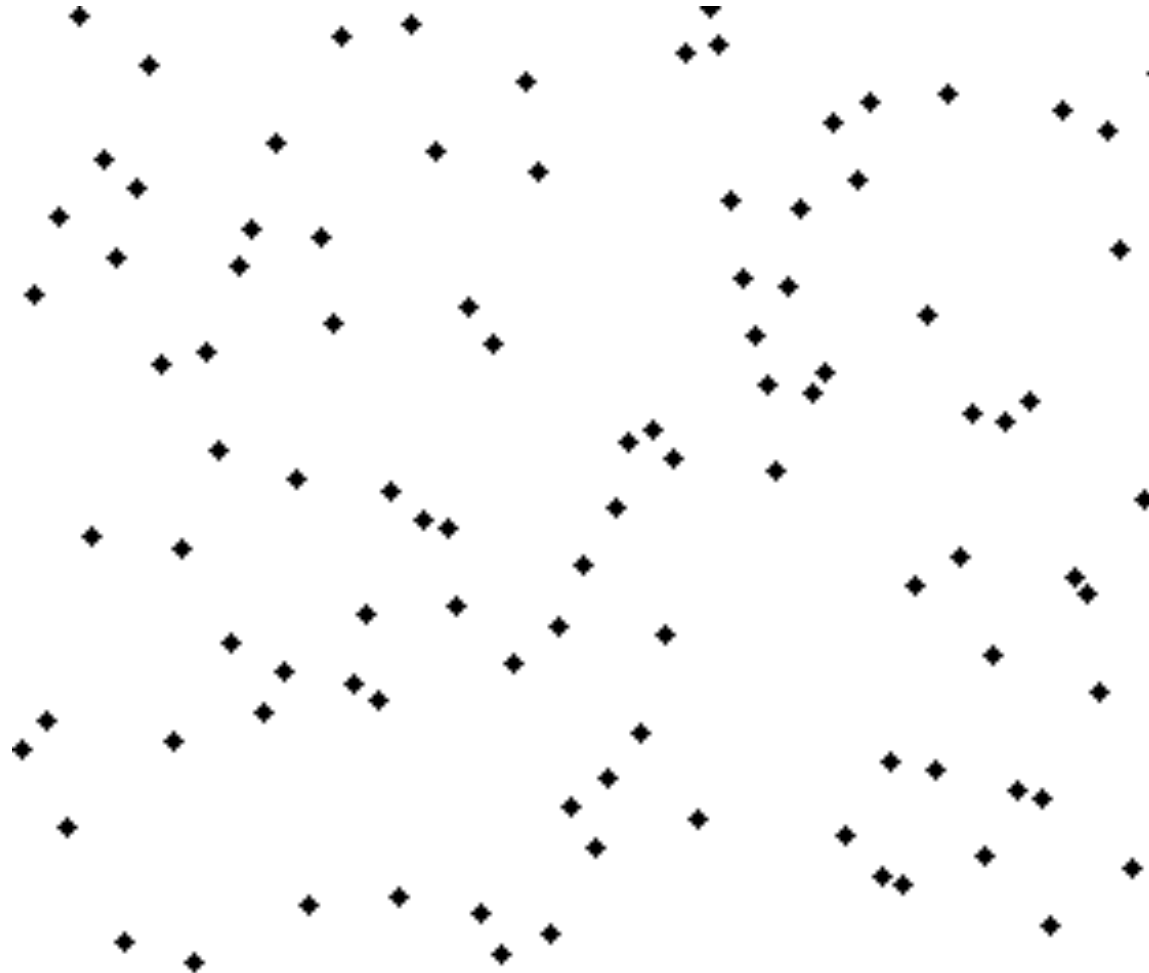
- The performance is guaranteed (unaffected by original ordering of the input)
- Stable sort
- Suitable for extremely large number of inputs
 - Can operate on the input portion by portion

□ Cons:

- Not easy to implement
- Require additional storage during merging operation



Merge Sort – Visualization



[Click here to view animation](#)



Quicksort – Idea

- Quicksort is a divide-and-conquer sorting algorithm
 - Divide its input elements according to their values.

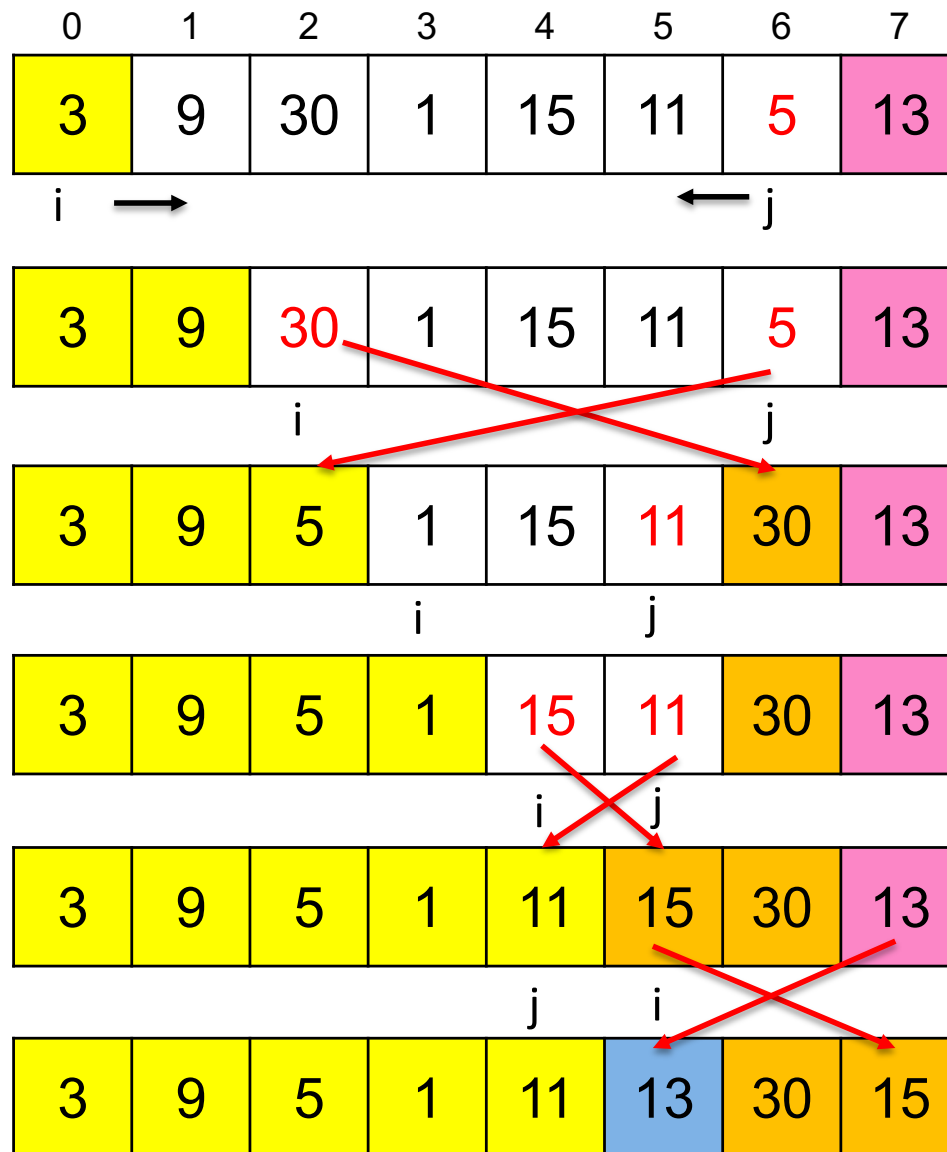


- Divide step:
 - Choose an item $A[p]$ (known as **pivot**) and partition the items of $A[i \dots j]$ into two parts
 - Items that are smaller than $A[p]$
 - Item that are greater than or equal to $A[p]$
 - Put $A[p]$ to the correct position (in the final sorted array)
 - Recursively sort the two parts $A[i \dots p-1]$ and $A[p+1 \dots j]$
- Conquer step:
 - Do nothing!

Quicksort – Idea

- How to choose pivot:
 - The first element $A[0]$
 - The last element $A[j-1]$
 - The middle element $A[(j-1)/2]$
 - Randomly
 - ...

Quicksort - Illustration



An inversion occurs:

- $A[j] < \text{pivot}$
 - $A[i] > \text{pivot}$
- Swap $A[i], A[j]$

X	$A[i] \leq \text{pivot}$
X	$A[j] > \text{pivot}$
X	Pivot
X	Sorted item

Finally, swap $A[i]$ and pivot
 → *pivot is at correct position*



0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

$$i \longrightarrow \quad \longleftarrow j$$

3	9	5	1	11	13	30	15
---	---	---	---	----	----	----	----

1	9	5	3	11	13	30	15
---	---	---	---	----	----	----	----

1	3	5	9	11	13	30	15
---	---	---	---	----	----	----	----

1	3	5	9	11	13	30	15
---	---	---	---	----	----	----	----

1	3	5	9	11	13	30	15
---	---	---	---	----	----	----	----

1	3	5	9	11	13	15	30
---	---	---	---	----	----	----	----

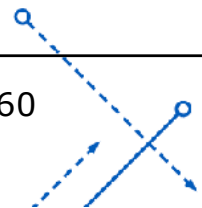


Quicksort – Algorithm

QUICKSORT(A,l,r) //Sort a subarray by quicksort //Input: Subarray of A[0..n – 1] defined by its left and right //indices l and r //Output: Subarray A[l..r] sorted in nondecreasing order	Cost times
1 if $l < r$ 2 $p \leftarrow$ Partition(A,l,r) 3 Quicksort(A,l,p-1) 4 Quicksort(A,p+1,r)	$C_{part}(n)$ $C(p)$ $C(n - 1 - p)$

Quicksort –Partition

Partition(A,l,r) //Partition a subarray using the last element as a pivot //Input: Subarray of A[0..n – 1] defined by its left and right //indices l and r (l<r) //Output: Partition of A[l..r] with the split position returned //as this function's value	Cost times
<pre> 1 p ← r 2 r ← r - 1 3 while l ≤ r do 4 while A[l] < A[p] and l ≤ r do l ← l + 1 6 while A[r] > A[p] and l ≤ r do r ← r - 1 8 if l ≤ r 9 swap A[l] with A[r] 10 l ← l + 1 and r ← r - 1 11 swap A[l] with p 12 return l </pre>	$n - 1$



Quicksort Analysis

1. Input size: n
2. Basic operations: 2 Quicksort + 1 Partition
3. The number of:
 - Key comparisons in partition step ***depends only on n***
(in fact, $n - 1$)
 - Quicksort calls depends on p
4. The sum of basic operations are calculated for 3 cases:
 - **Best case:** all the splits happen in the middle of corresponding subarrays
 - **Worst case:** 1 of the 2 subarrays is empty
 - **Average case:** p can be any position with the same probability $1/n$



Quicksort Analysis

□ Best case:

$$C_{best}(n) = 2C_{best}(n/2) + n - 1 \text{ for } n > 1, C_{best}(1) = 0$$

$$\in \mathbf{O(n \log_2 n)} \text{ by Master Theorem}$$

□ Worst case:

$$C_{worst}(n) = n - 1 + n - 2 + \dots + 1 = \frac{(n - 1)n}{2} \in \mathbf{O(n^2)}$$

□ Average case:

$$C_{avg}(n) = \frac{1}{n} \sum_{p=0}^{n-1} [n - 1 + C_{avg}(p) + C_{avg}(n - 1 - p)] \text{ for } n > 1,$$

$$C_{avg}(0) = 0, C_{avg}(1) = 0$$

$$C_{avg}(n) \approx 1.39n \log_2 n \in \mathbf{O(n \log_2 n)}$$



Quicksort – Pros and Cons

□ Pros:

- The performance in average case is far better than its worst case
 - Only 39% more comparisons than in the best case
- In most situation, quick sort is better than merge sort
 - This is because its innermost loop is so efficient

□ Cons:

- Not stable
- In the worst case, quick sort is significant slower than merge sort.

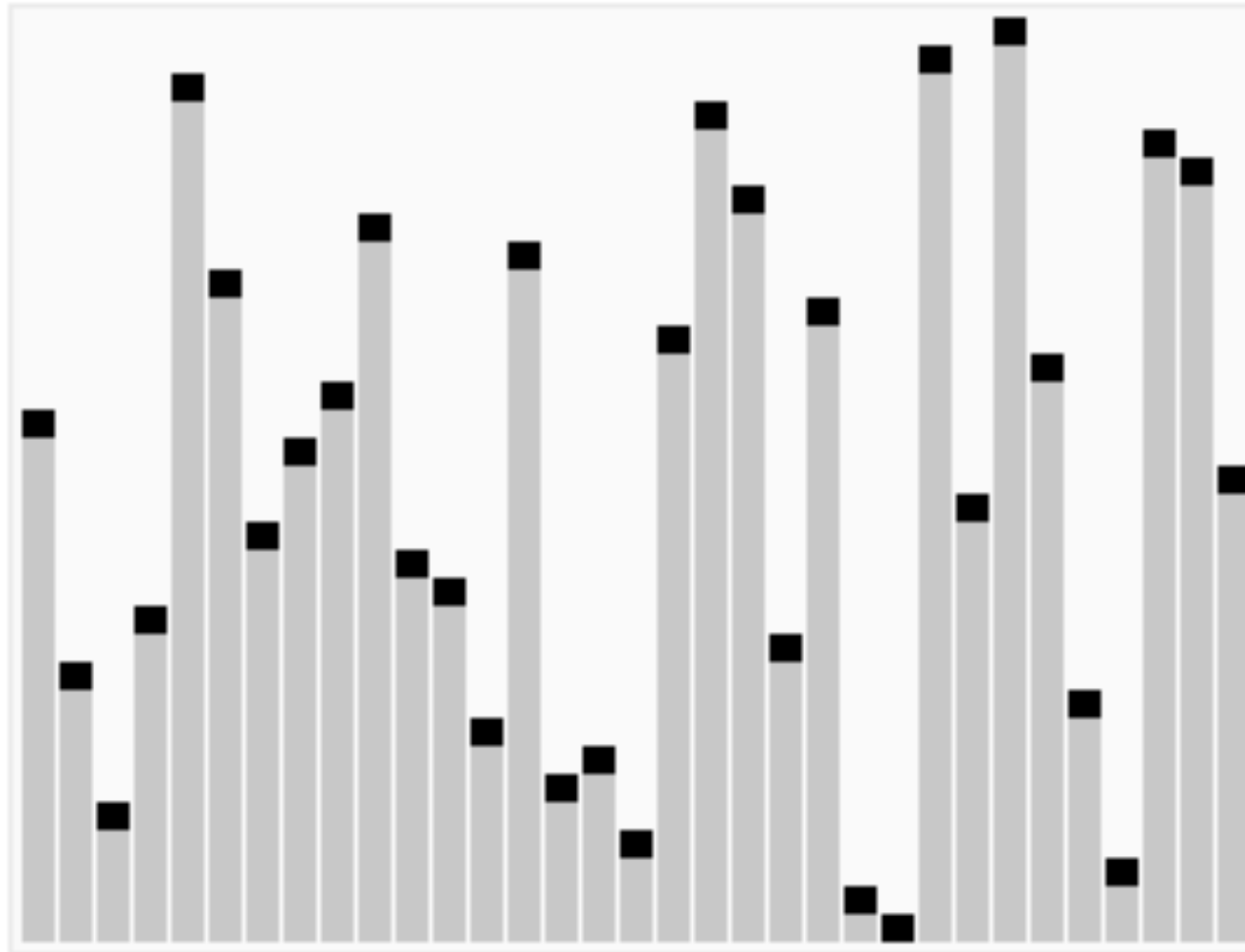
Quicksort – Improvements

- Better **pivot selection** methods:
 - Randomized quicksort
 - Median-of-three
- Switching to **insertion sort** on very small subarrays (e.g., <15 elements)
- Modifications of **the partitioning algorithm** such as the three-way partition:
 - Smaller than pivot
 - Equal to pivot
 - Larger than pivot

Cut the running time of the algorithm by 20% - 30%



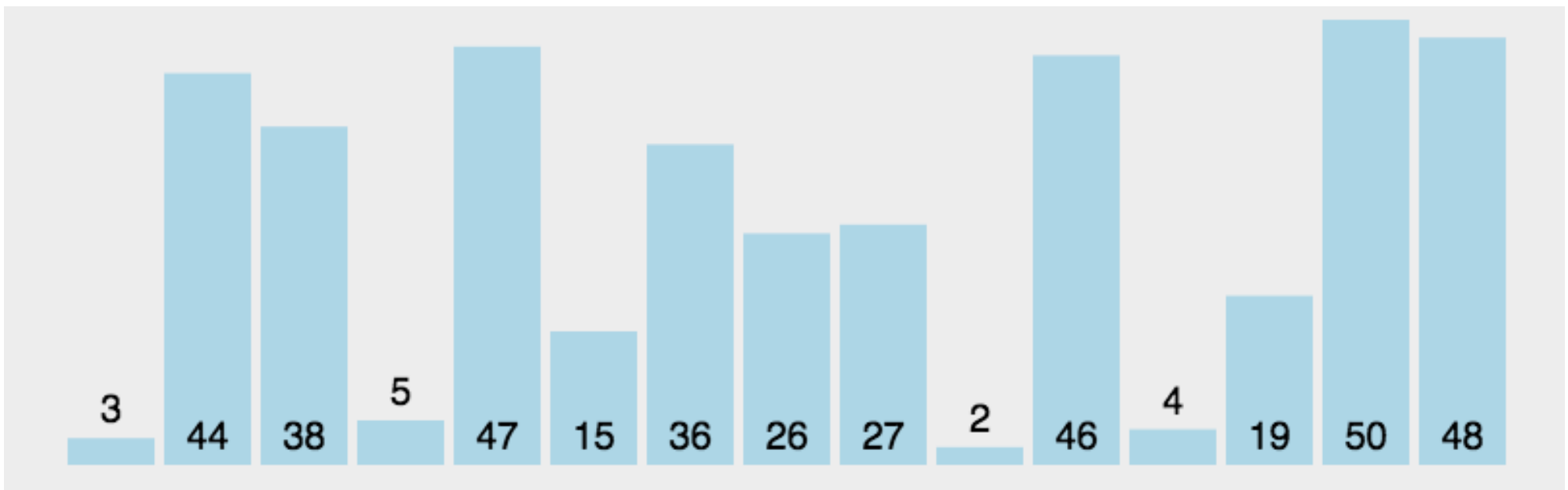
Quicksort – Visualization



[Click here to view animation](#)

Quicksort – Visualization

- Another version of Quick Sort:
 - Pivot it the first element
 - Different way to partition 2 sub arrays at each step



[Click here to view animation](#)

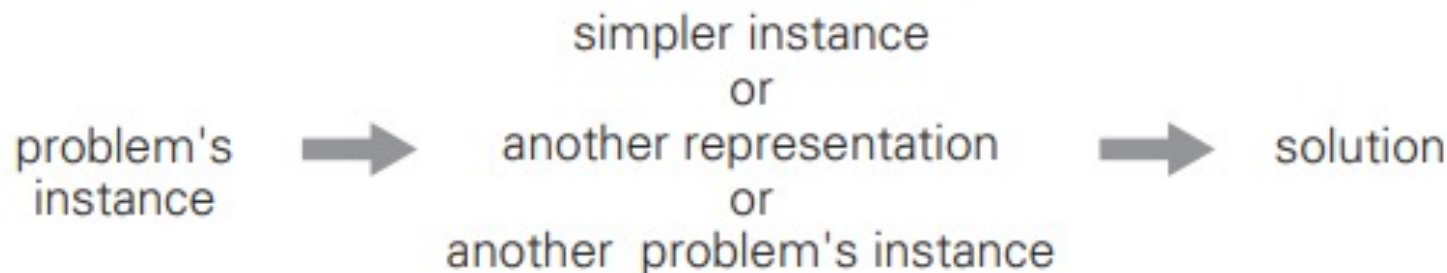
TRANSFORM-AND- CONQUER ALGORITHM

Heapsort

Transform-and-Conquer Algorithms

□ **Transform-and-Conquer** technique is based on 2 steps:

1. The problem's instance is modified to be more amenable to solution
2. Solve it



□ Example: Heapsort algorithm

- Transfer the original array to a heap
- Apply heapsort to solve it easily

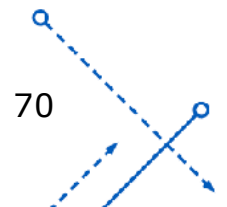
Heapsort – Heap Data Structure

- “Heap” is an array object that can be viewed as a **complete binary tree**.
 - Each **node** of the tree corresponds to **an element** of the array that stores the value in the node.
 - The tree is **completely filled** on all levels except possibly the lowest, which is filled from the left up to a point.
- Two kinds of binary heaps:
 - **Max-heaps:**
 - Max-heap property: for every node i other than the root, $A[\text{Parent}(i)] \geq A[i]$
 - The largest element is?
 - **Min-heaps:**
 - Min-heap property?



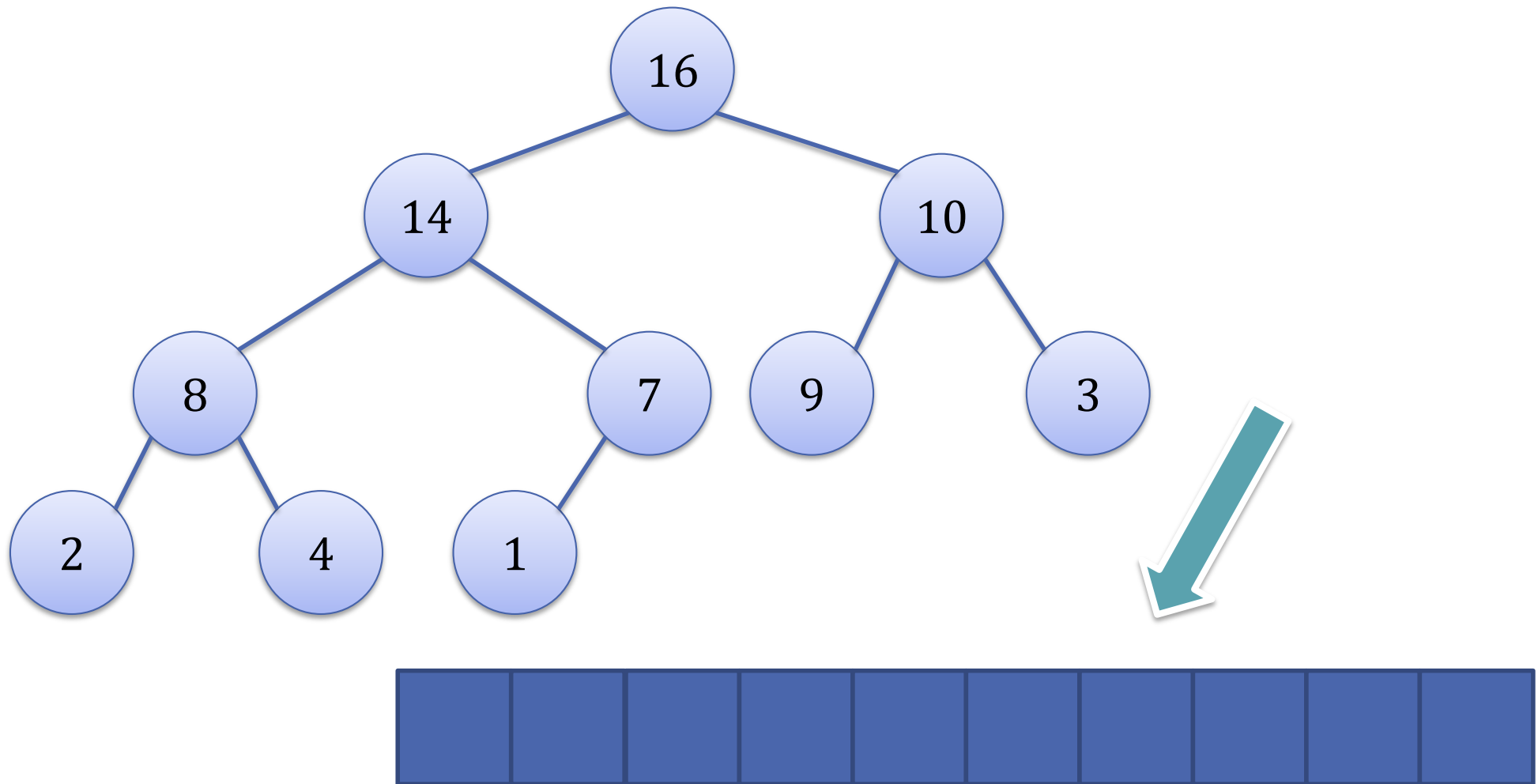
Heapsort – Heap Data Structure

- Height of a **node**:
 - The number of edges on the longest simple downward path from the node to a leaf
- Height of a **heap**:
 - The height of its root
 - The height of a heap of n -elements is $\lceil \log_2 n \rceil$
- Basic operations on heaps:
 - **MAX-HEAPIFY**
 - **BUILD-MAX-HEAP**
 - **HEAPSORT**



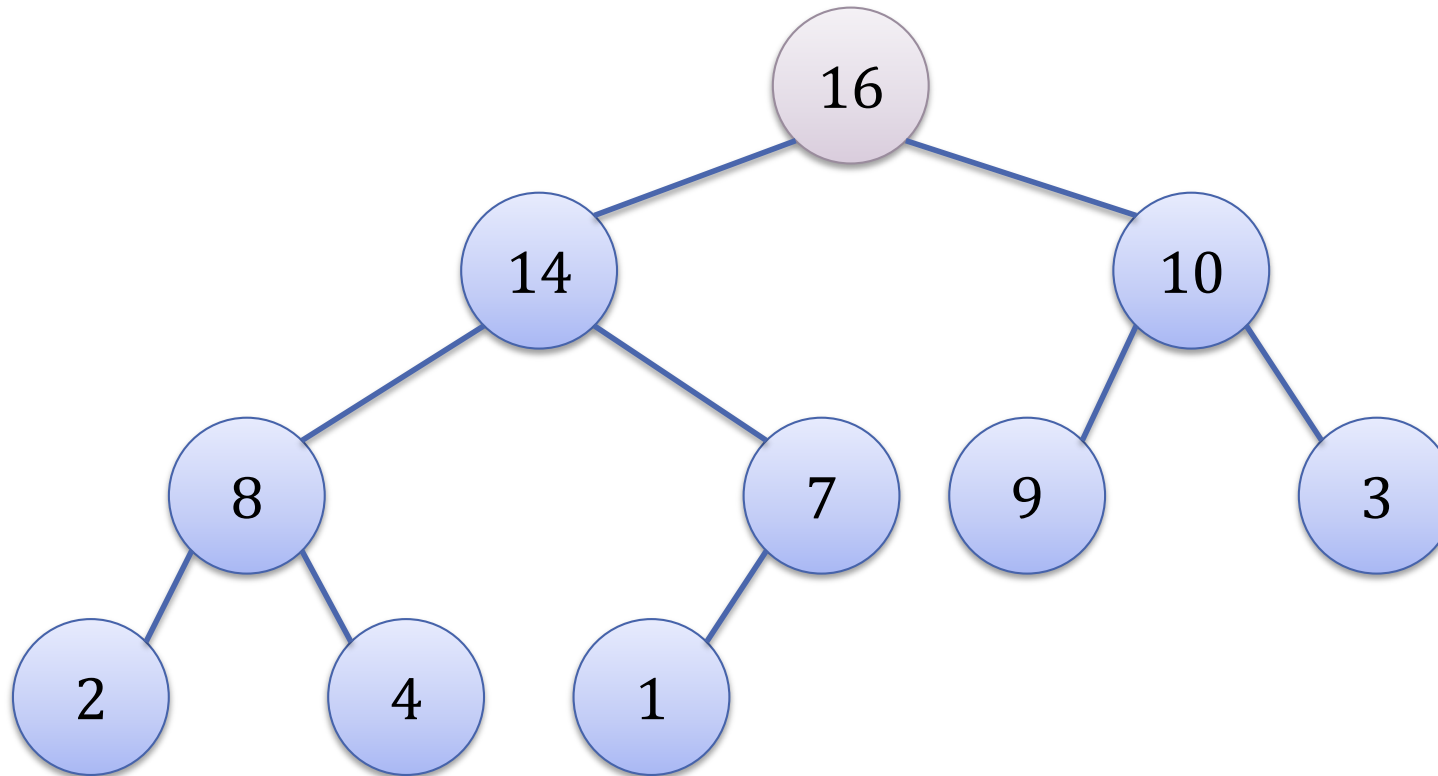
Heap and its array representation

- Save node values in array

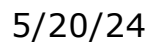


Heap and its array representation

- Root node at the first place

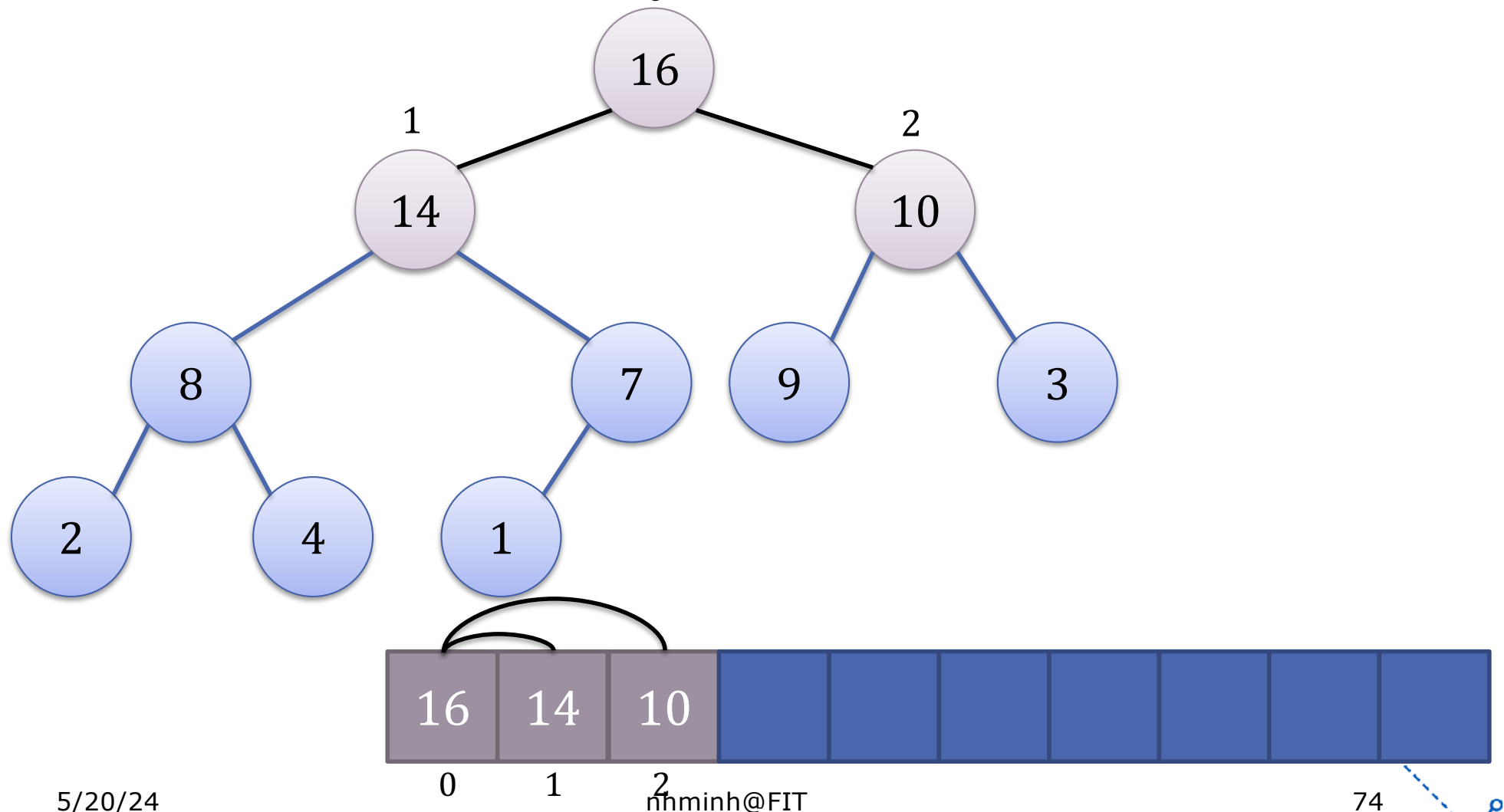


-
- ```
graph TD; 16((16)) --- 14((14)); 16 --- 10((10)); 14 --- 8((8)); 14 --- 7((7)); 8 --- 2((2)); 8 --- 4((4)); 7 --- 1((1)); 10 --- 9((9)); 10 --- 3((3));
```
- A binary tree diagram illustrating a search for the value 1. The root node is 16. Its left child is 14, and its right child is 10. Node 14 has a left child 8 and a right child 7. Node 8 has left child 2 and right child 4. Node 7 has a left child 1. Node 10 has a left child 9 and a right child 3. The path from 16 to 14 is highlighted with a thick black line, and the number 1 is placed above node 14.



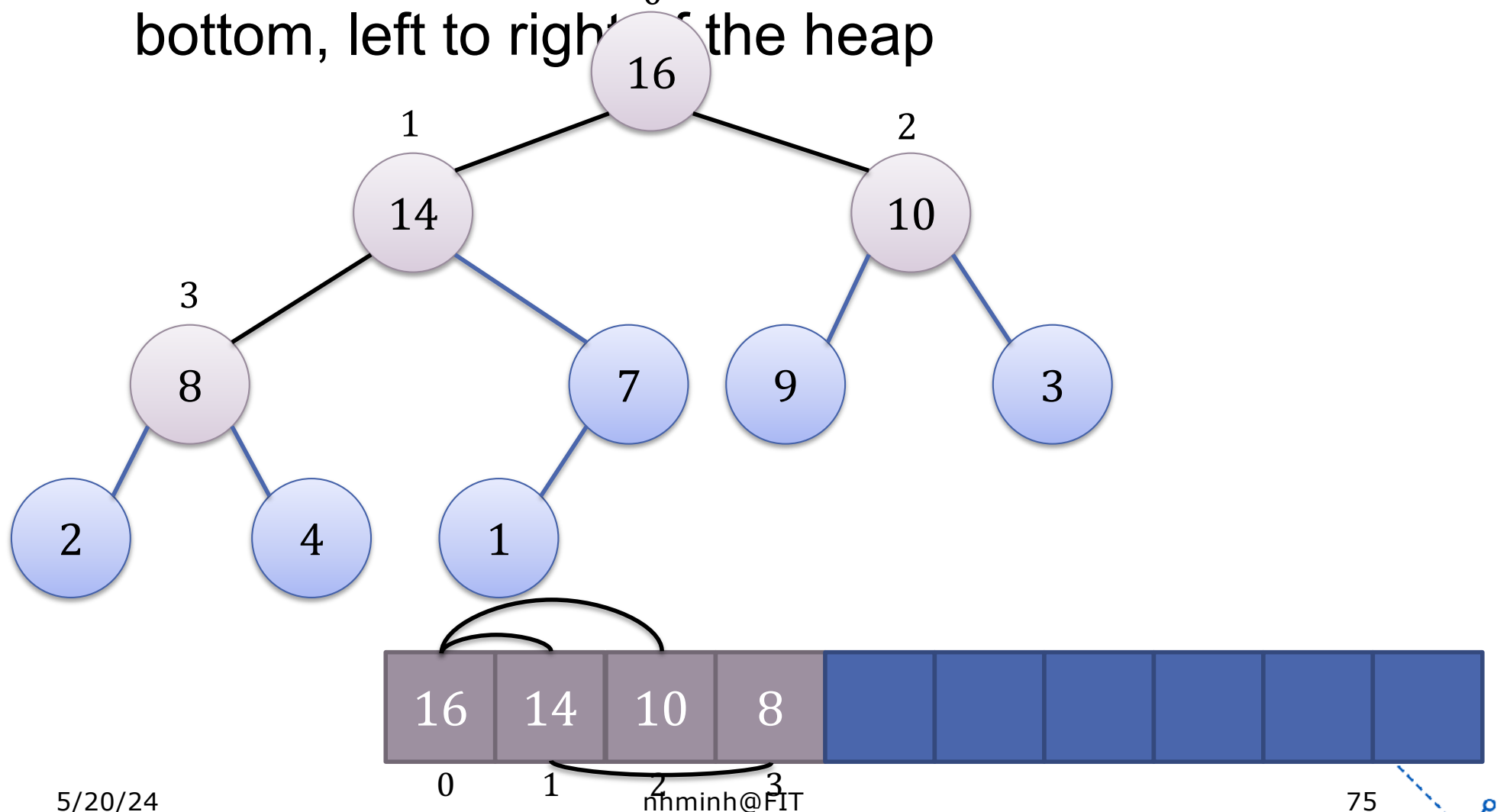
# Heap and its array representation

- The right child of root node is next

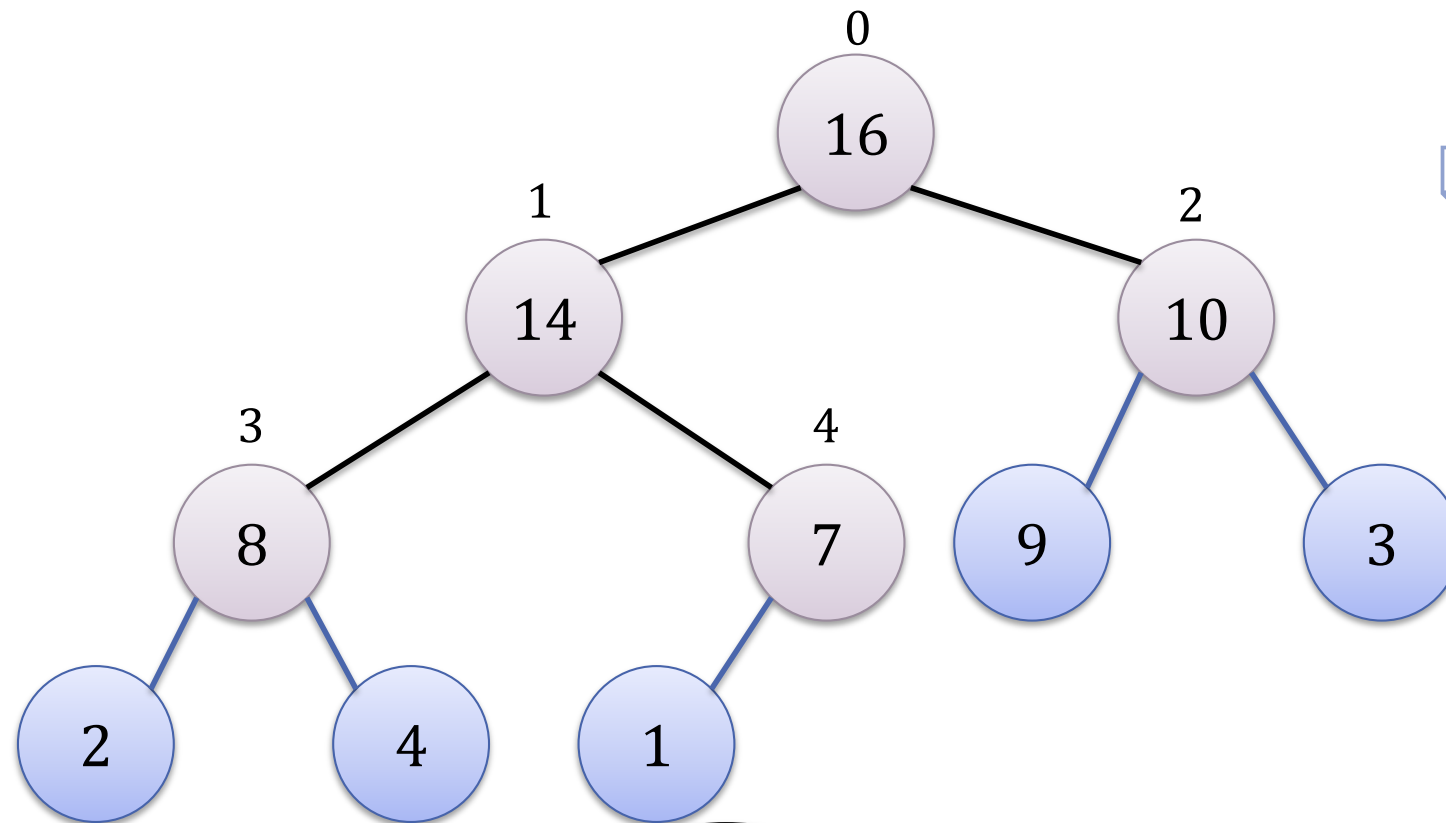


# Heap and its array representation

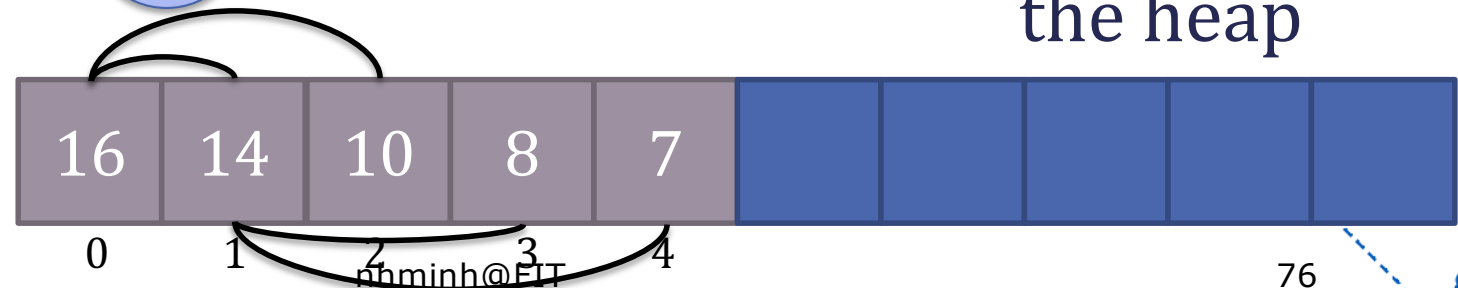
- Continue to fill the array with the values from top to bottom, left to right of the heap



# Heap and its array representation

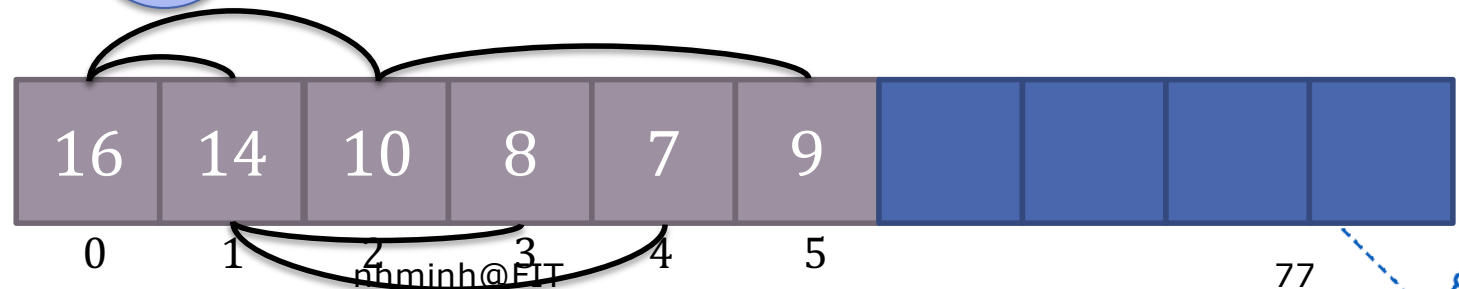


□ Continue to fill the array with the values from top to bottom, left to right of the heap

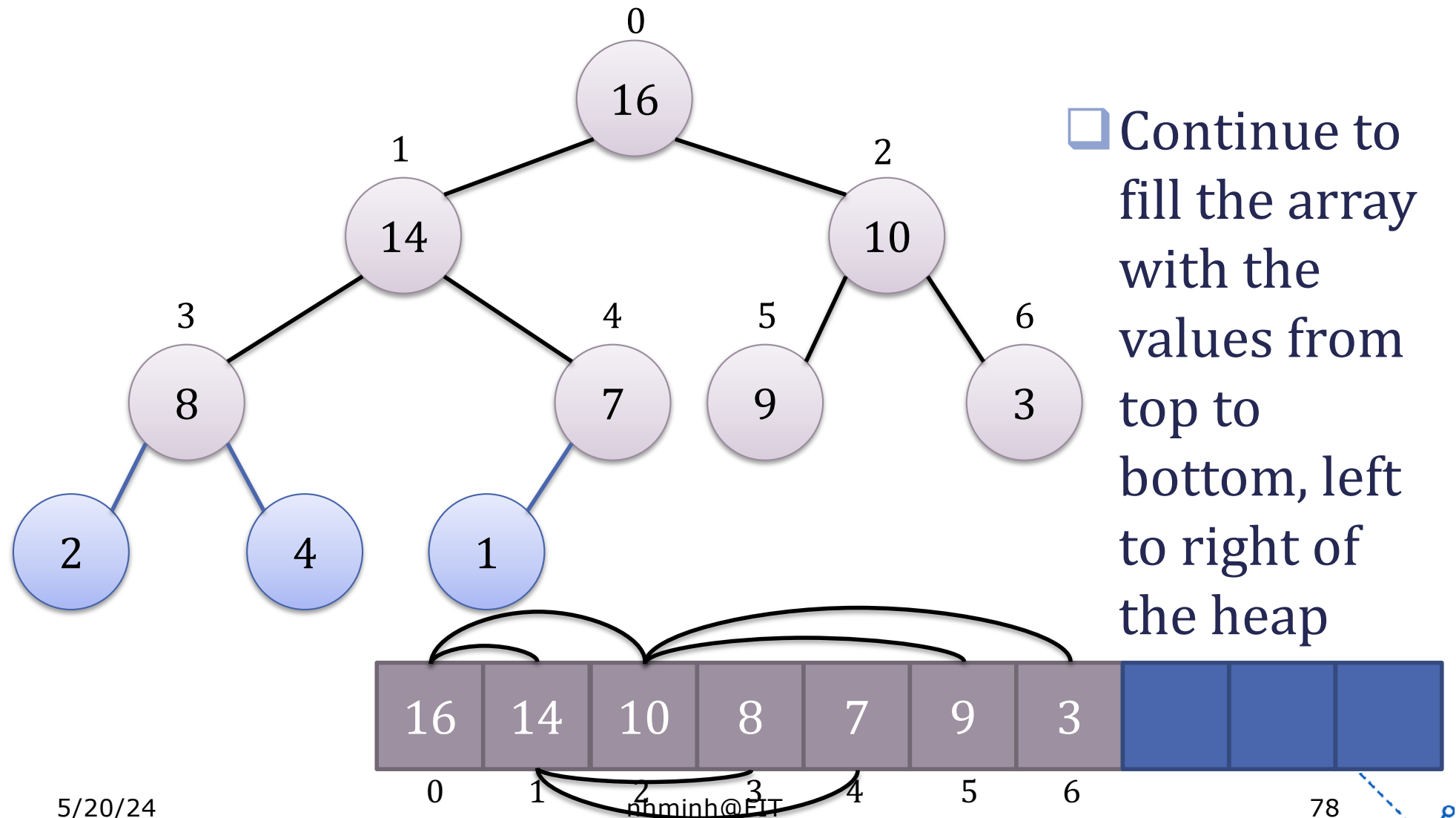




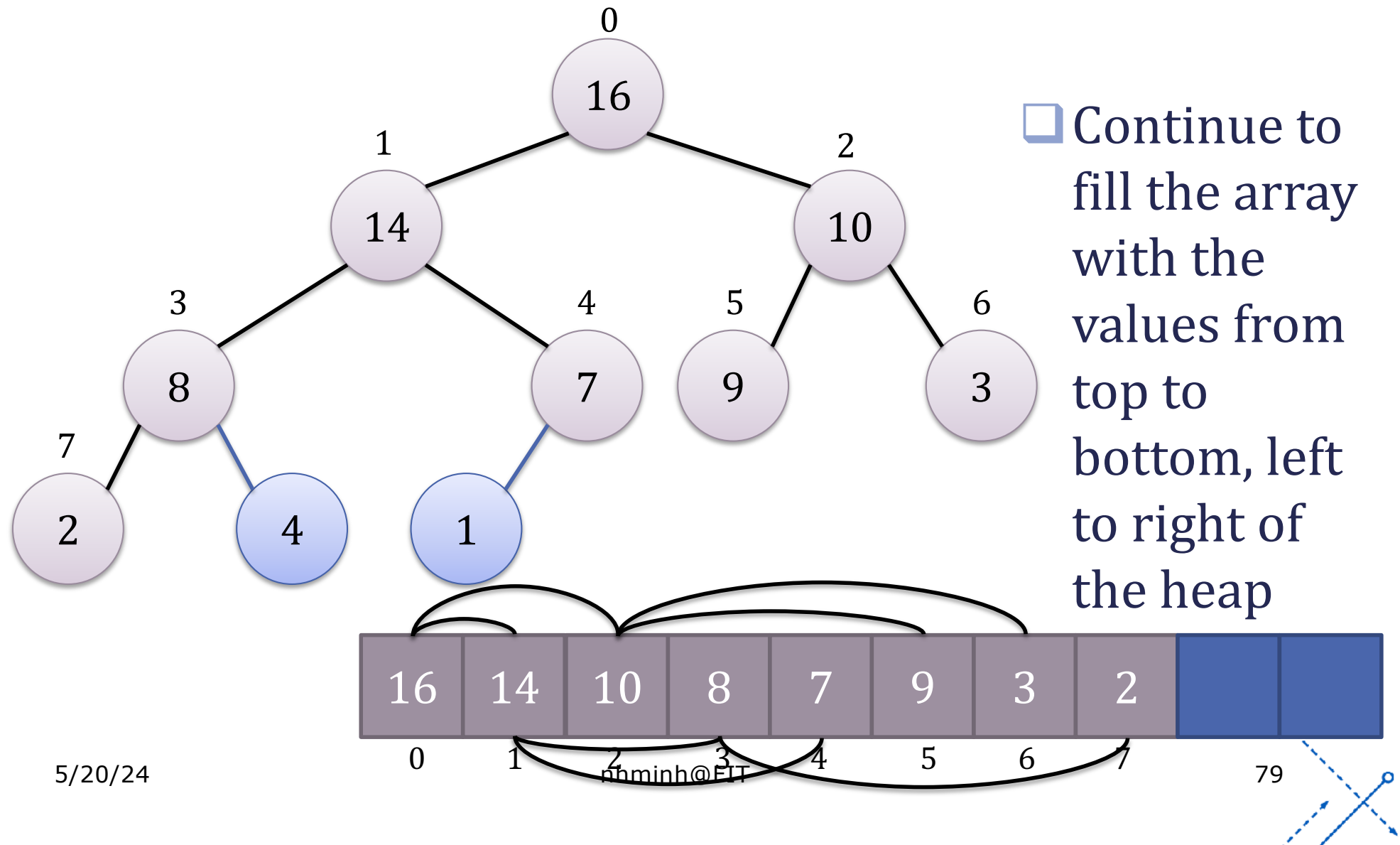
- bottom, left to right of the heap
- 
- ```
graph TD; 16((16)) --- 14((14)); 16 --- 10((10)); 14 --- 8((8)); 14 --- 7((7)); 10 --- 9((9)); 10 --- 3((3)); 8 --- 2((2)); 8 --- 4((4)); 7 --- 1((1)); style 16 fill:#d8bfd8; style 14 fill:#d8bfd8; style 10 fill:#d8bfd8; style 8 fill:#d8bfd8; style 7 fill:#d8bfd8; style 2 fill:#add8e6; style 4 fill:#add8e6; style 1 fill:#add8e6; style 9 fill:#add8e6; style 3 fill:#add8e6;
```



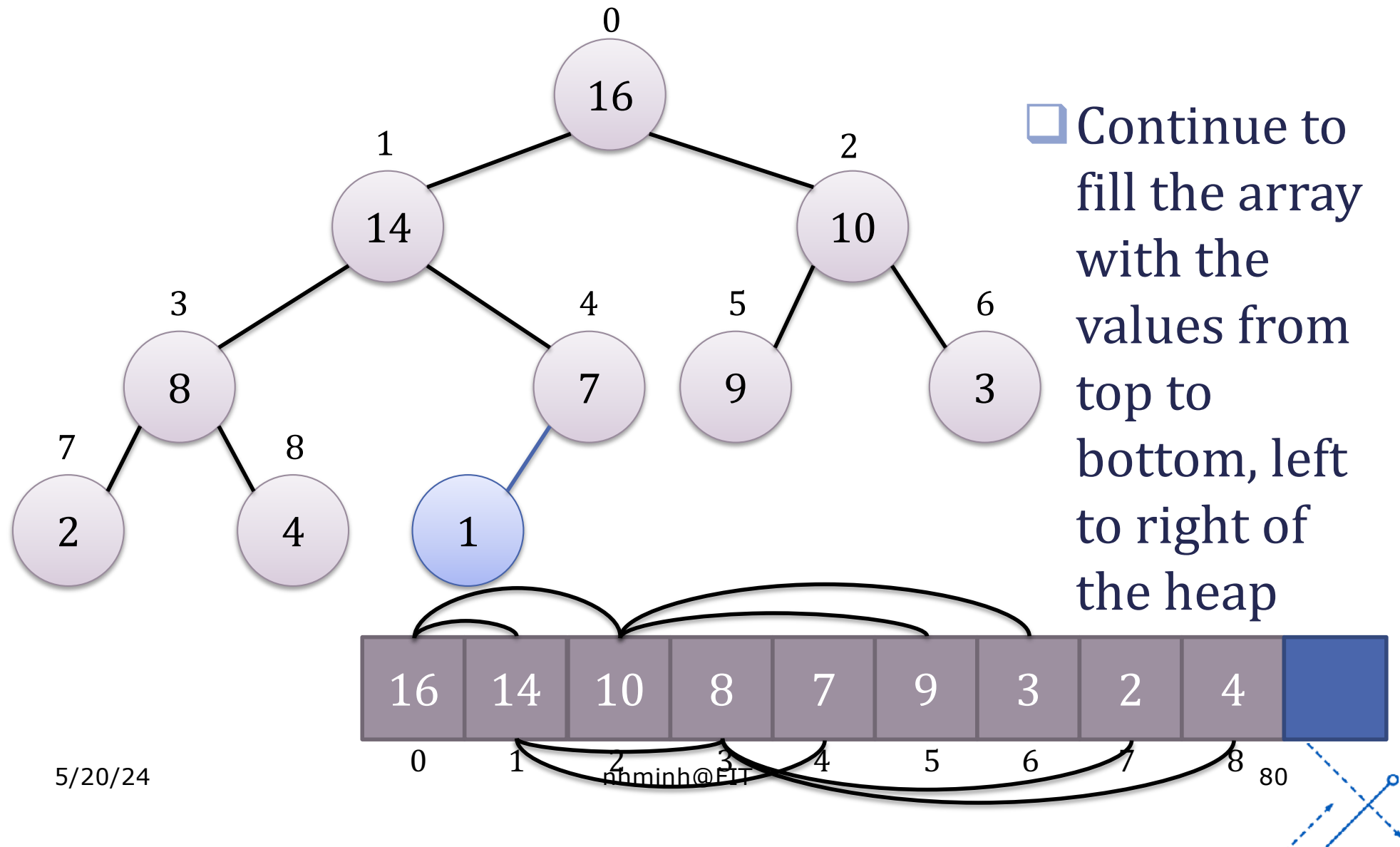
Heap and its array representation



Heap and its array representation

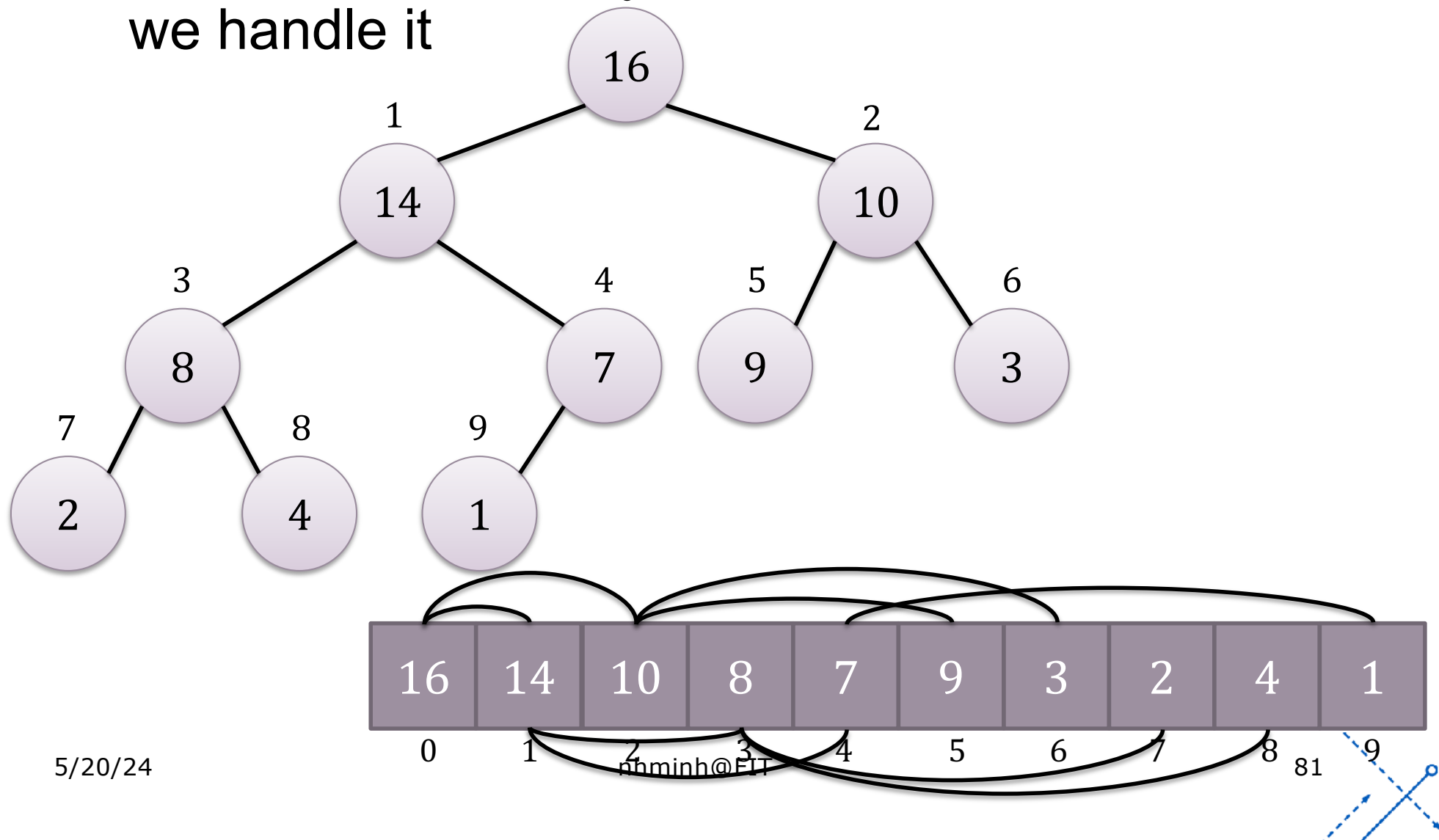


Heap and its array representation

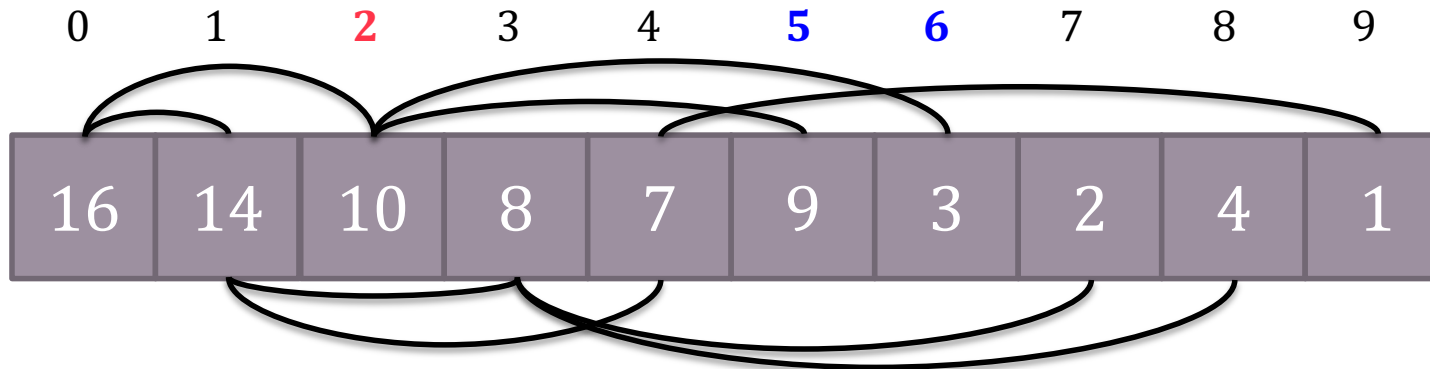


Heap and its array representation

- An array is viewed as a heap just because the way we handle it



Heap and its array representation



- Given the index i of a node, the indices of its parent, left child and right child are:

PARENT(i)

return $(i-1)/2$;

LEFT(i)

return $2i+1$;

RIGHT(i)

return $2i+2$;



Building a Heap

- Build a heap from the following array:

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

Building a Heap

- Each leaf node is a 1-element heap

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

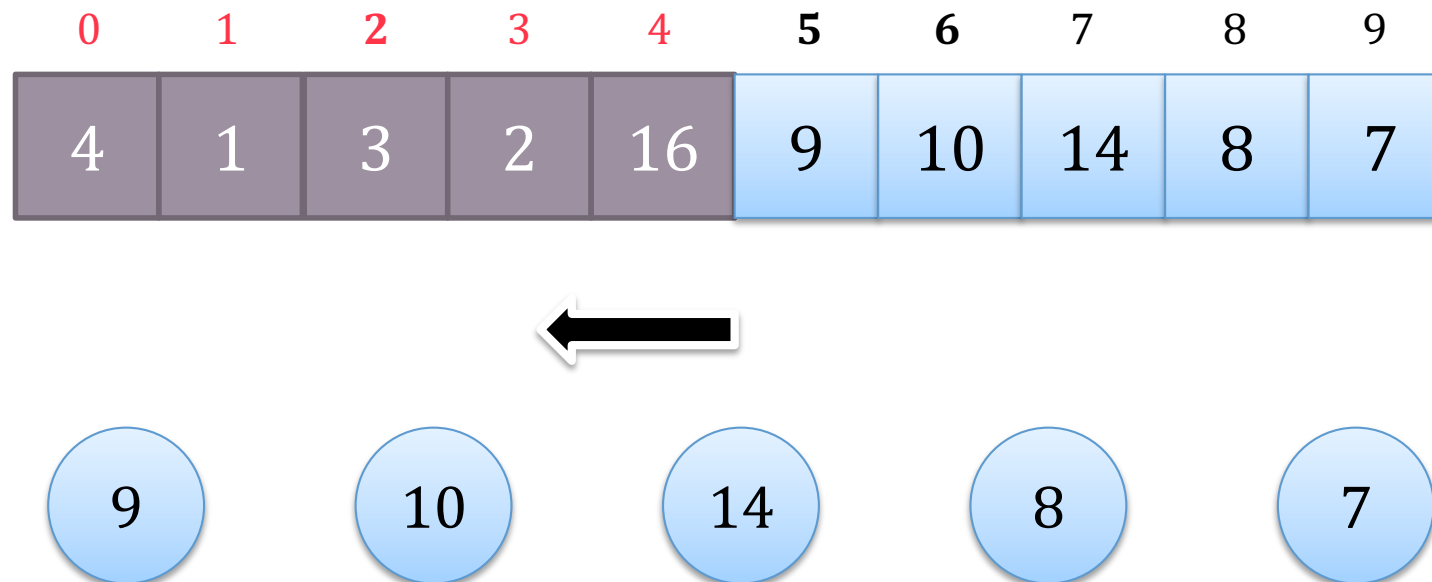


(The elements in the subarray $A[n/2 \dots n-1]$ are all leaves)



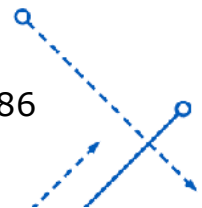
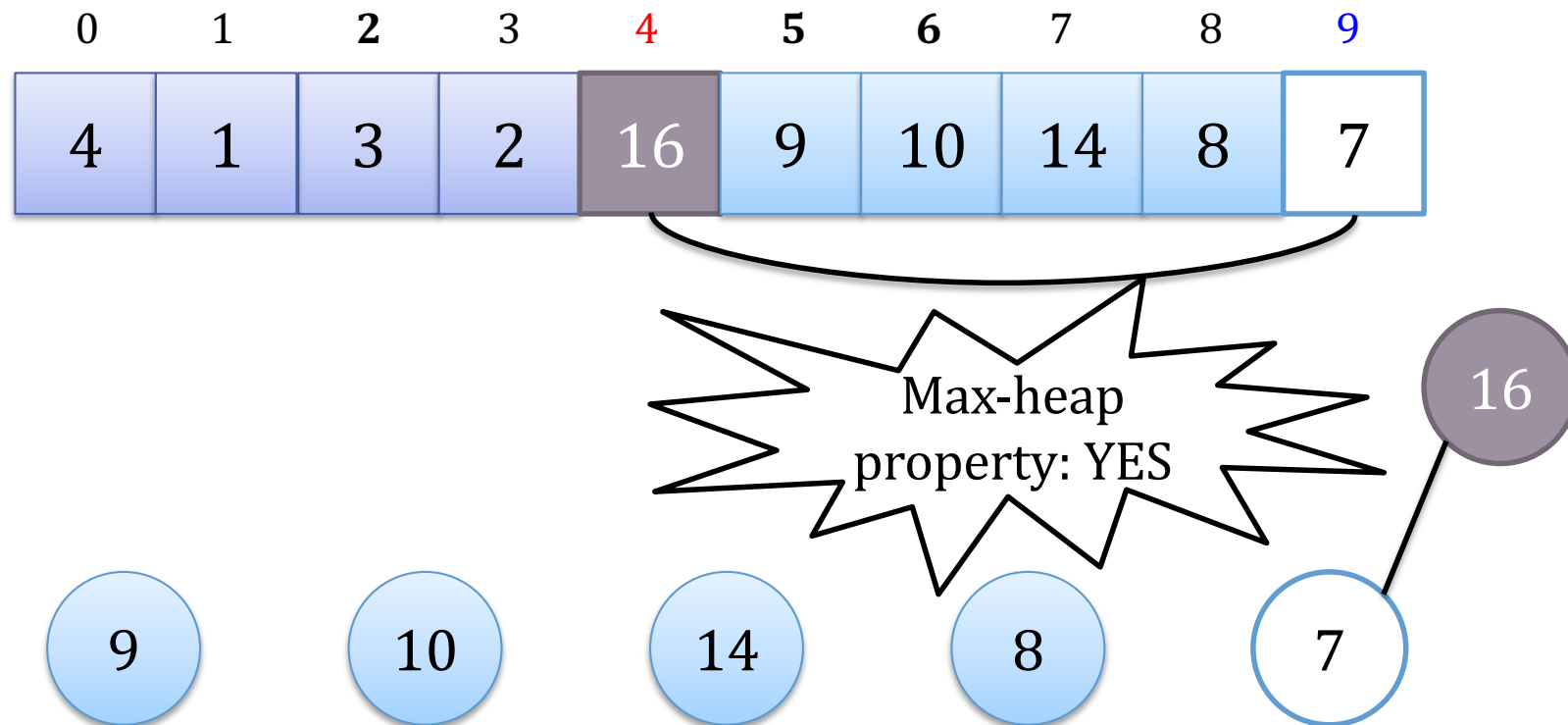
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



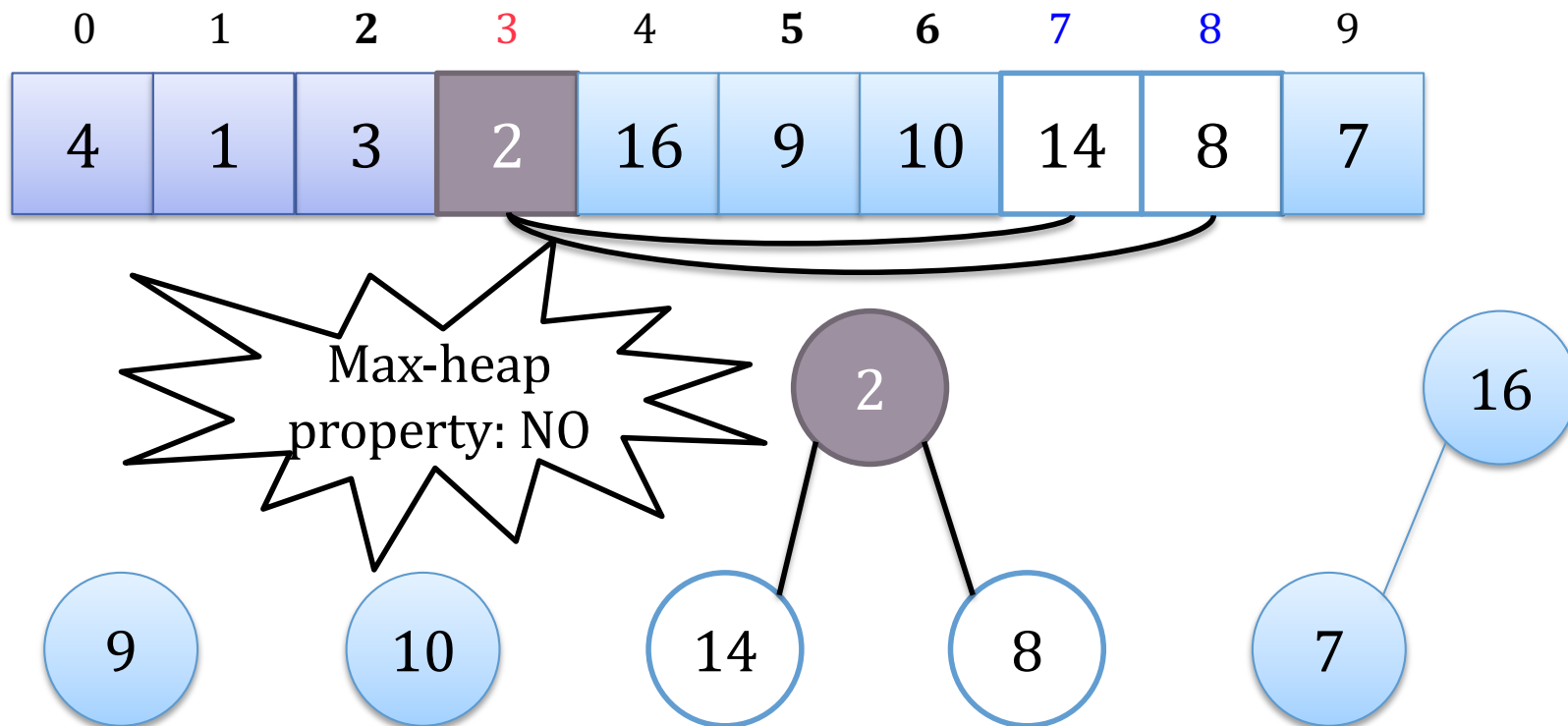
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



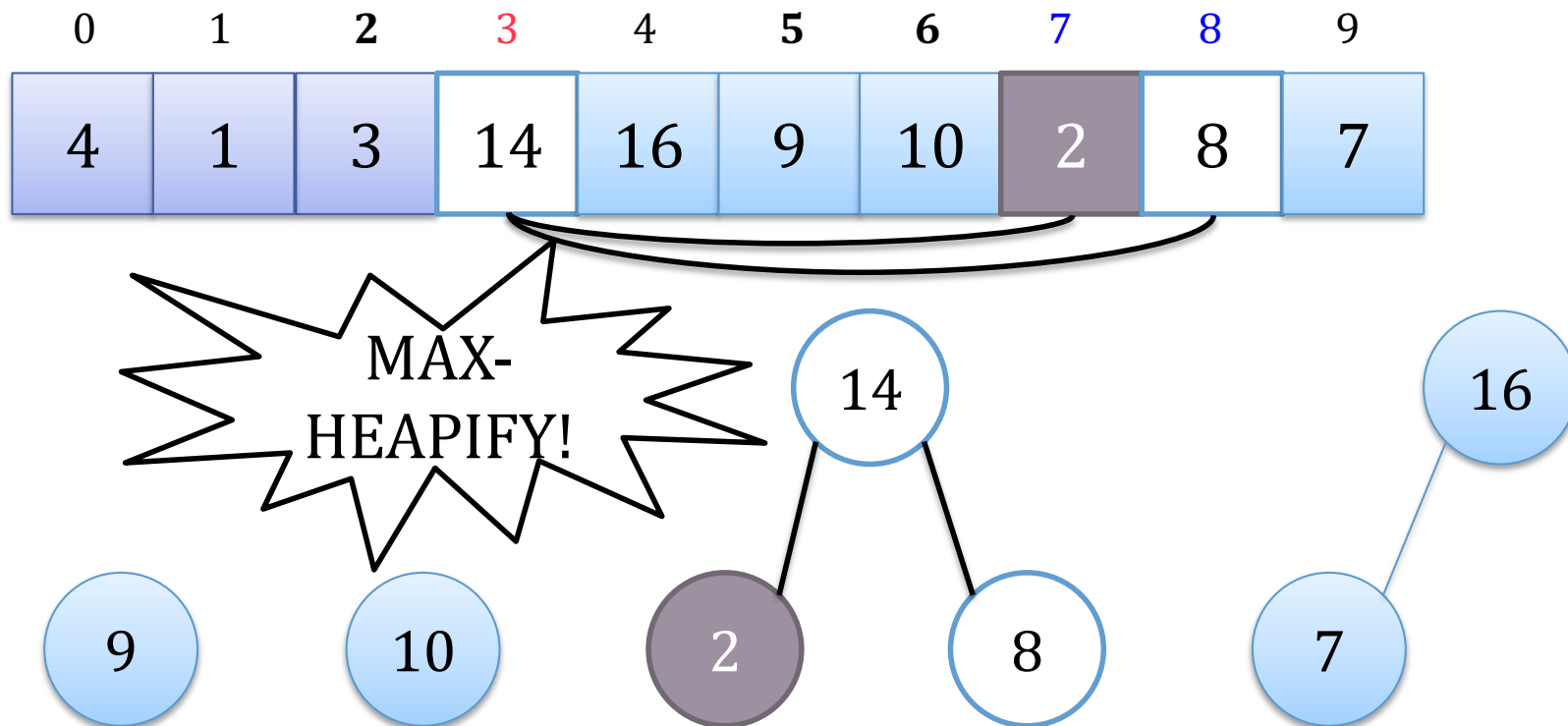
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



Building a Heap

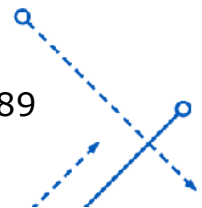
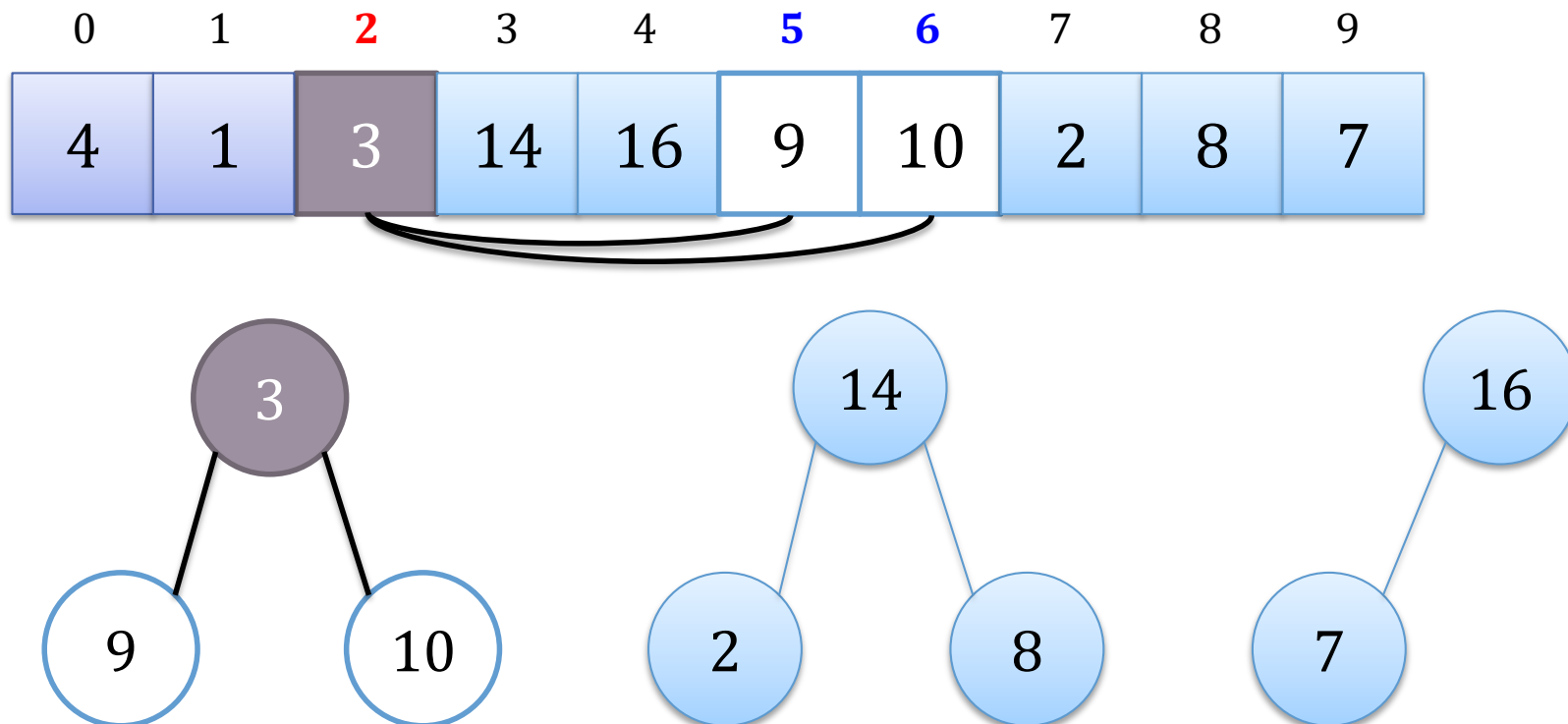
- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



Exchange with the
largest node from left
child and right child

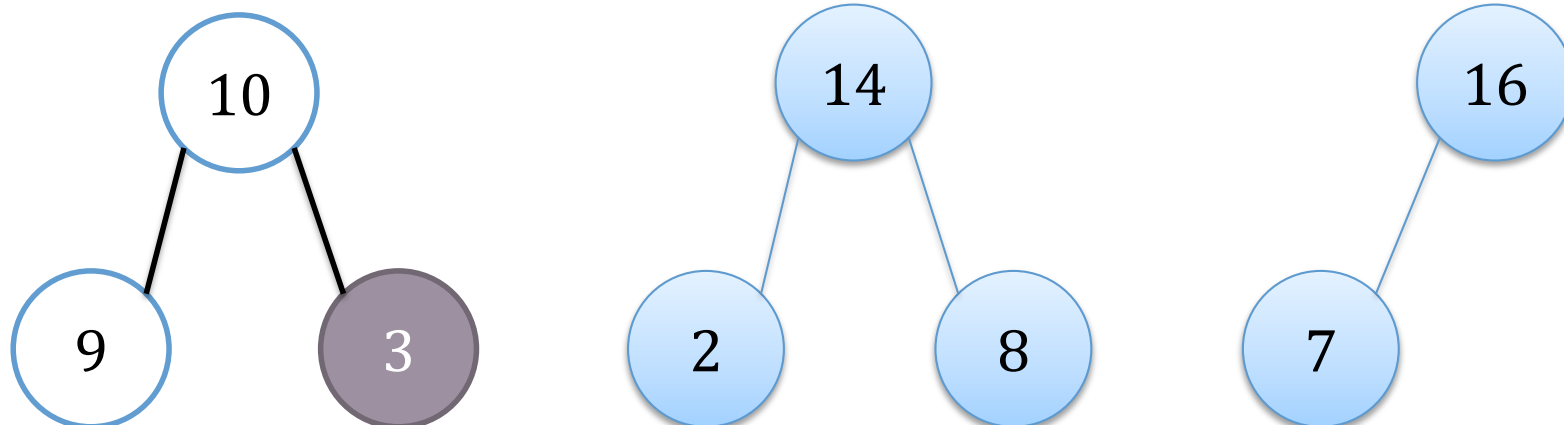
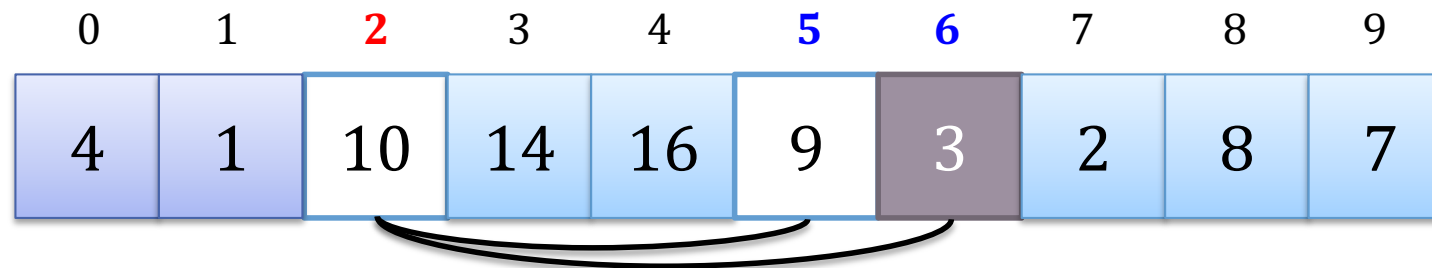
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



Building a Heap

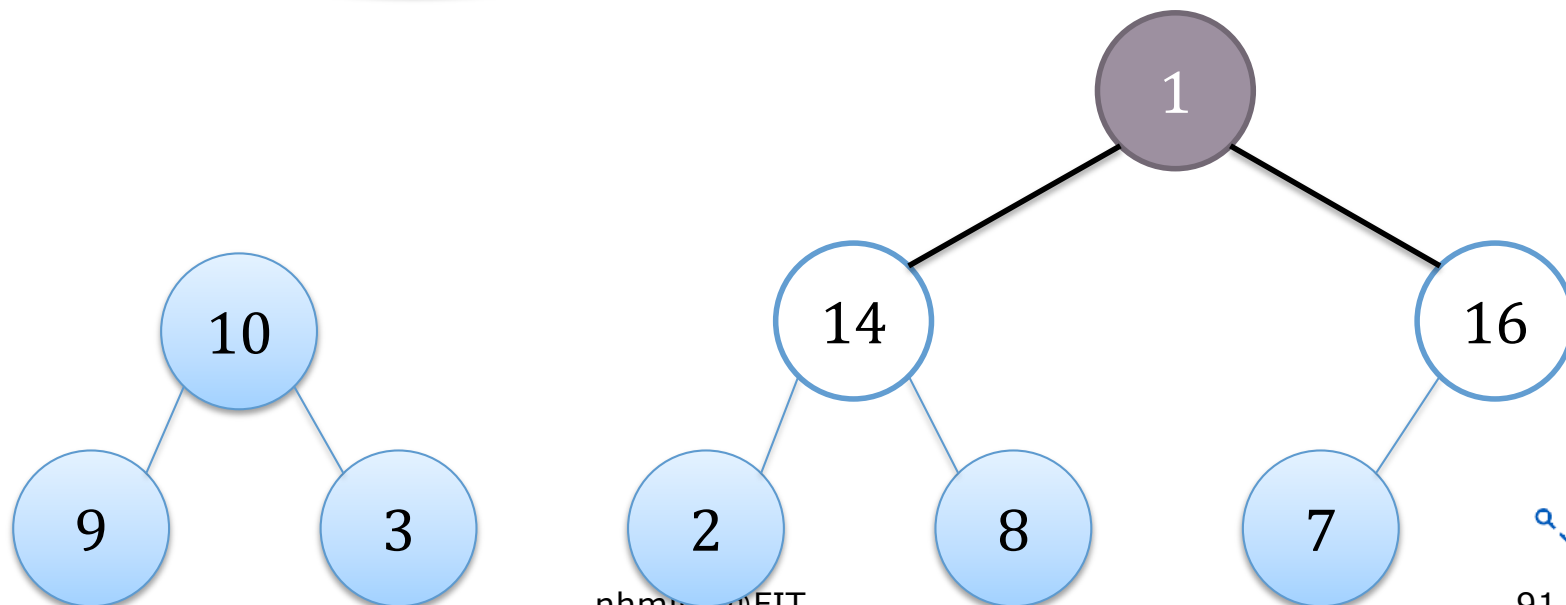
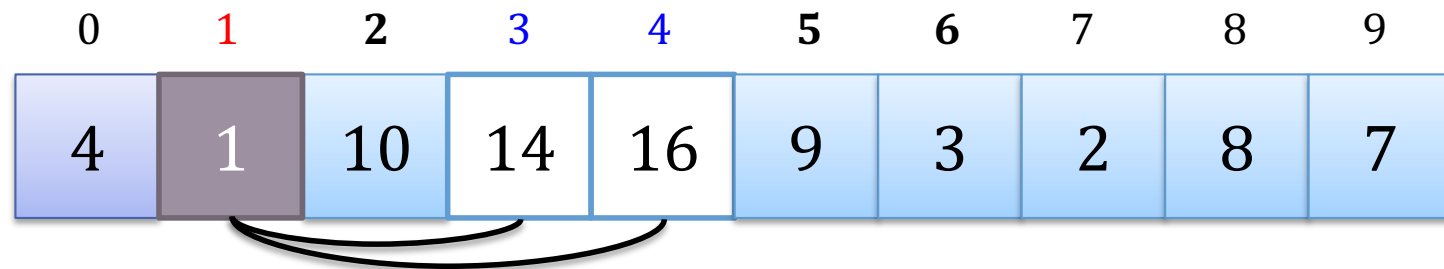
- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



Exchange with the
largest node from left
child and right child

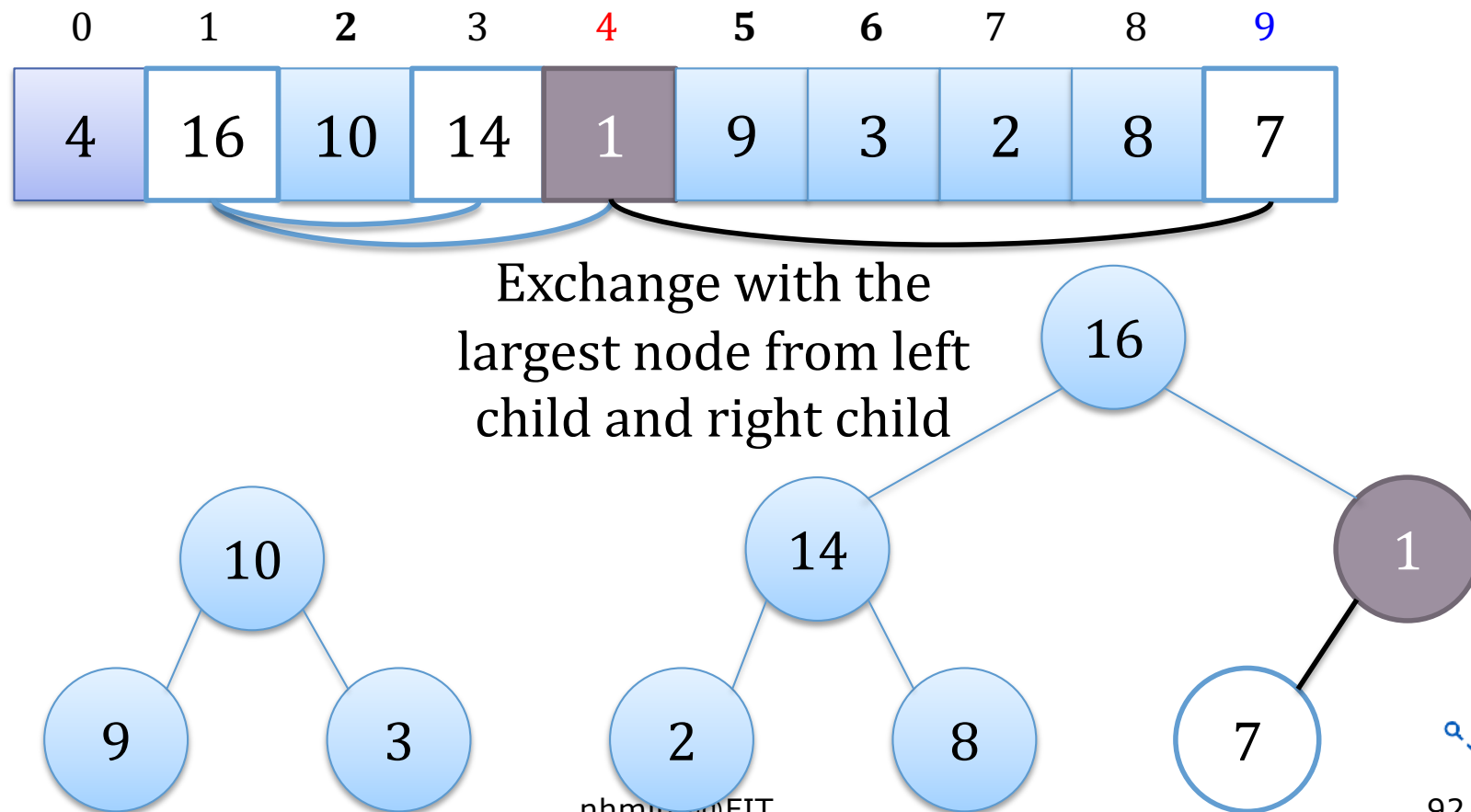
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



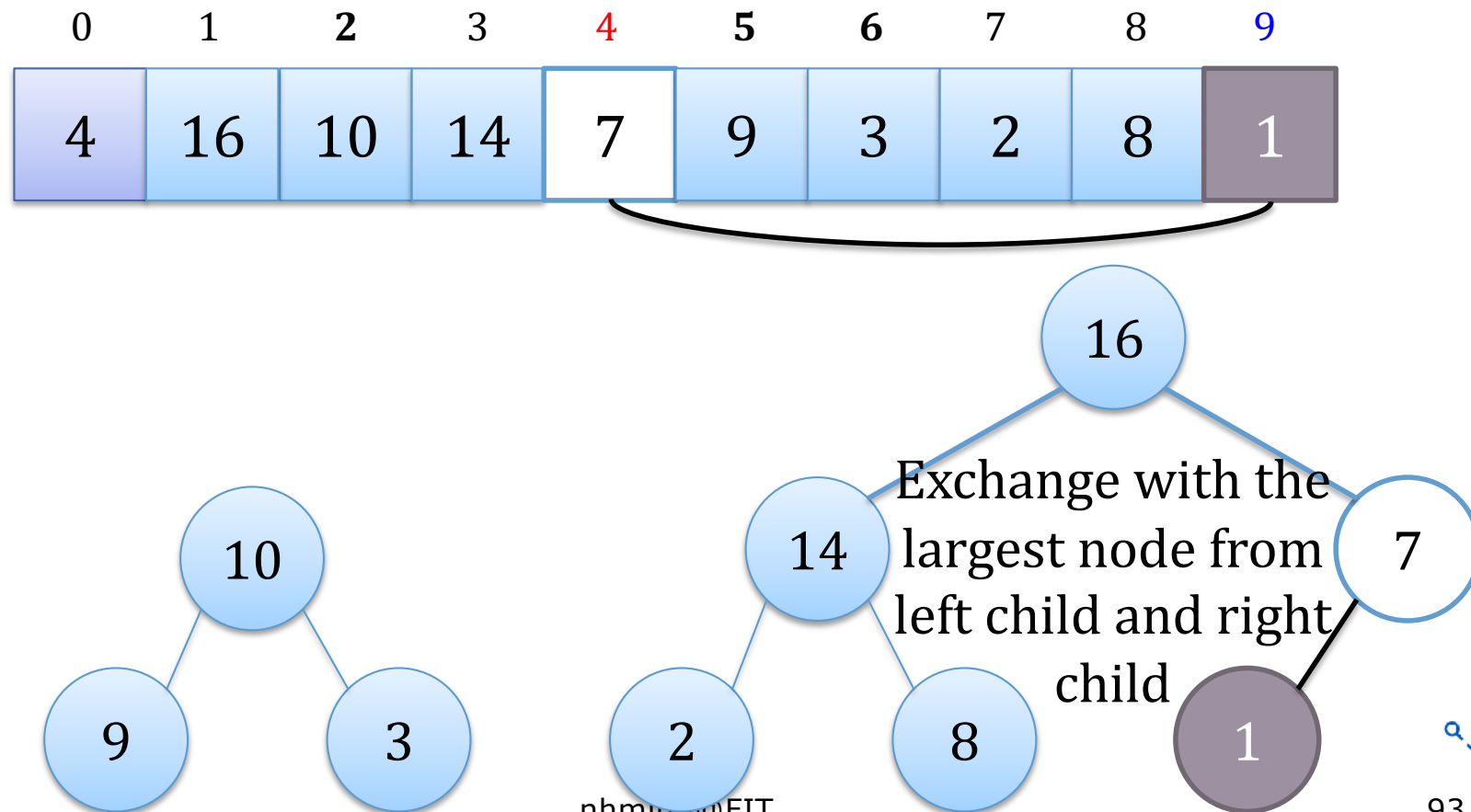
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



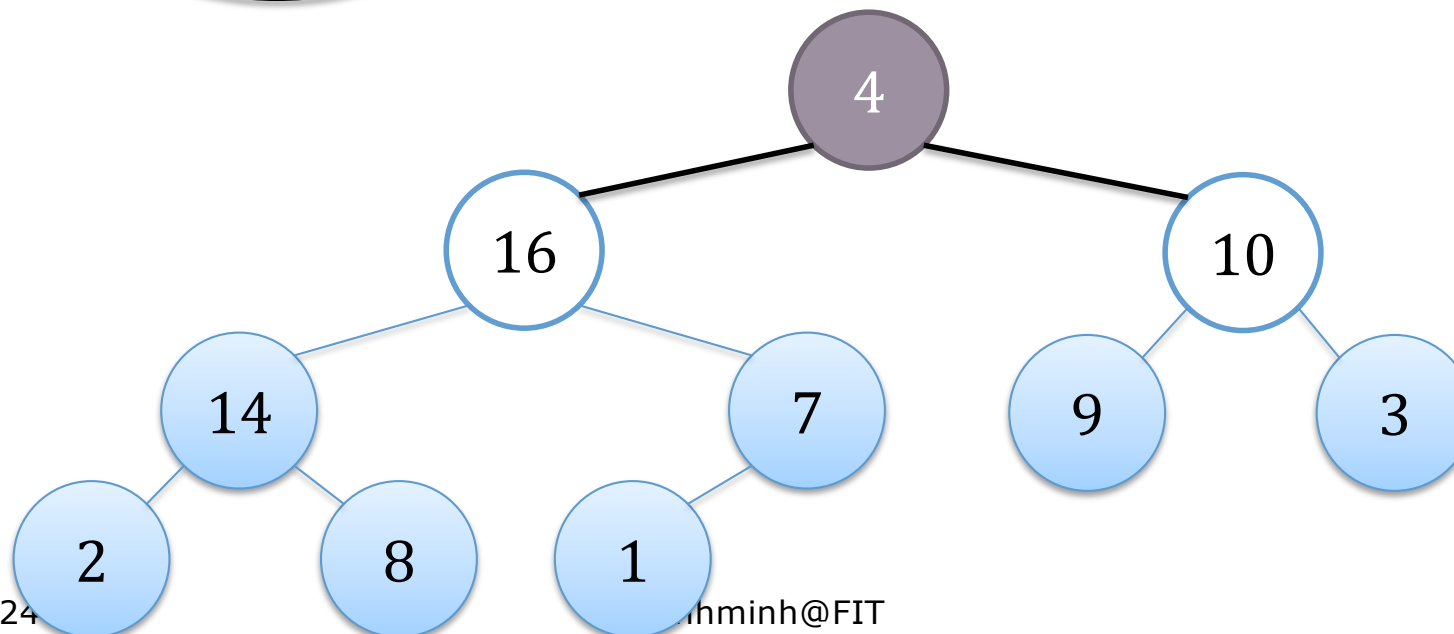
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



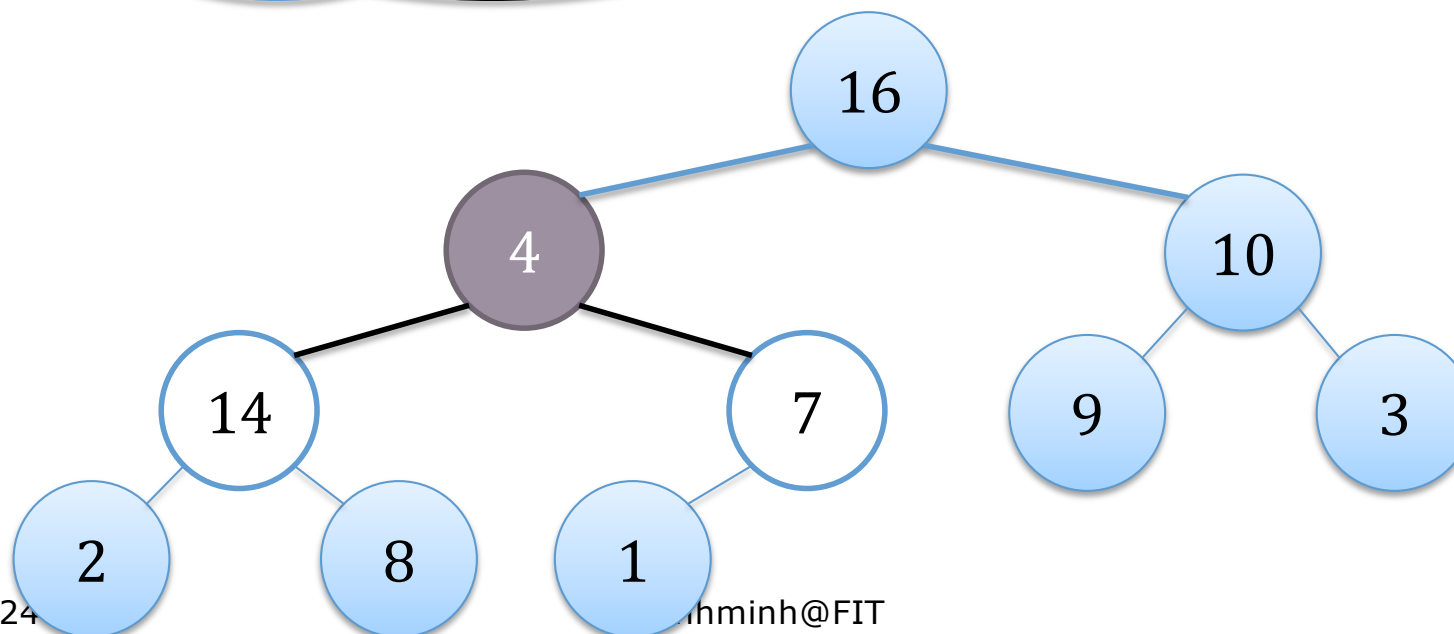
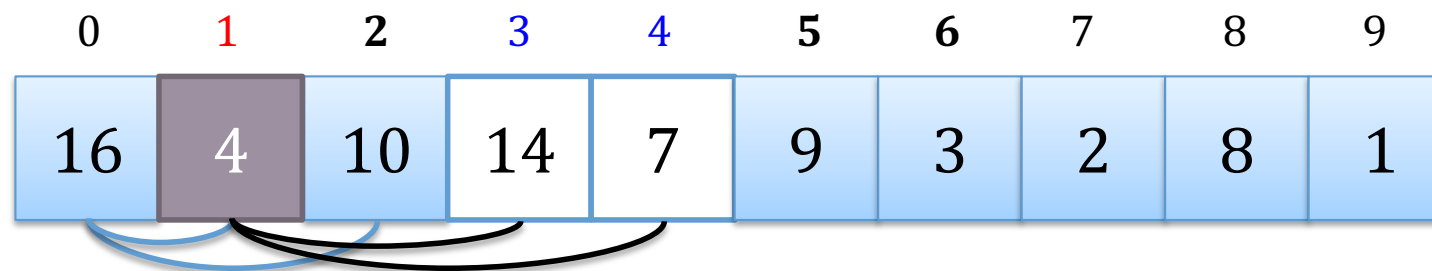
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



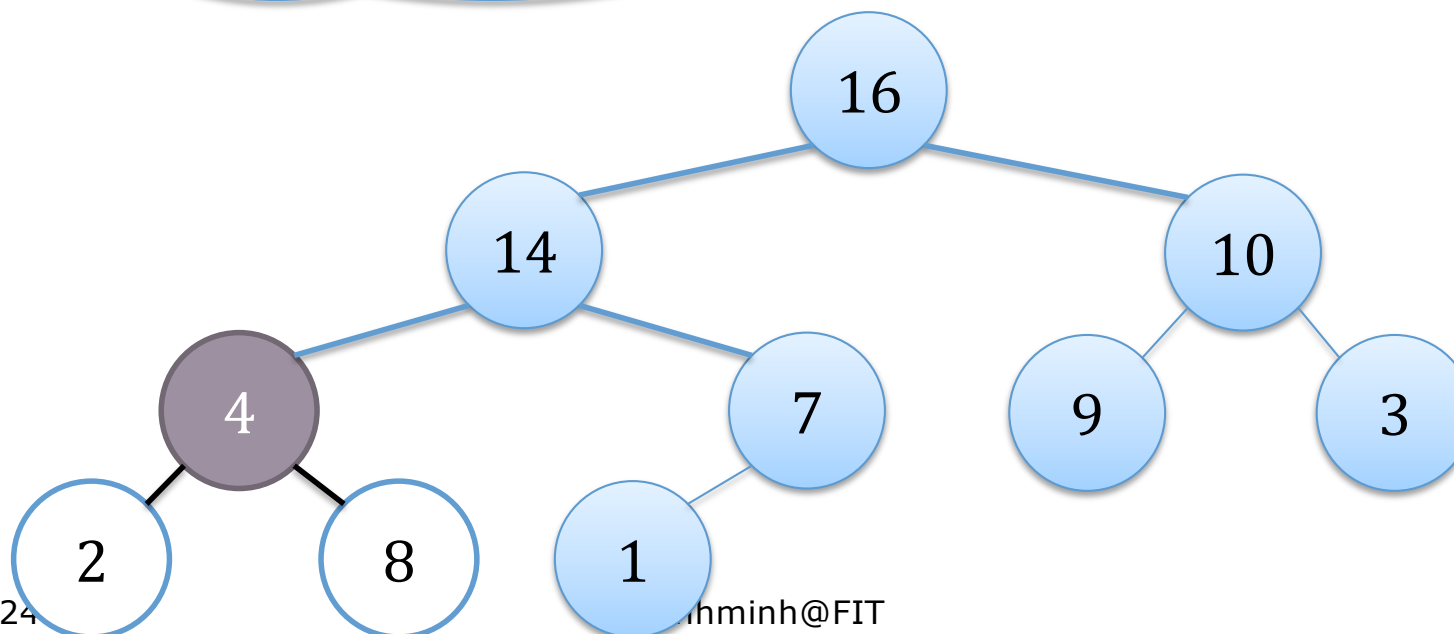
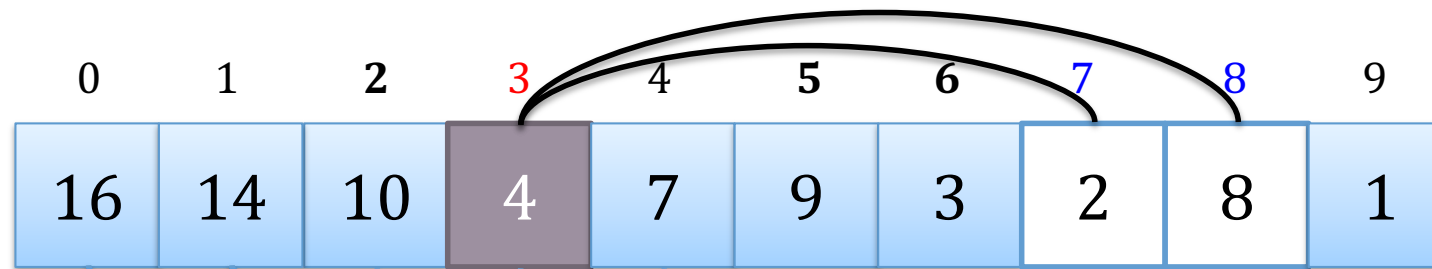
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



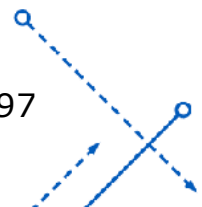
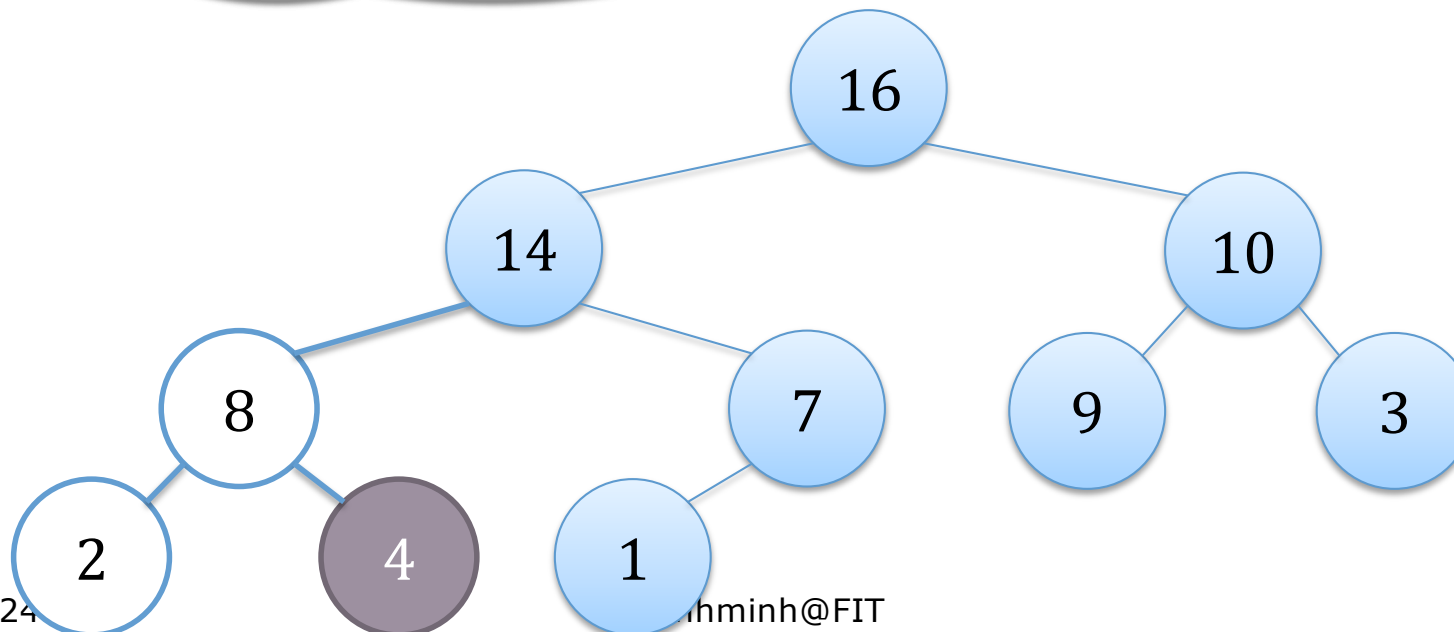
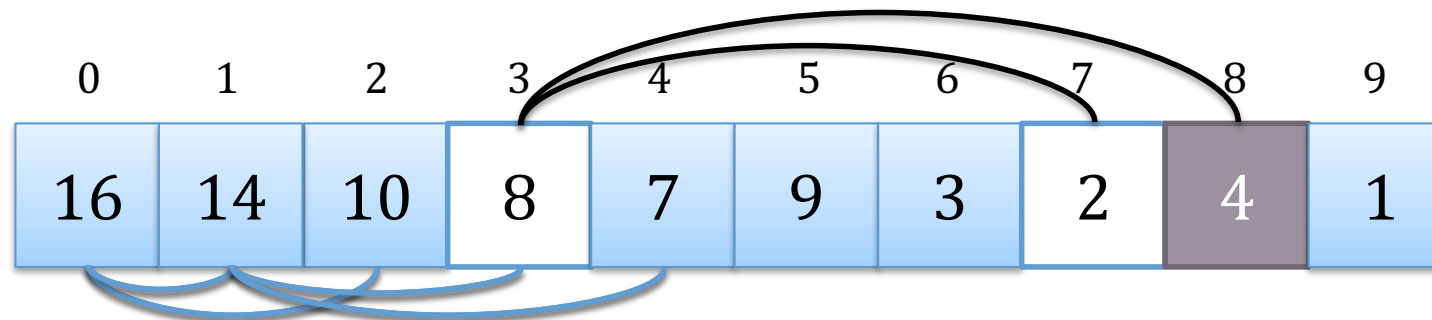
Building a Heap

- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



Building a Heap

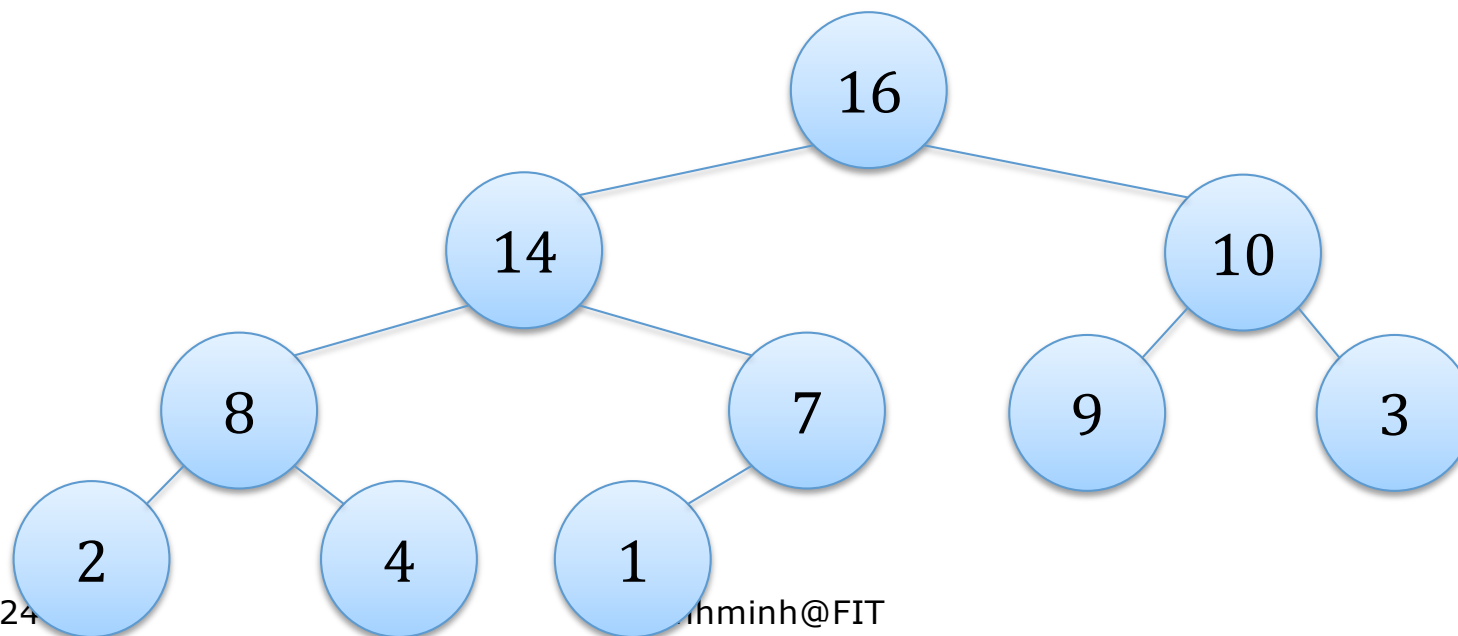
- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine



Building a Heap

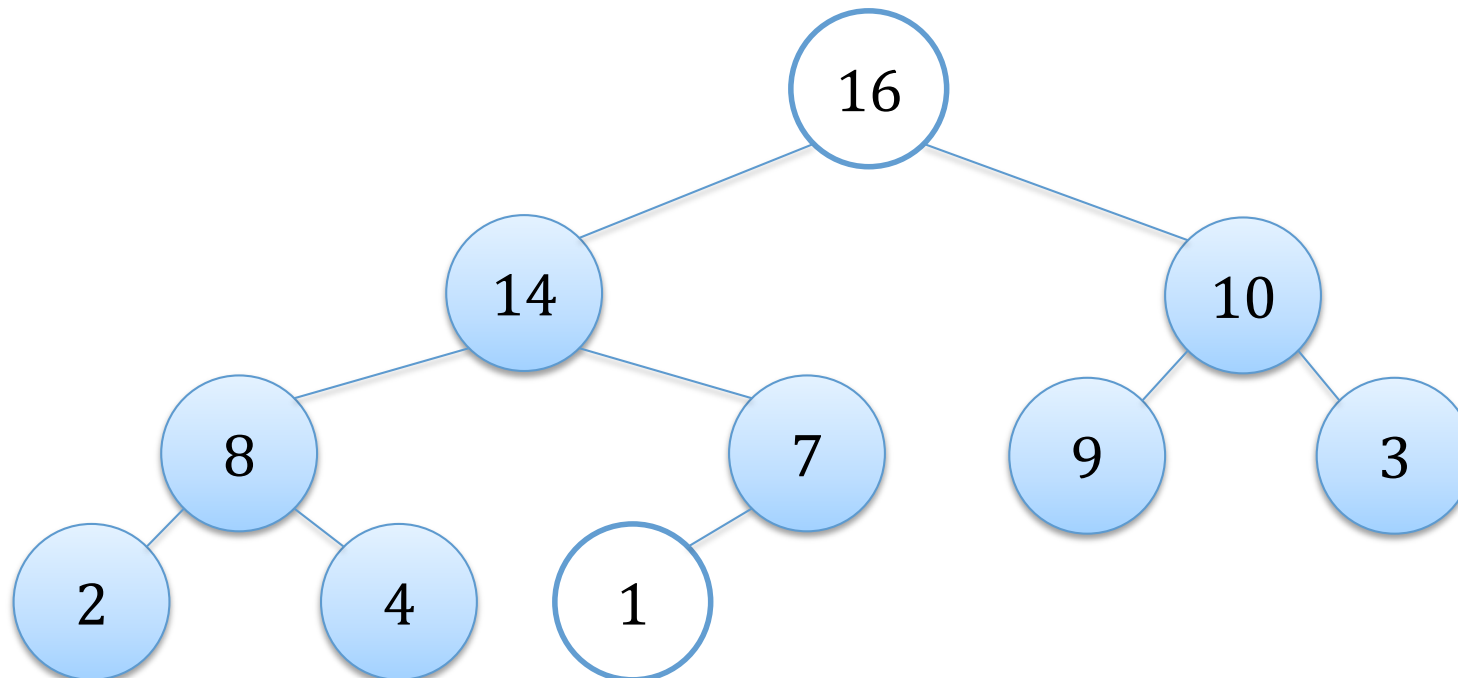
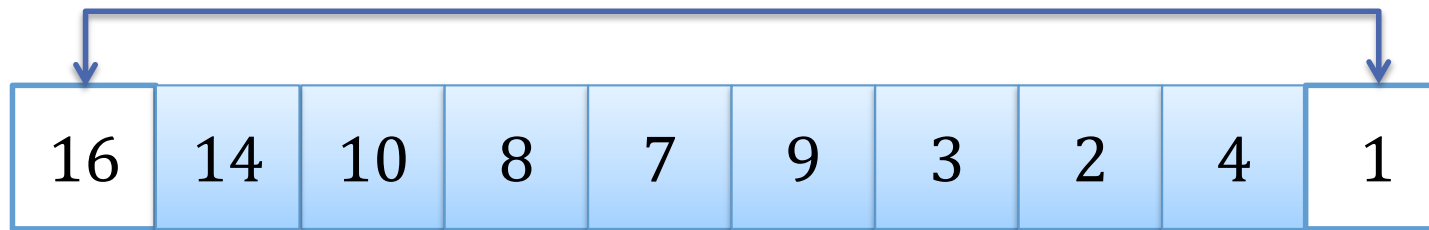
- Adjust the remaining nodes from right to left using MAX-HEAPIFY subroutine

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1



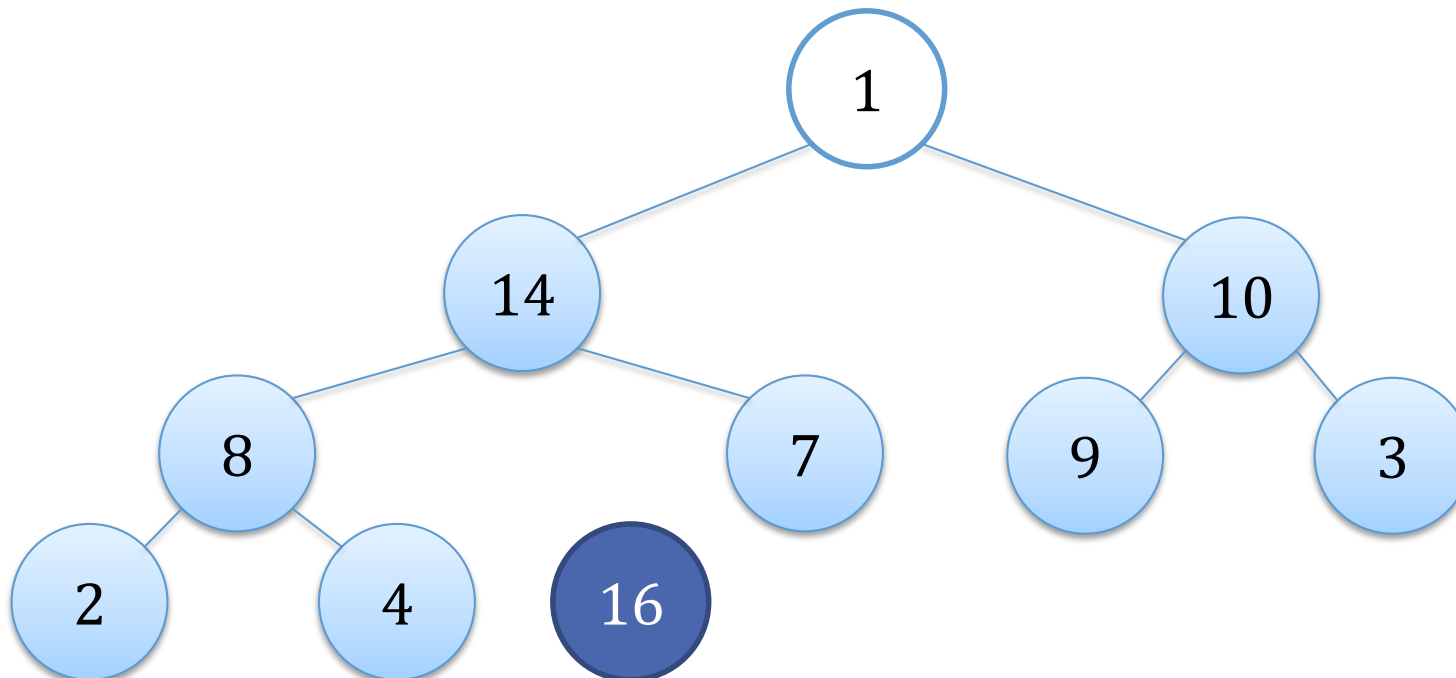
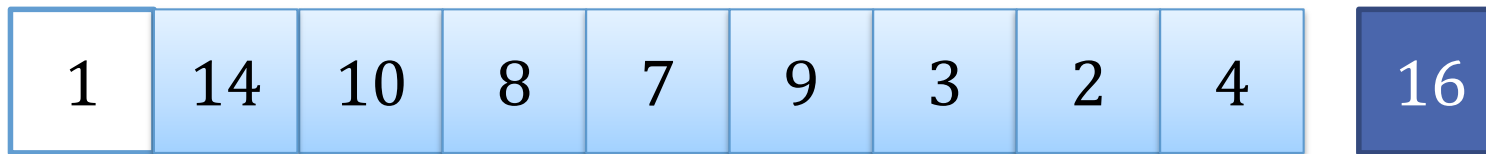
Heap sort Algorithm

Exchange



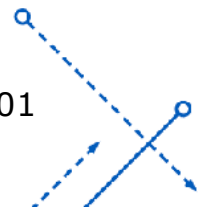
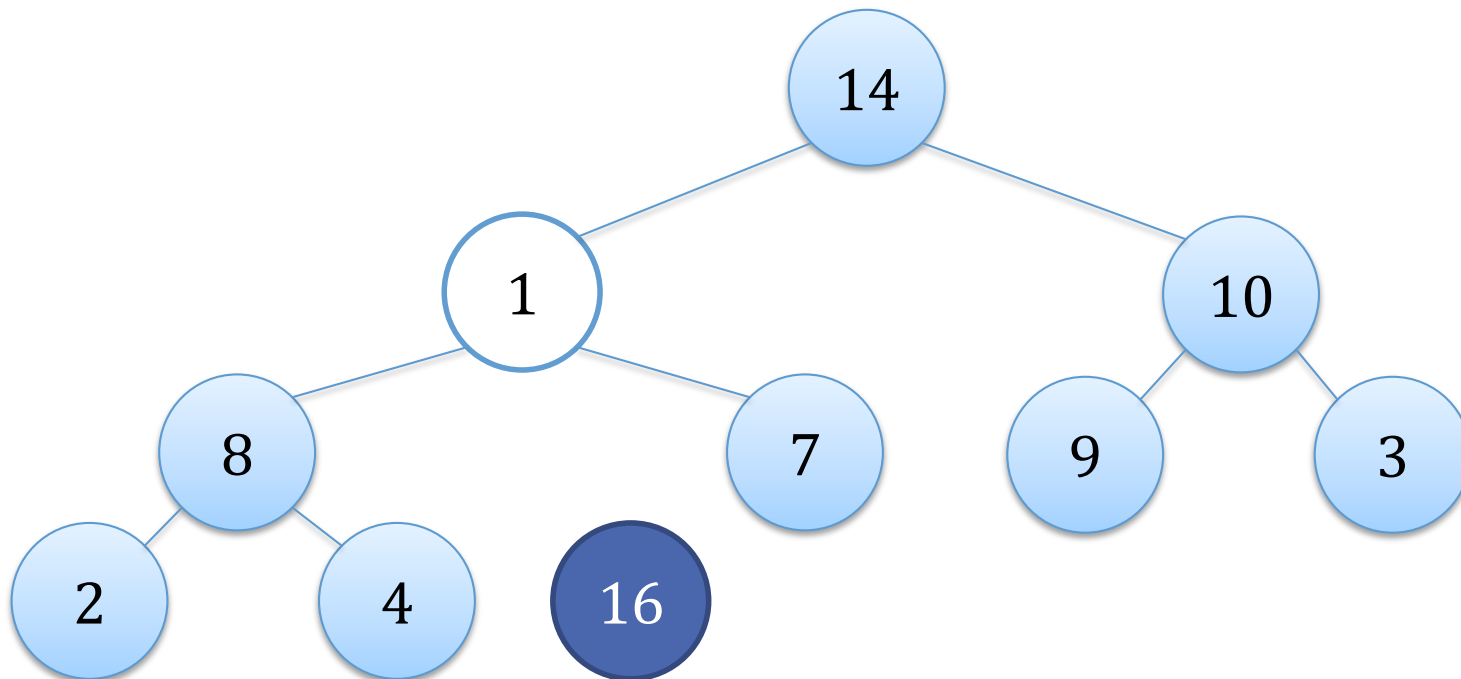
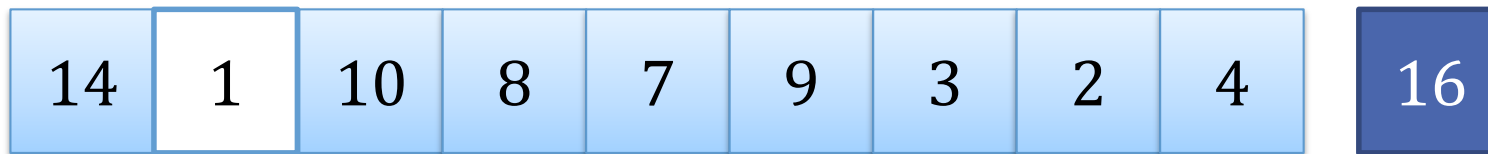
Heap sort Algorithm

- Discard the last node



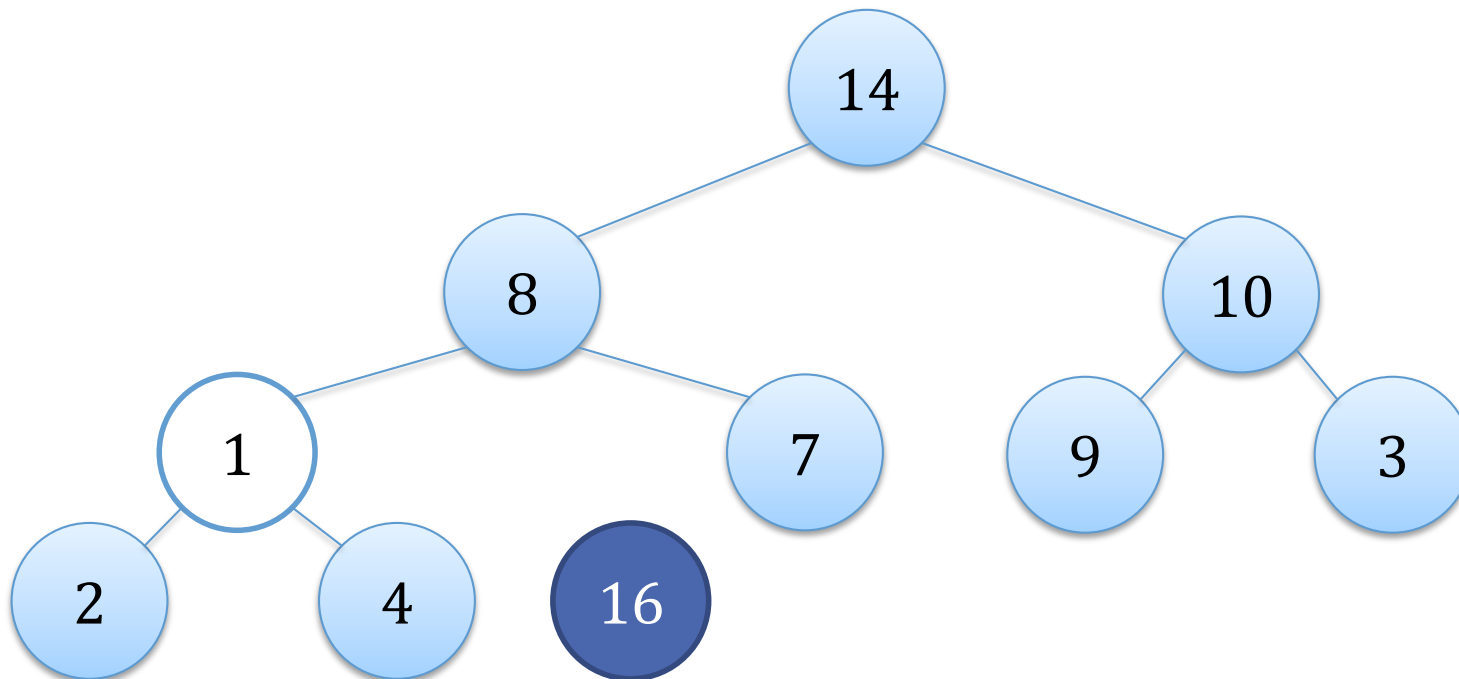
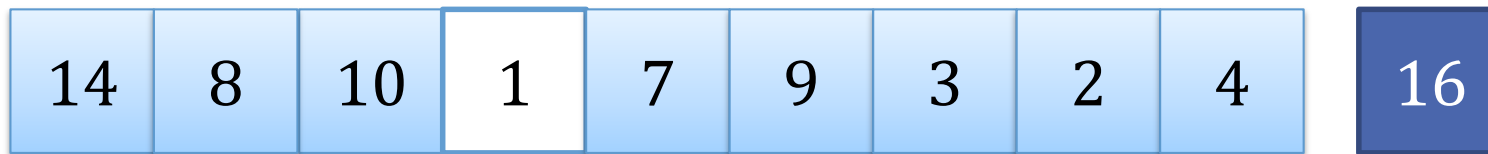
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



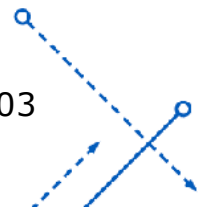
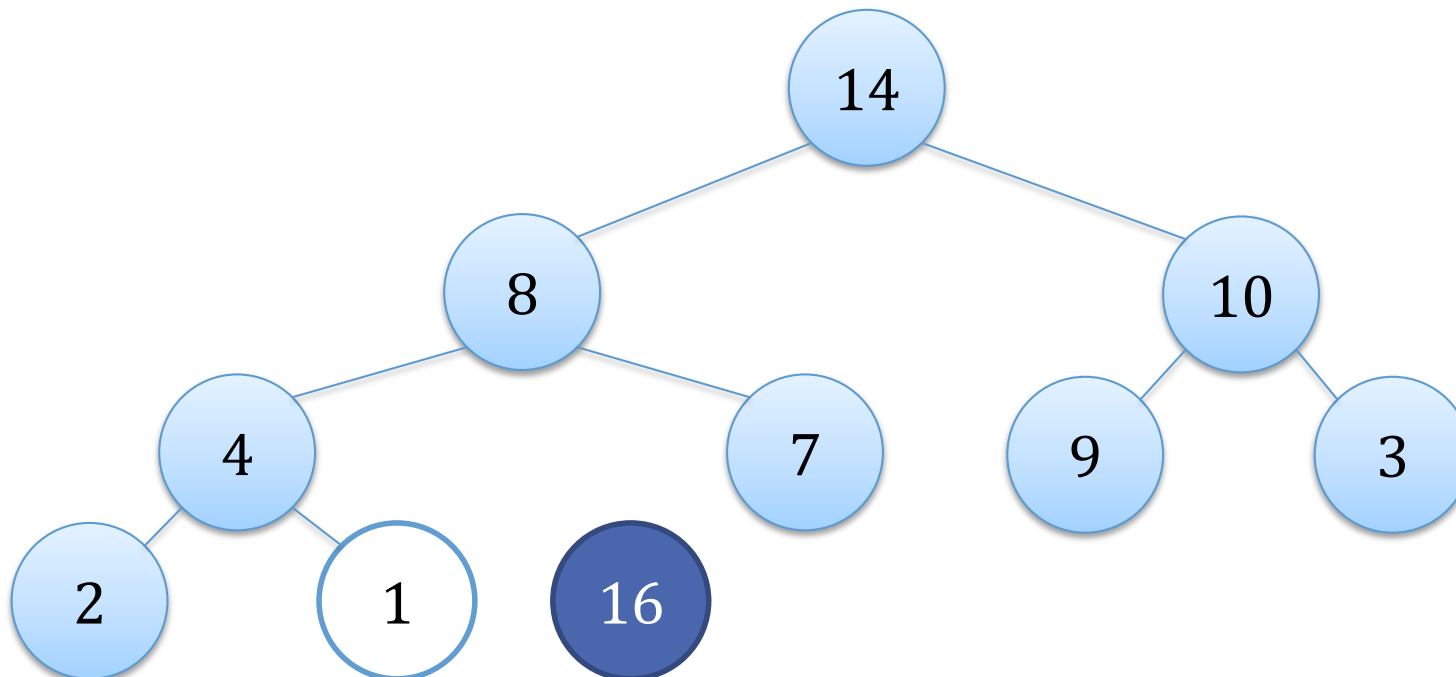
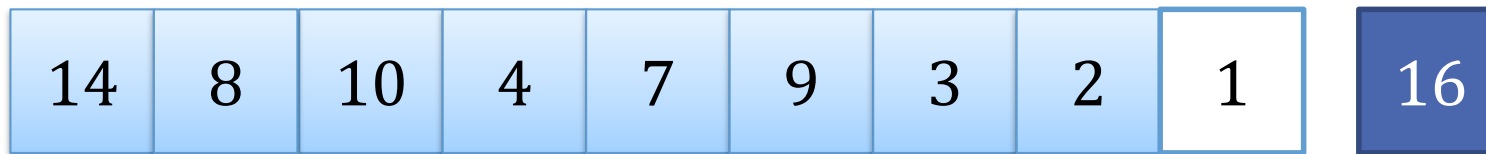
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



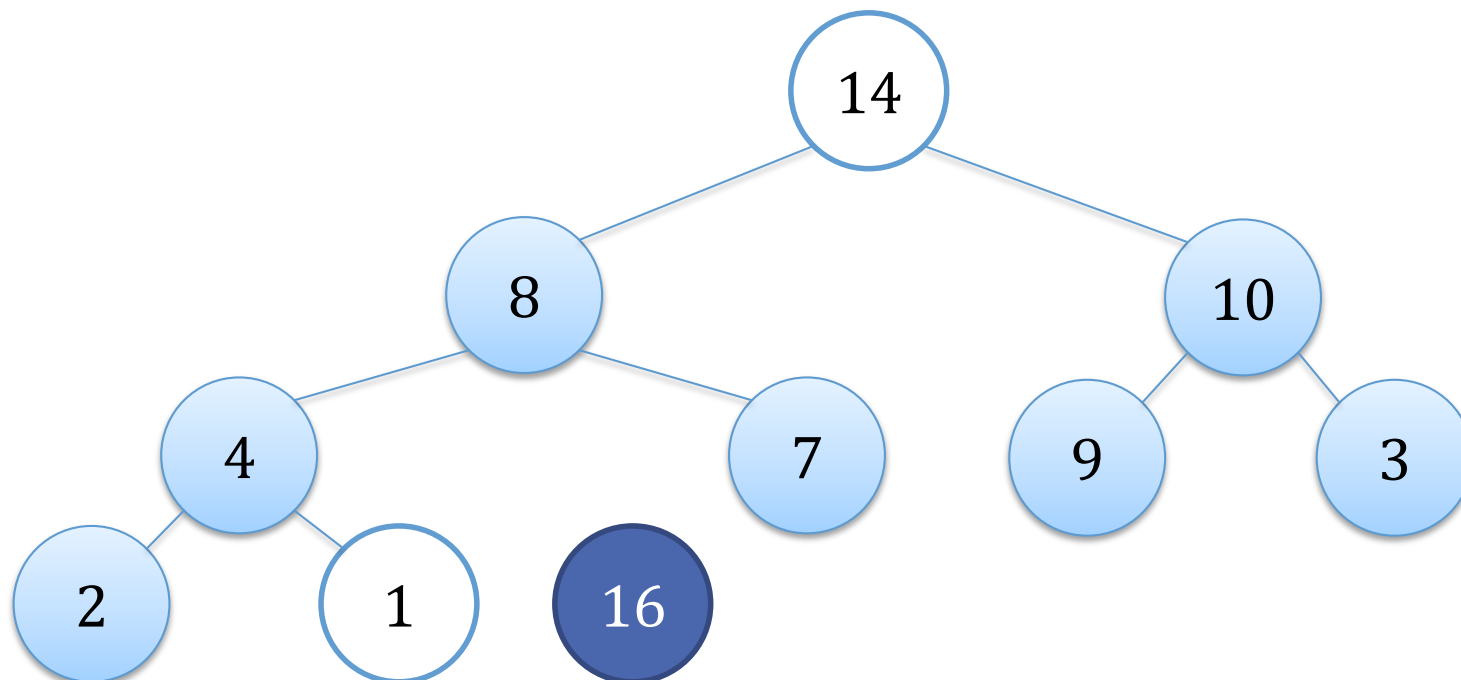
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



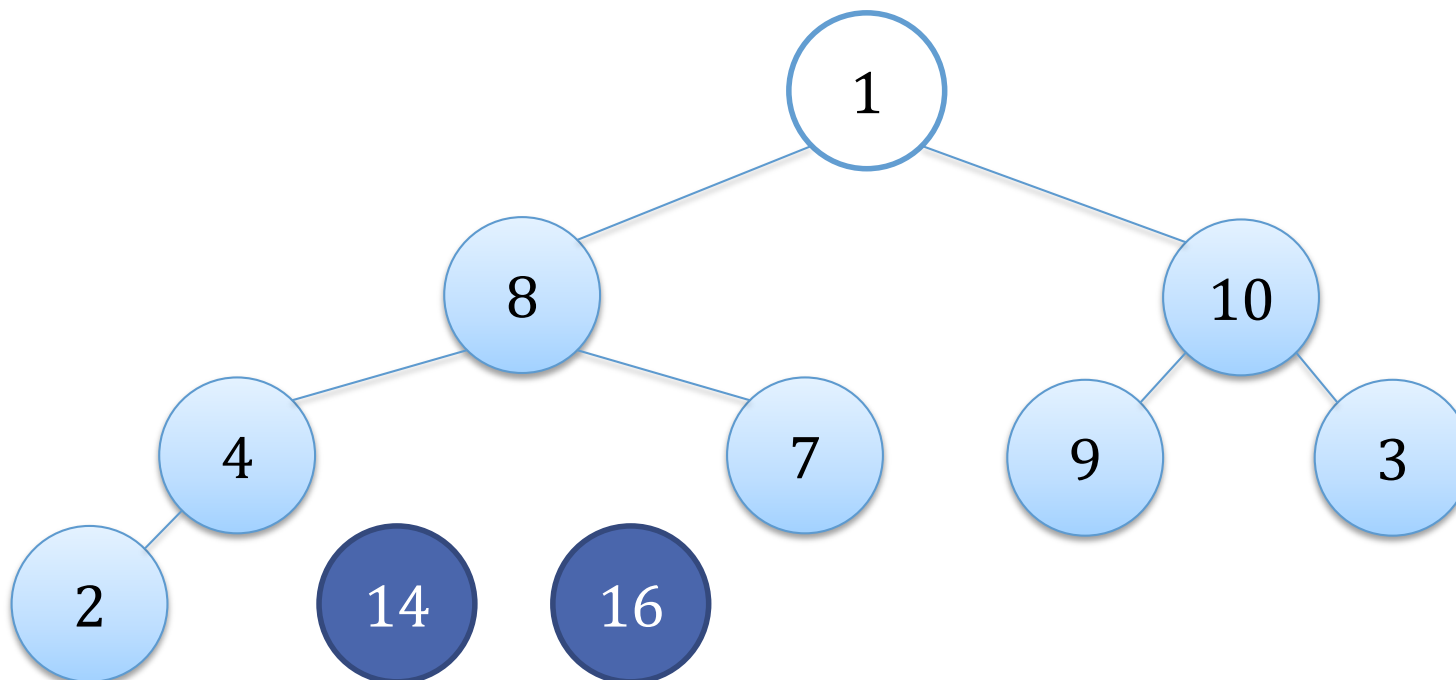
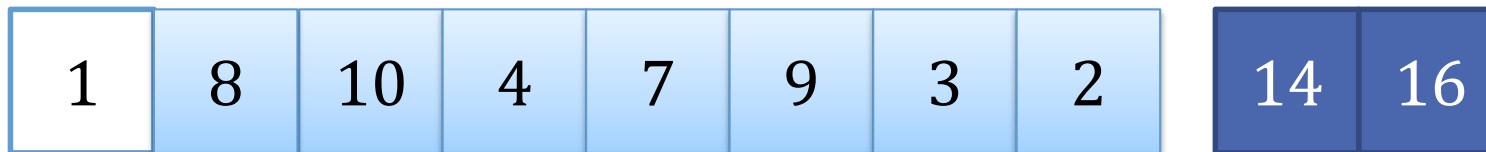
Heap sort Algorithm

Exchange



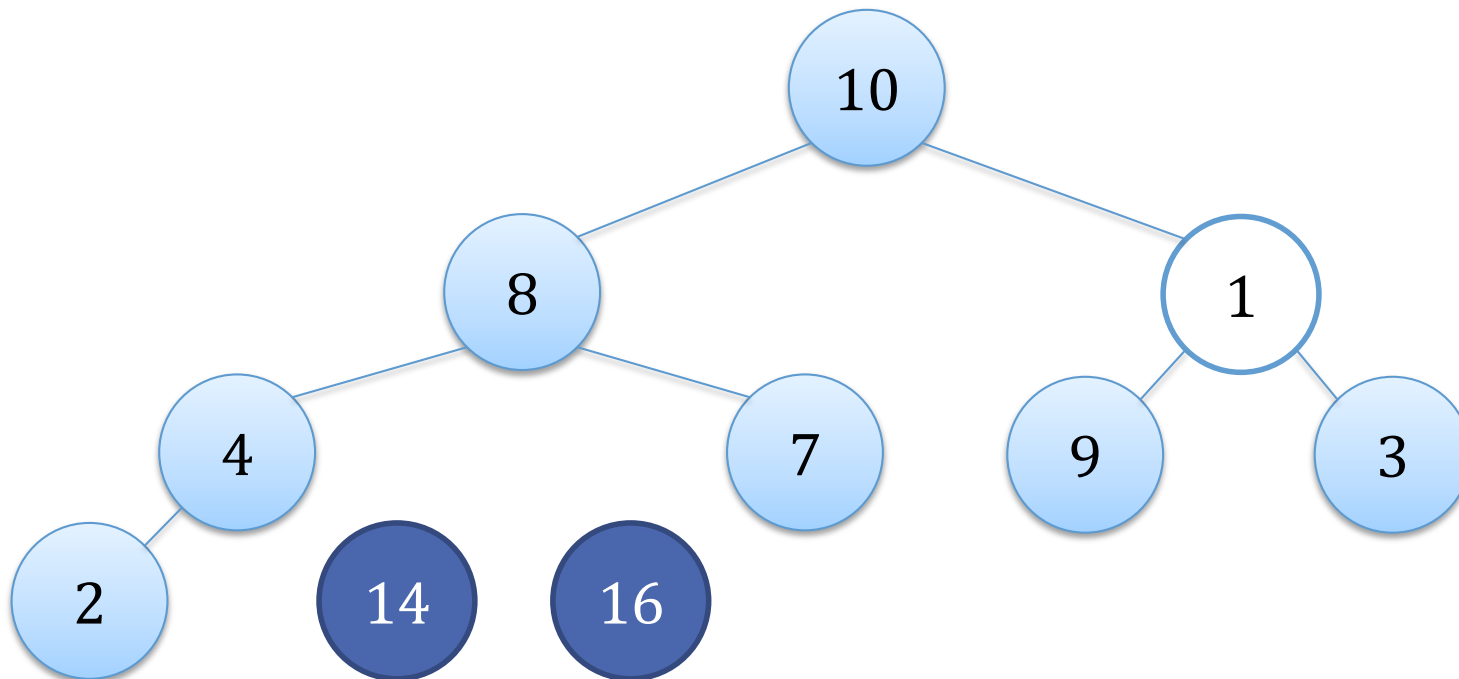
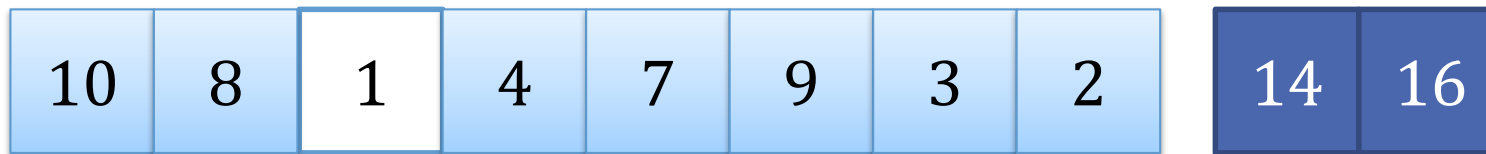
Heap sort Algorithm

- Discard the last node



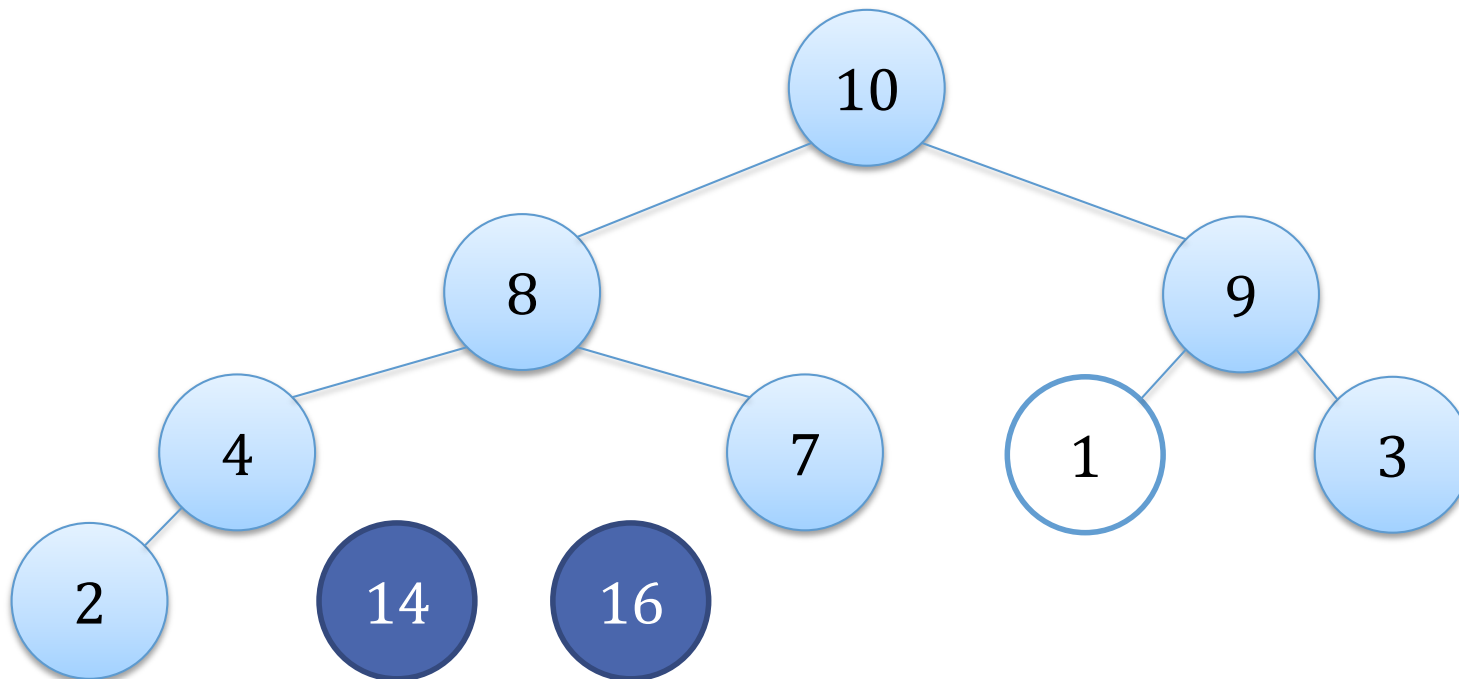
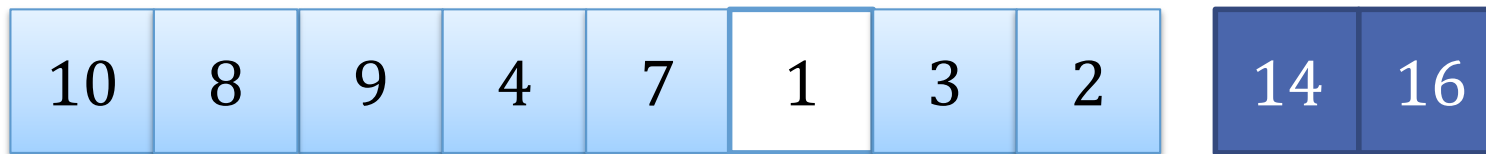
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



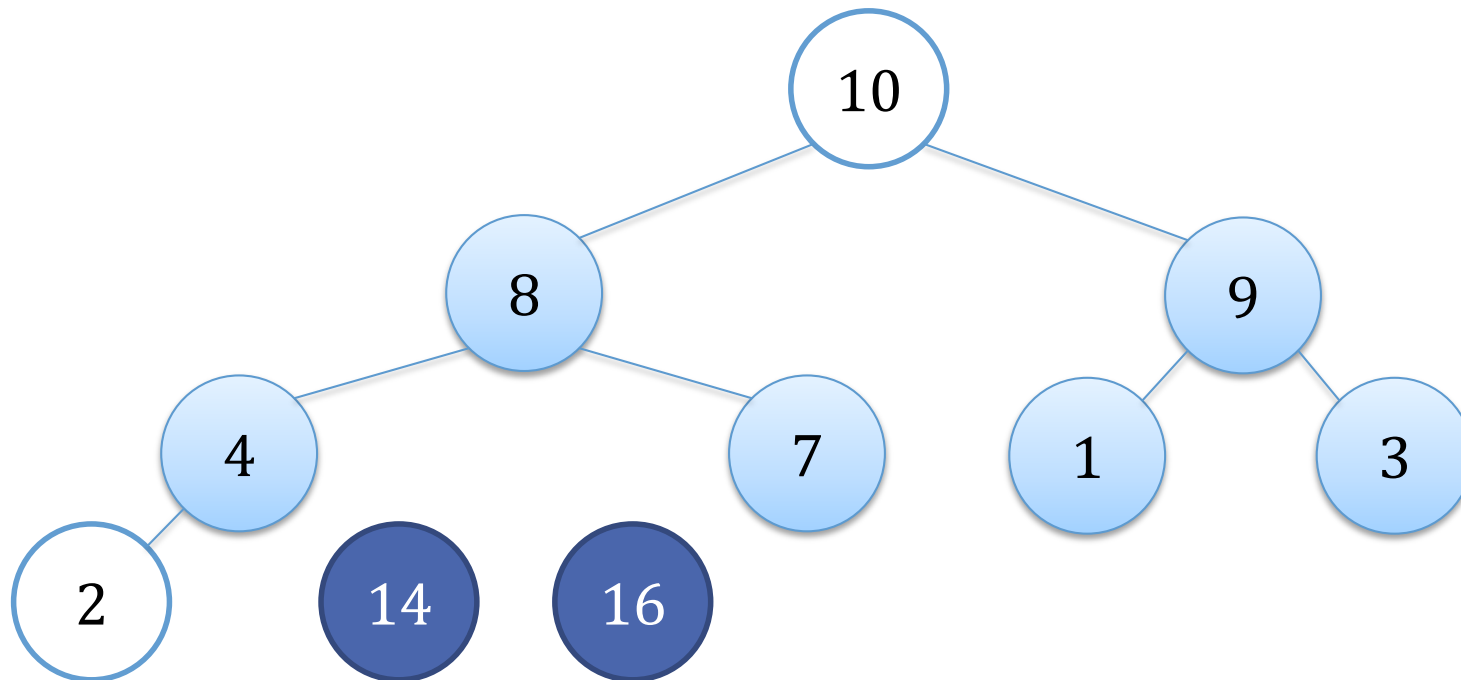
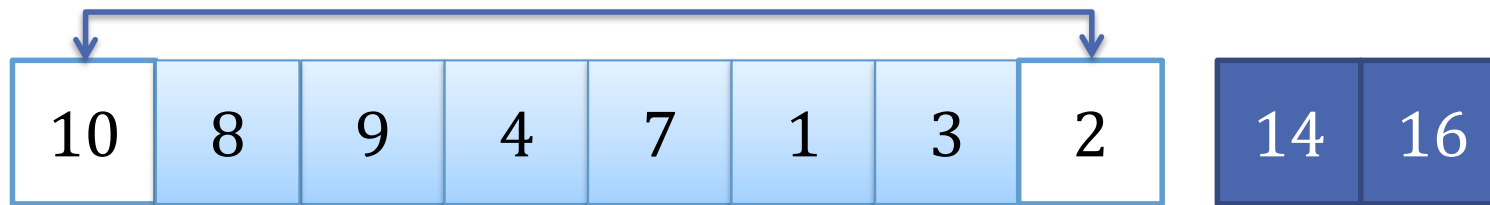
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



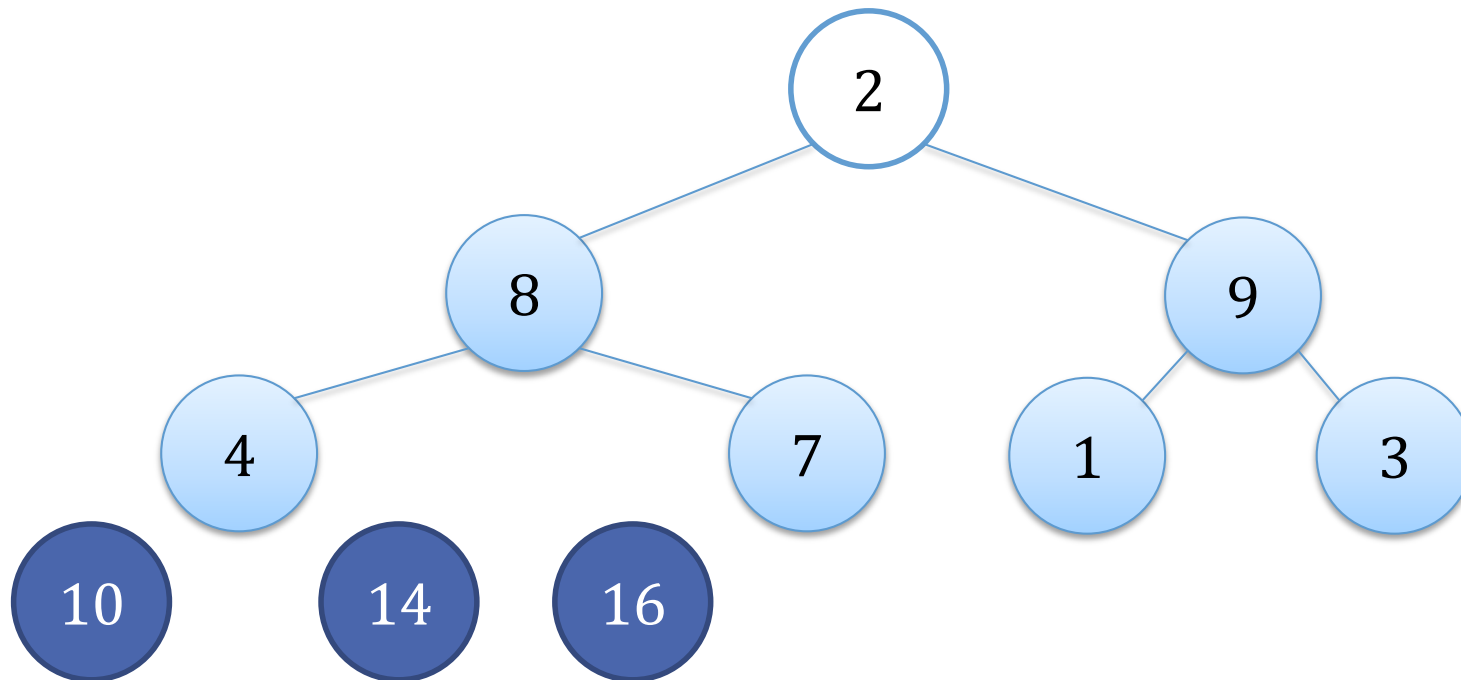
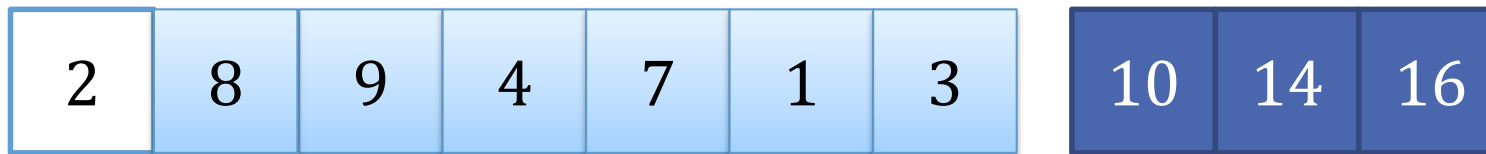
Heap sort Algorithm

Exchange



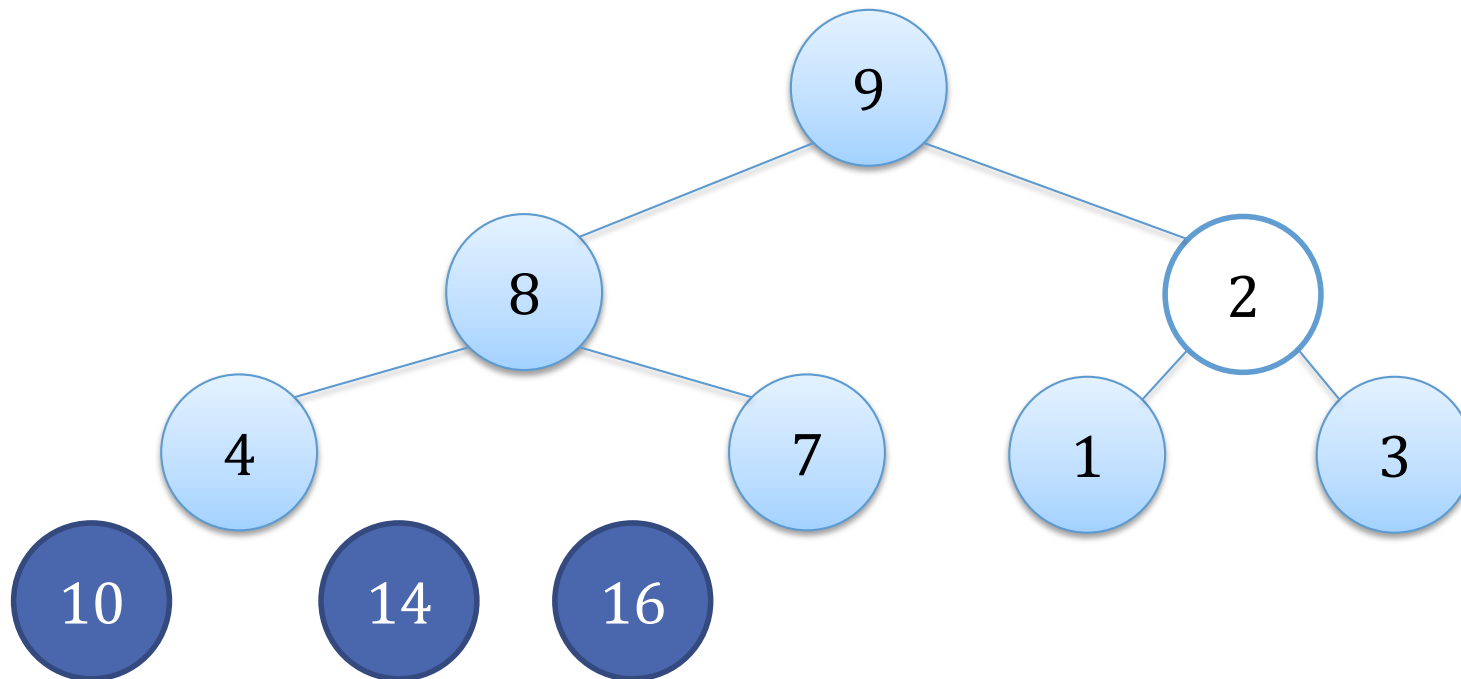
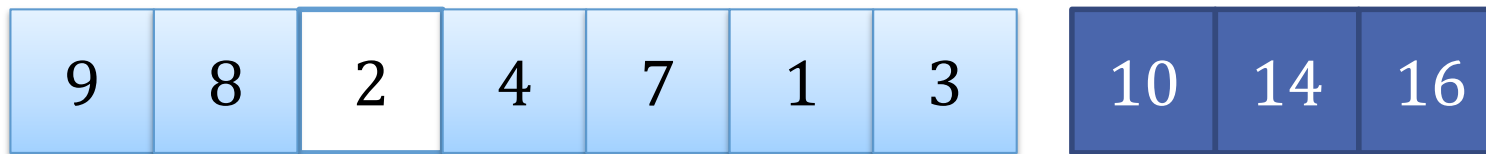
Heap sort Algorithm

- Discard the last node



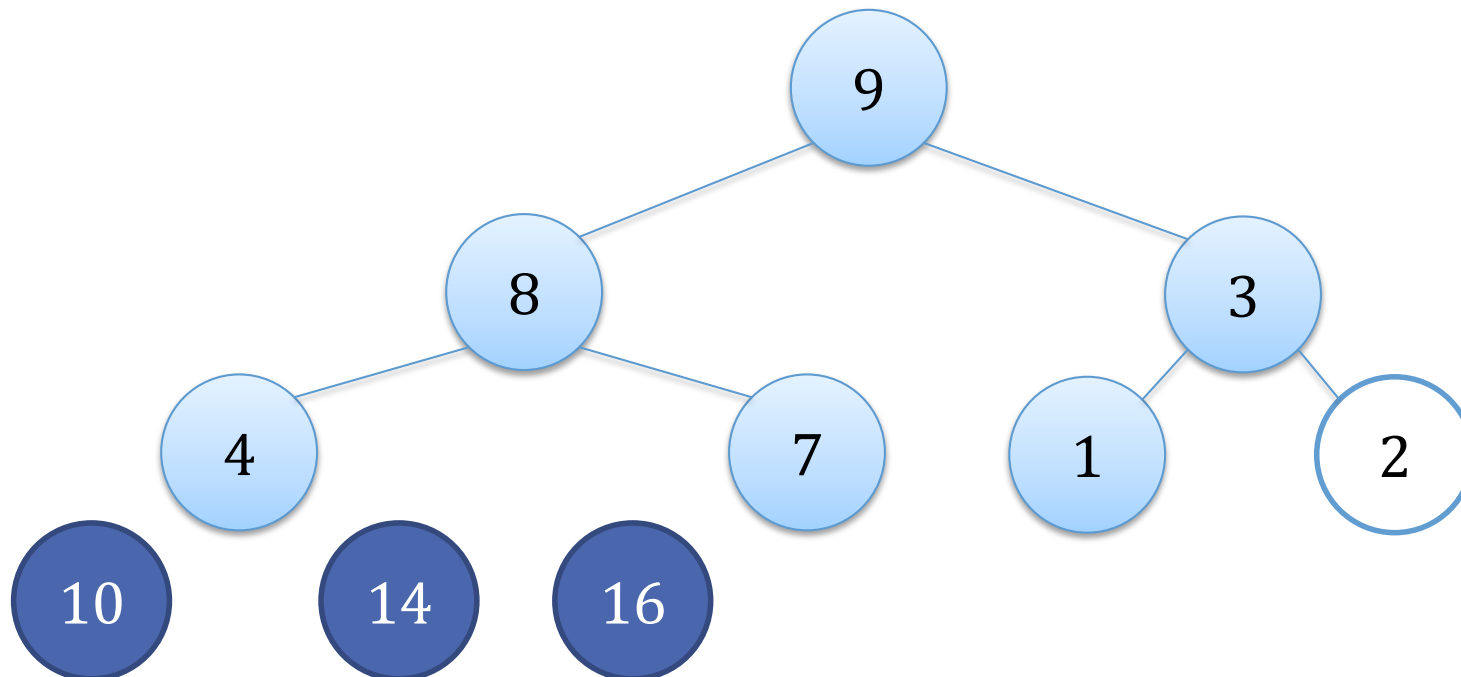
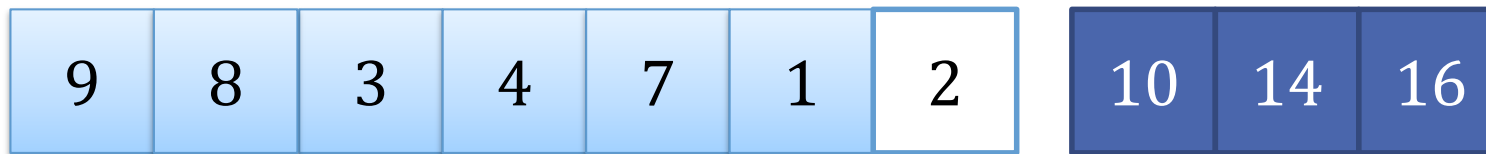
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



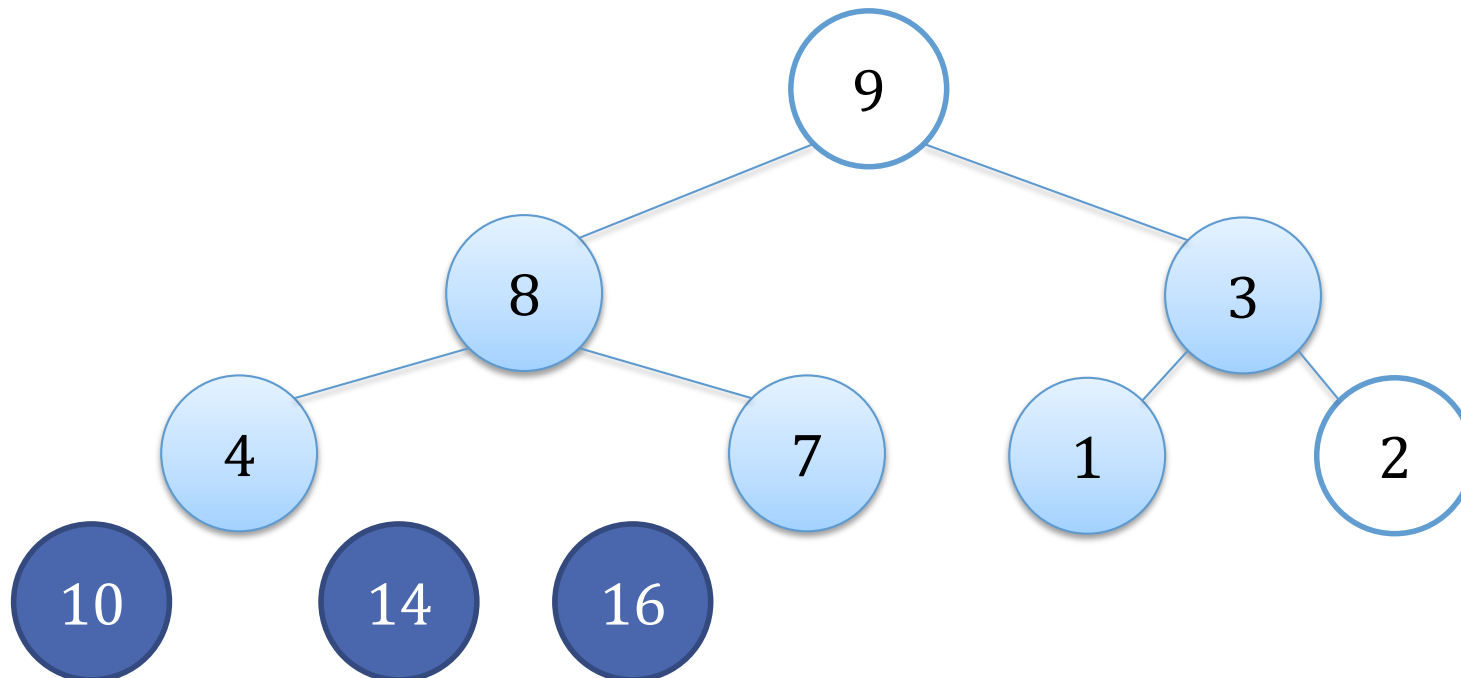
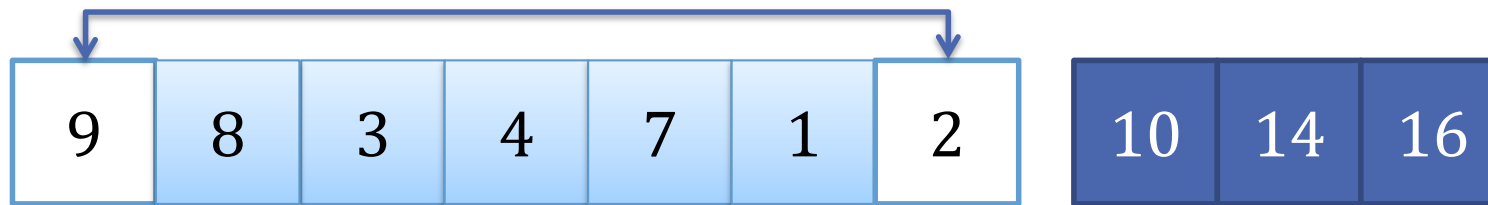
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



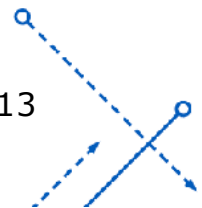
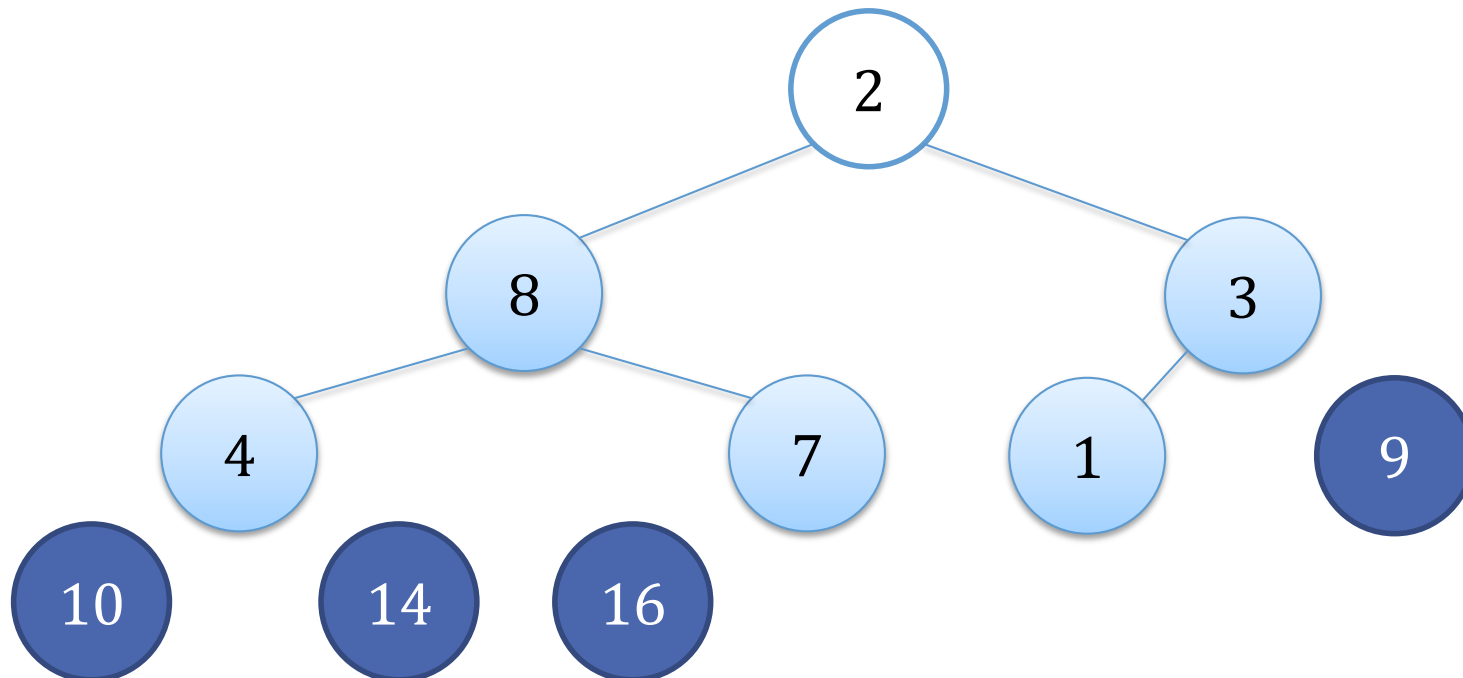
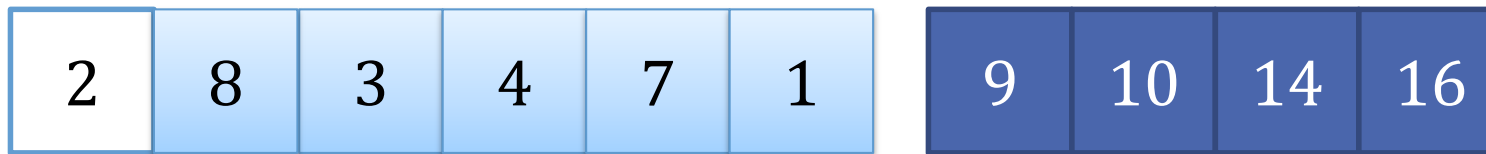
Heap sort Algorithm

Exchange



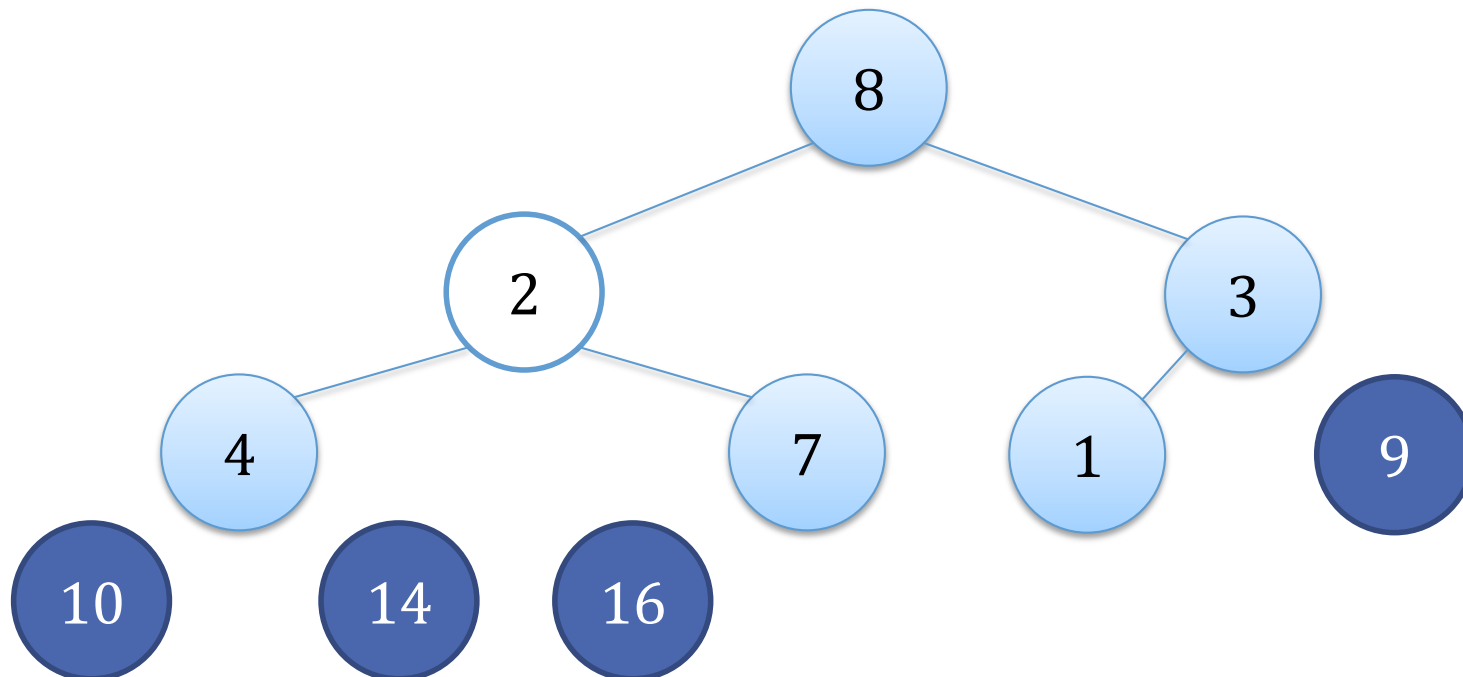
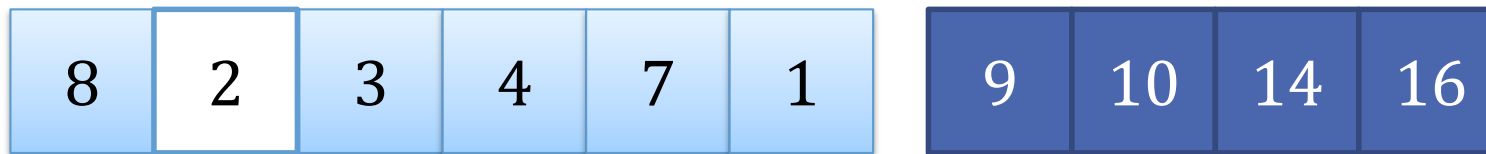
Heap sort Algorithm

- Discard the last node



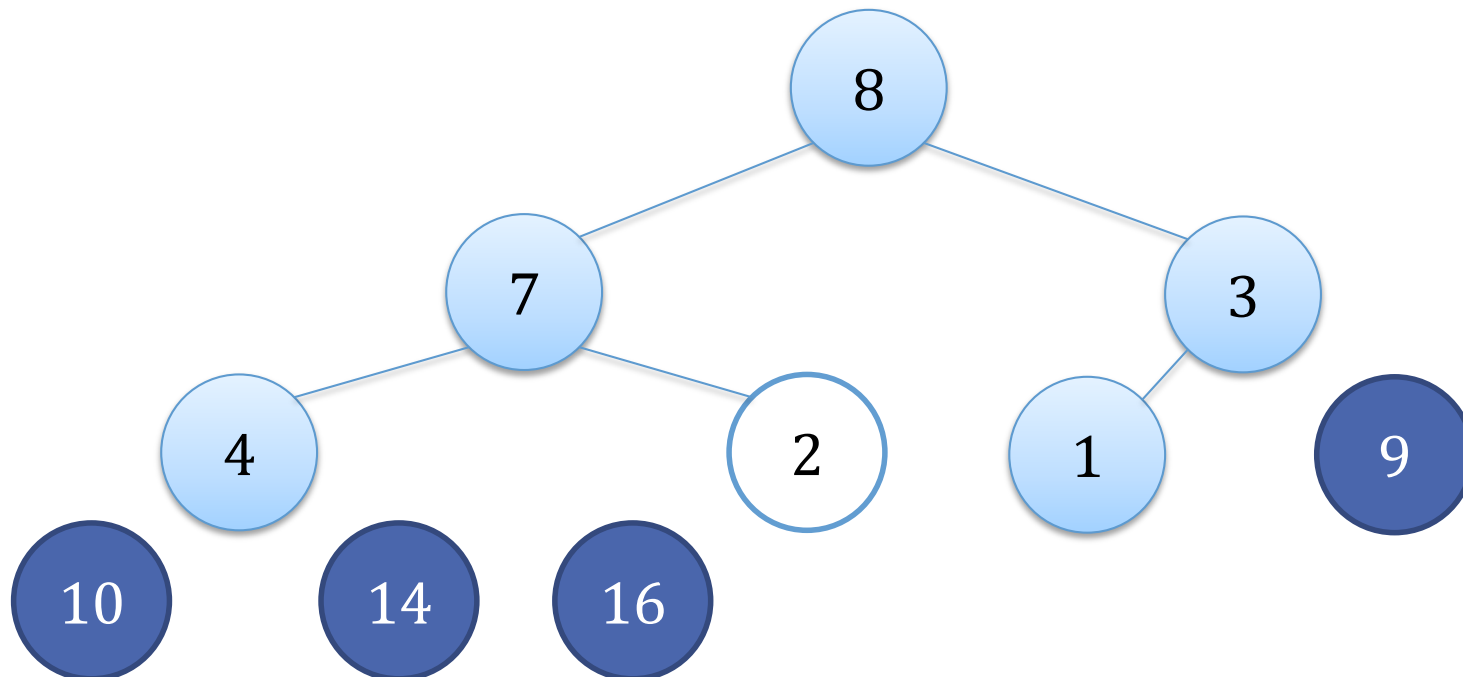
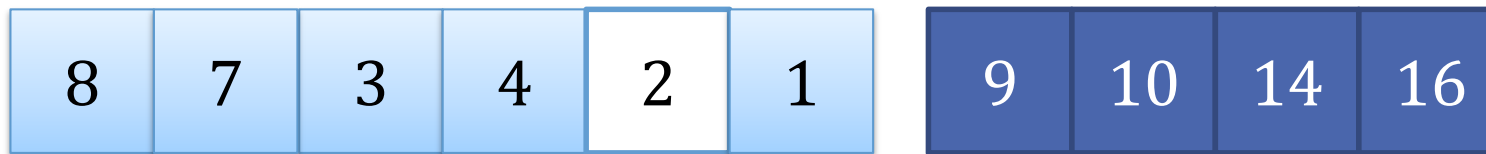
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



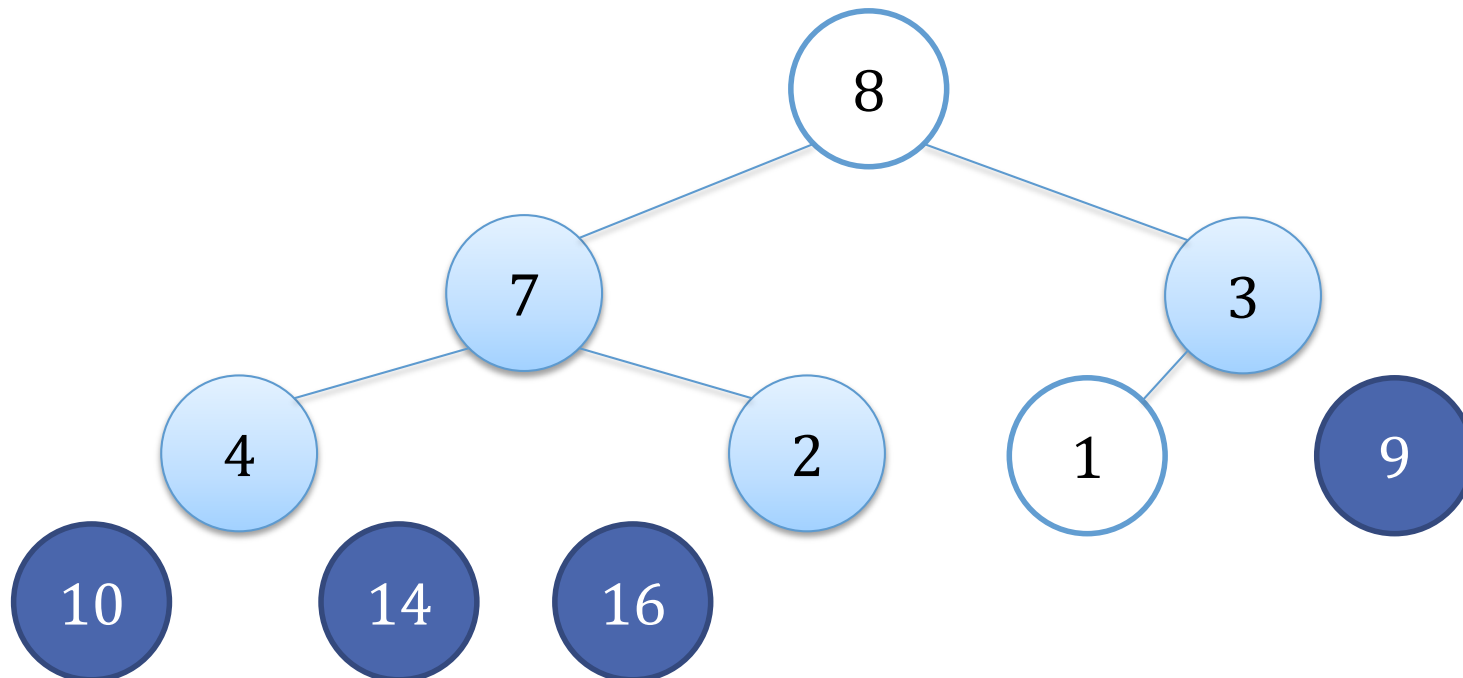
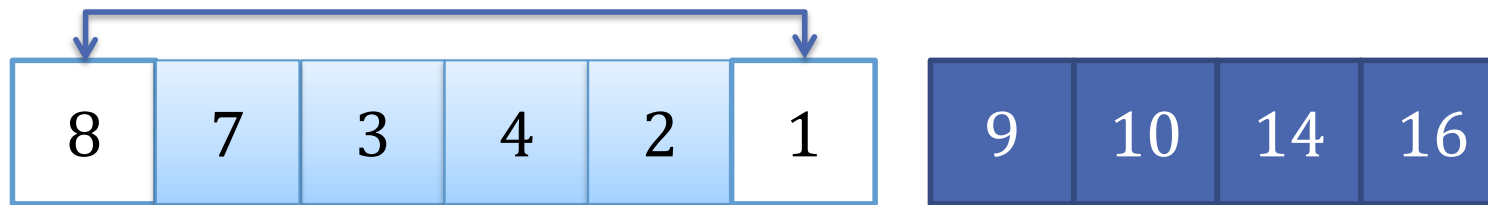
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



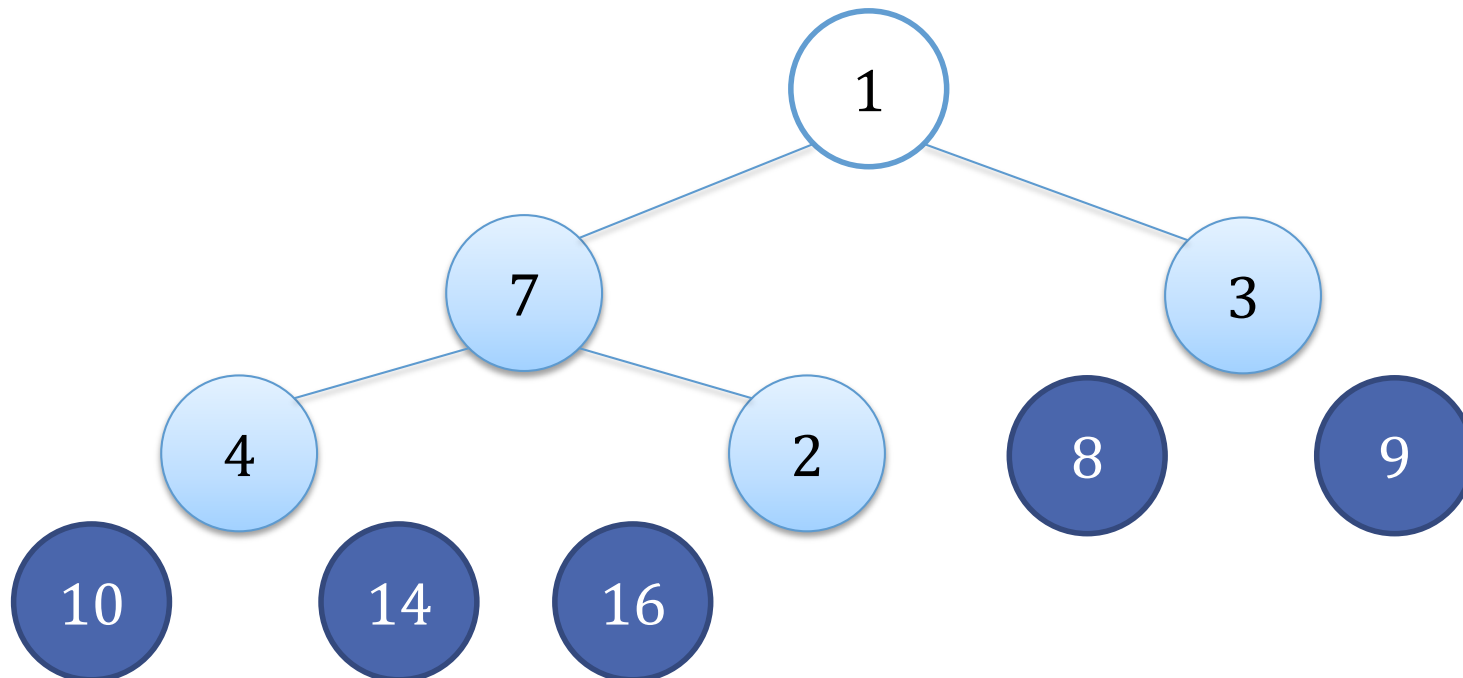
Heap sort Algorithm

Exchange



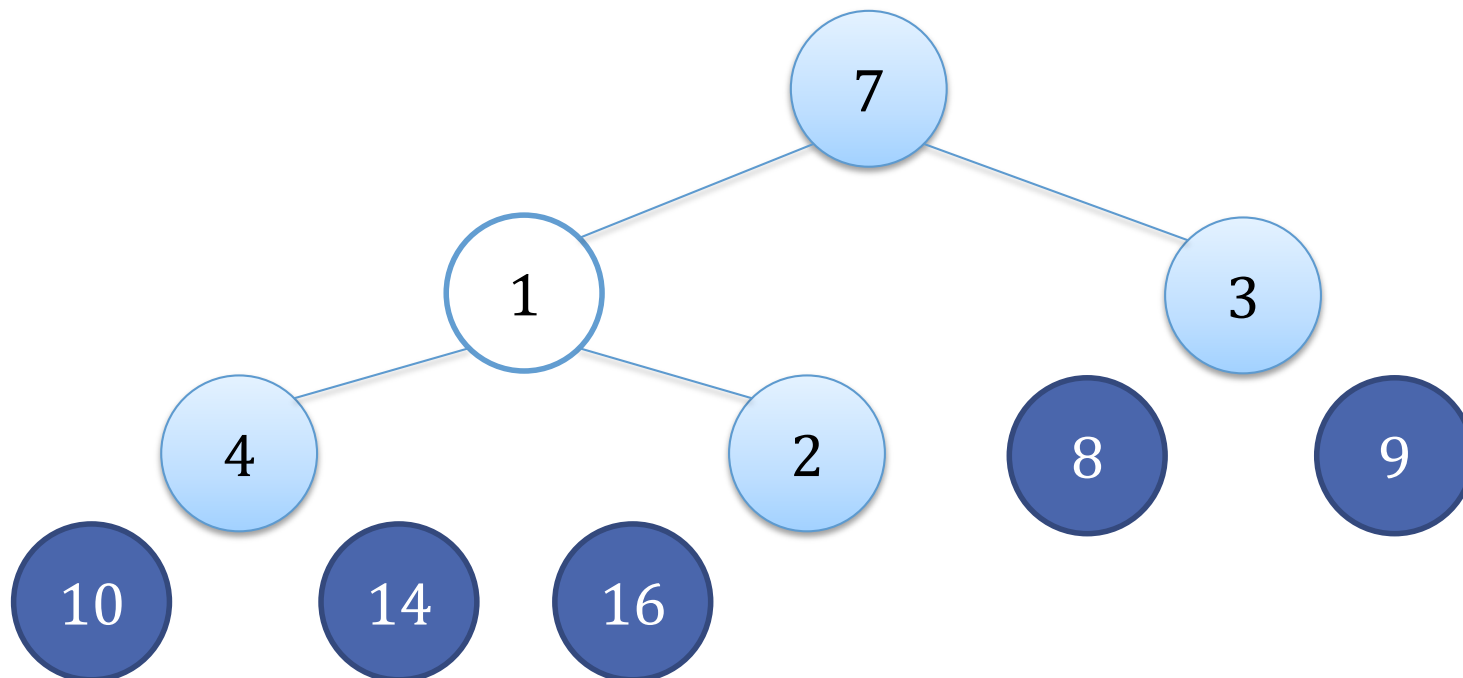
Heap sort Algorithm

❑ Discard the last node



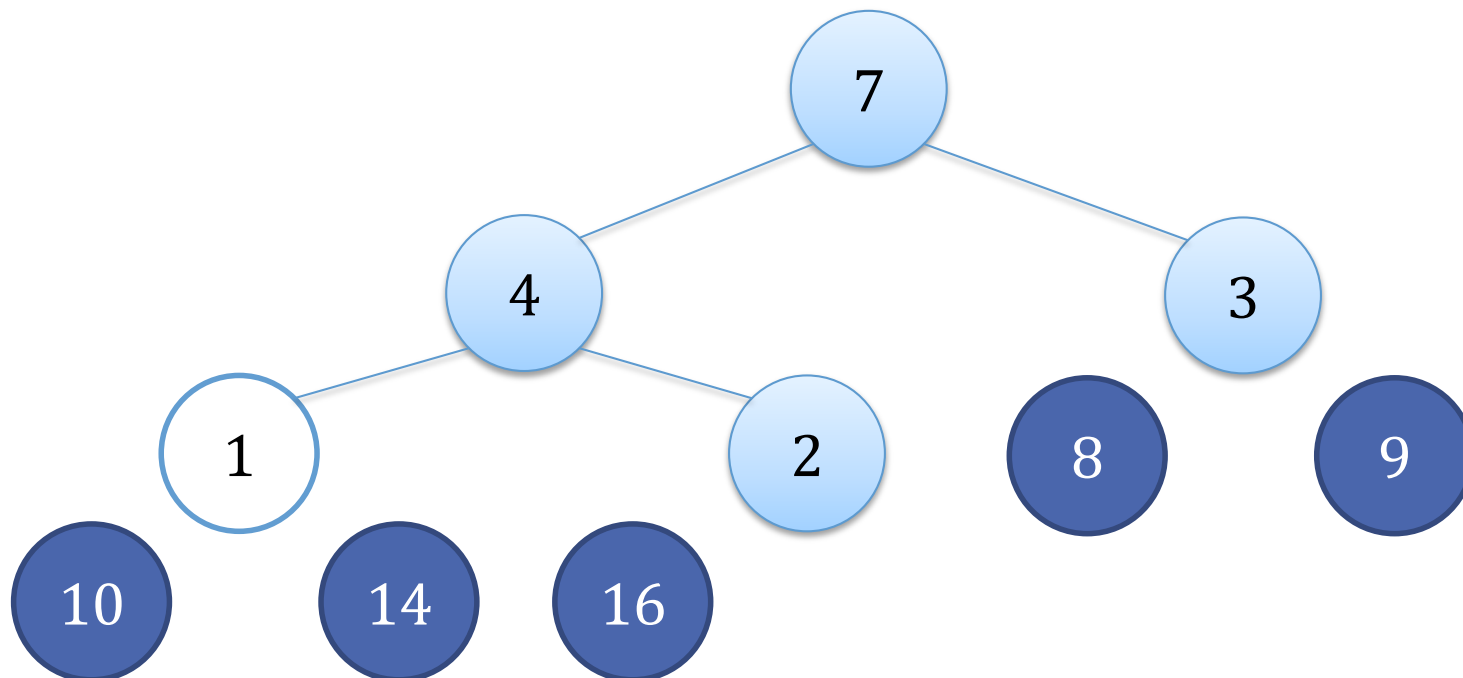
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



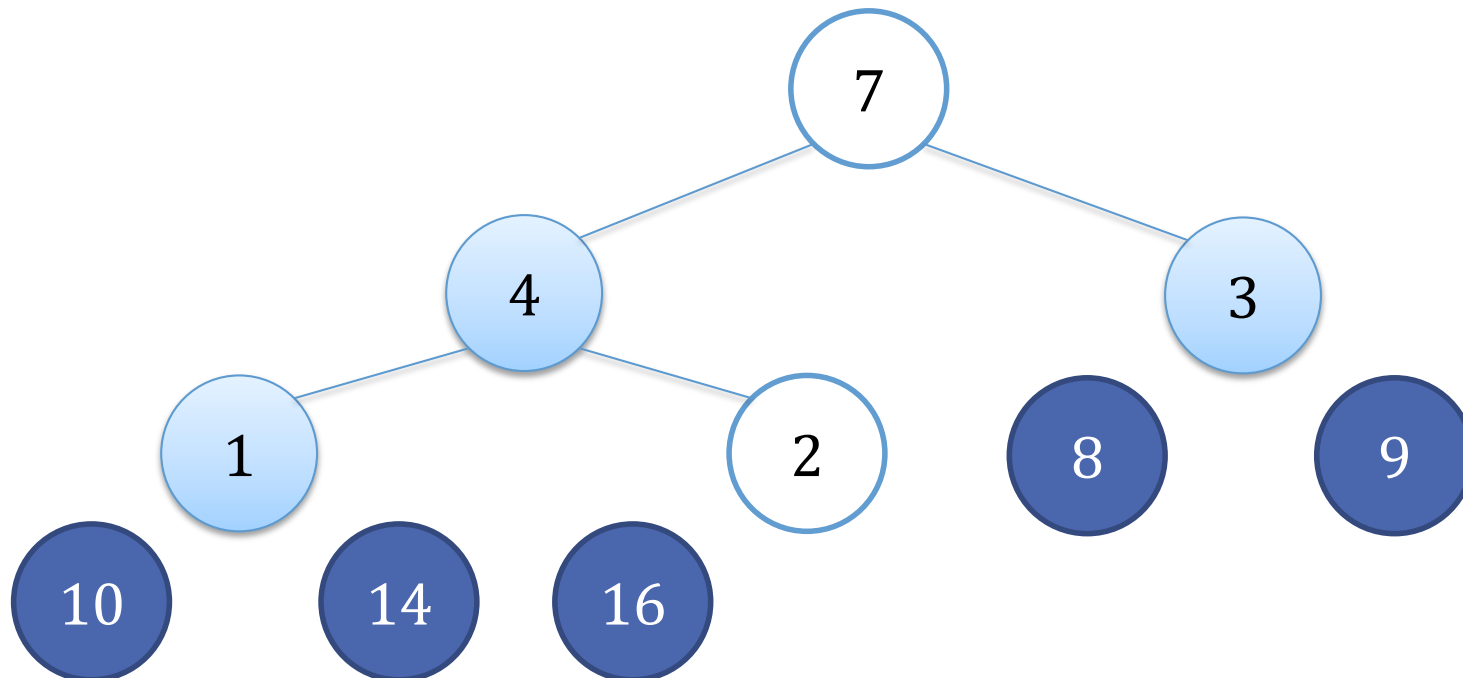
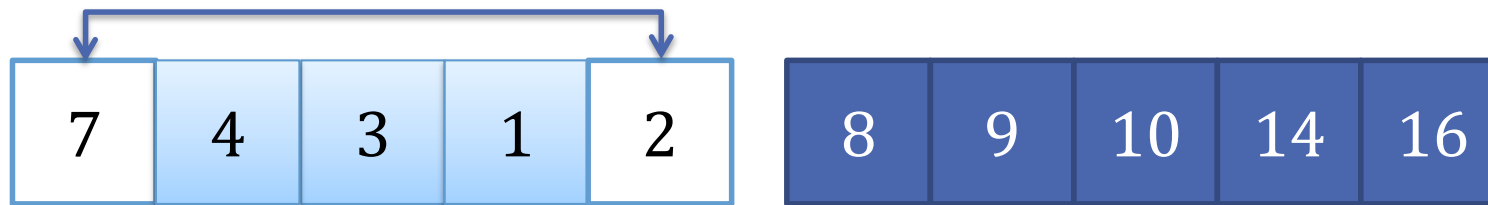
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



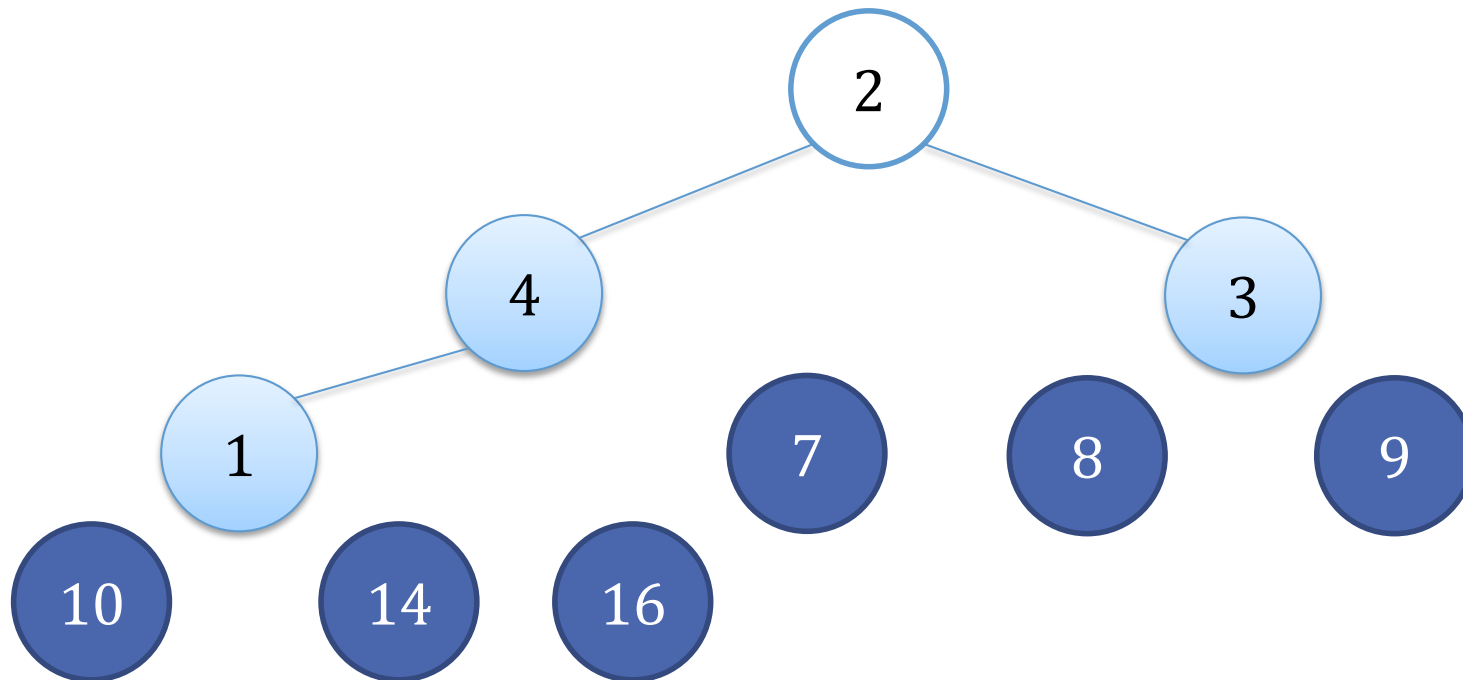
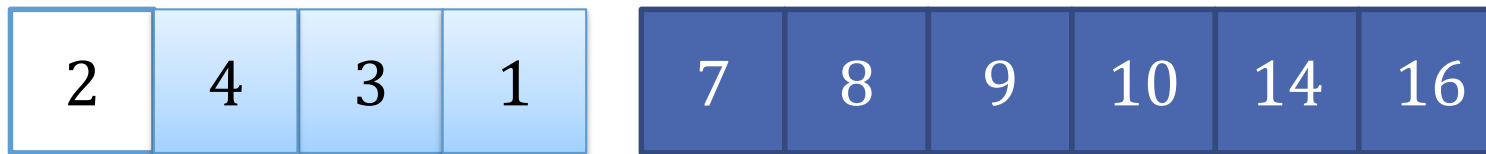
Heap sort Algorithm

Exchange



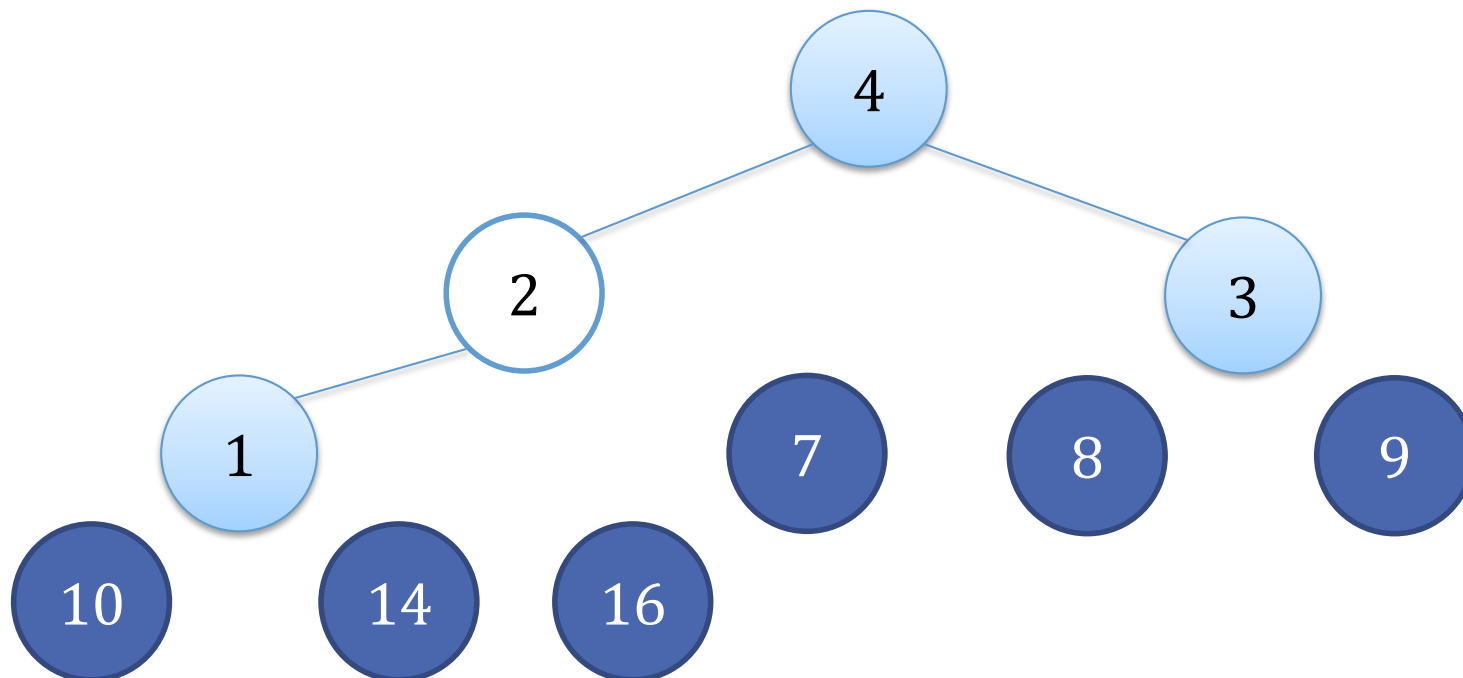
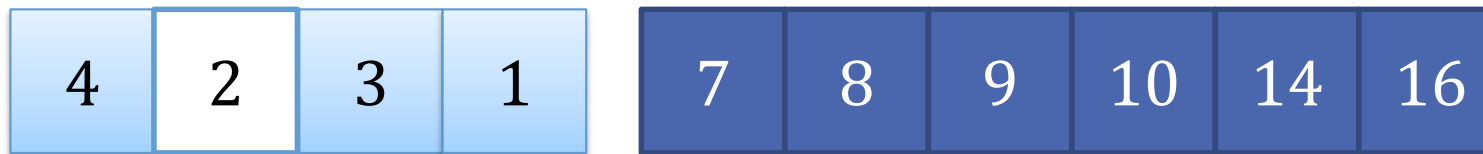
Heap sort Algorithm

❑ Discard the last node



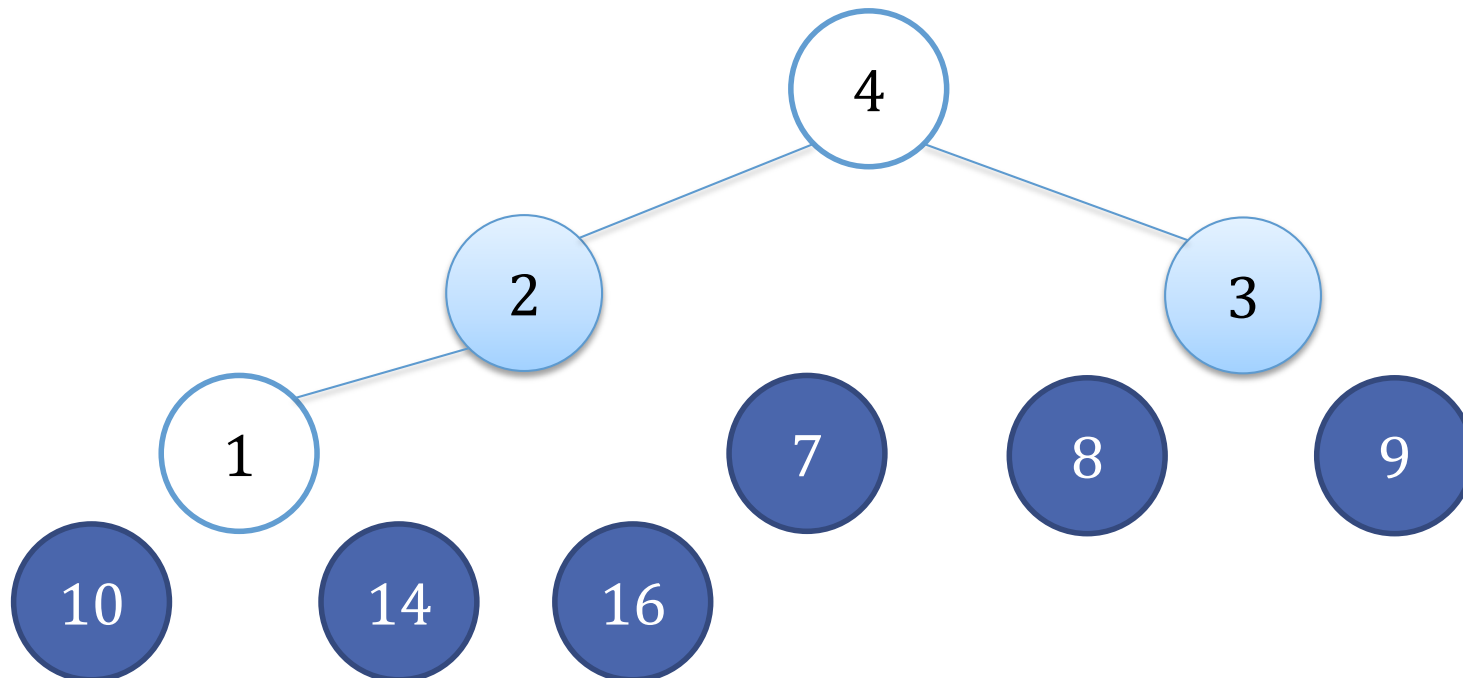
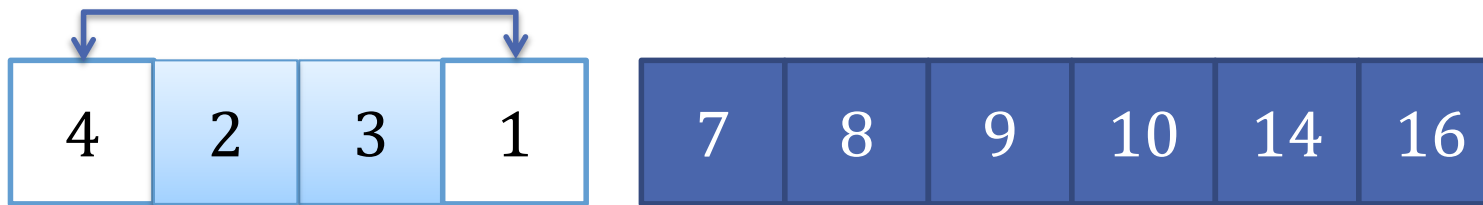
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



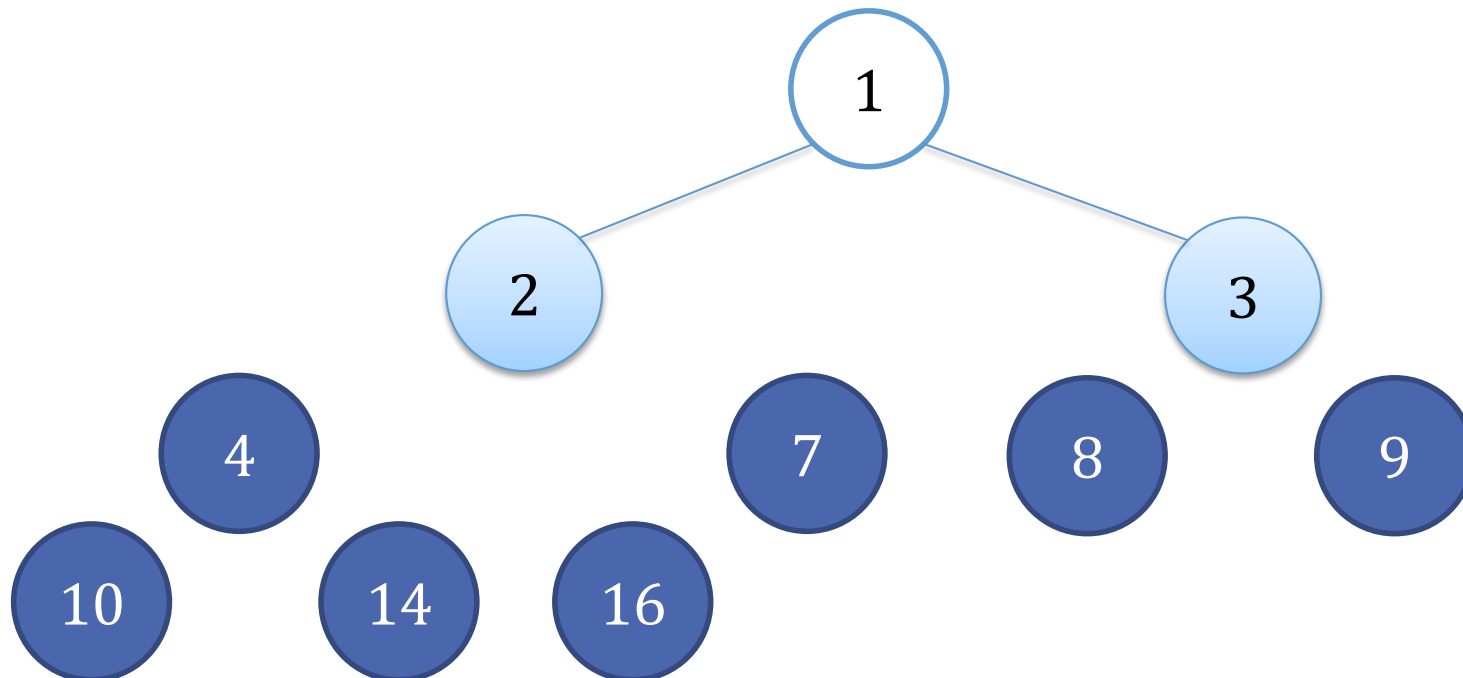
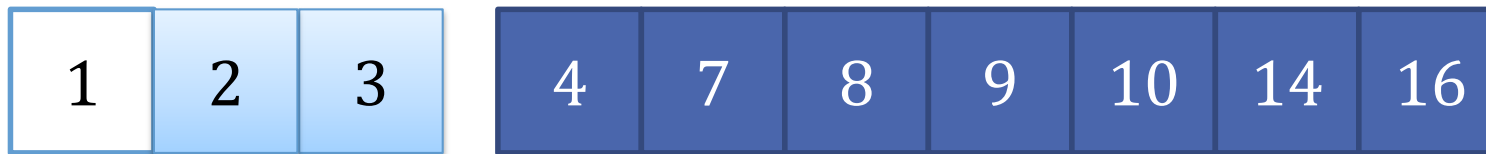
Heap sort Algorithm

Exchange



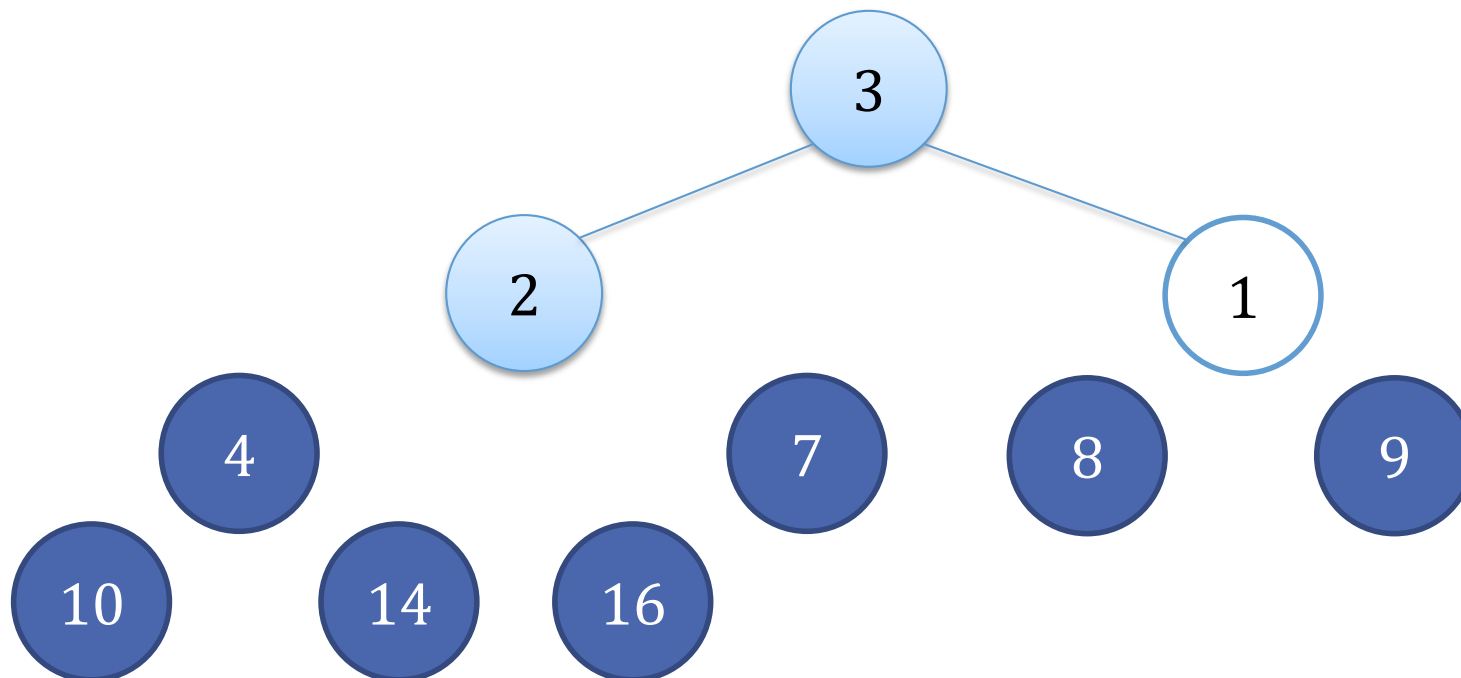
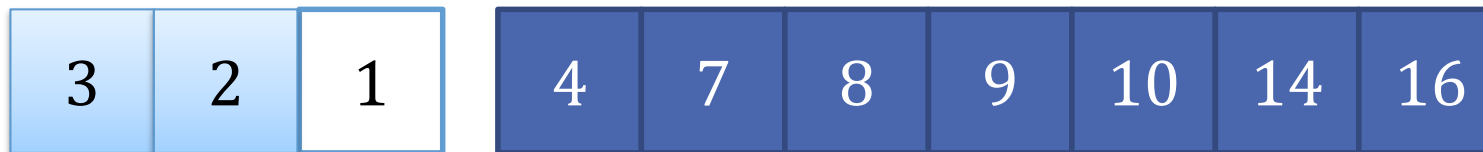
Heap sort Algorithm

❑ Discard the last node



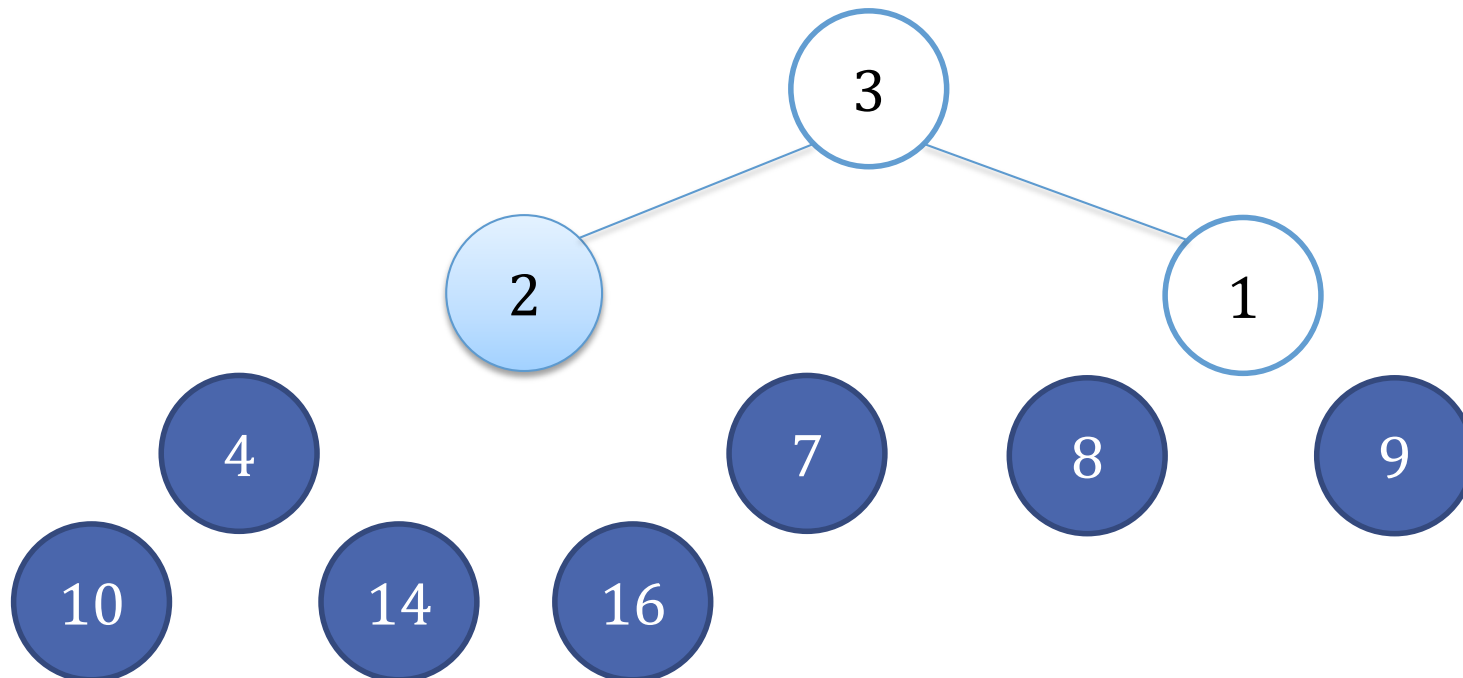
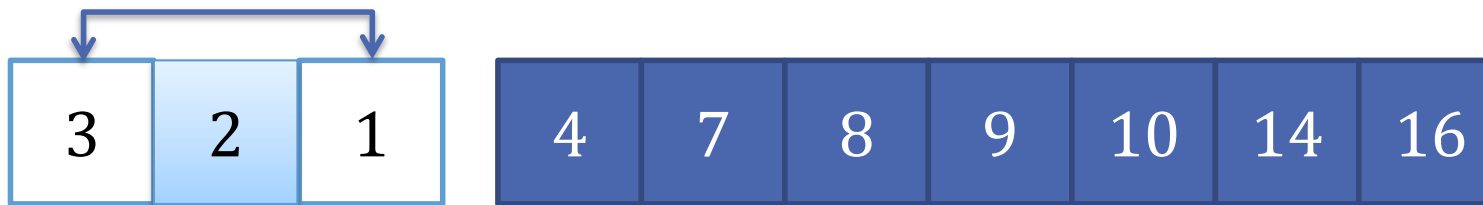
Heap sort Algorithm

❑ MAX-HEAPIFY (A,0)



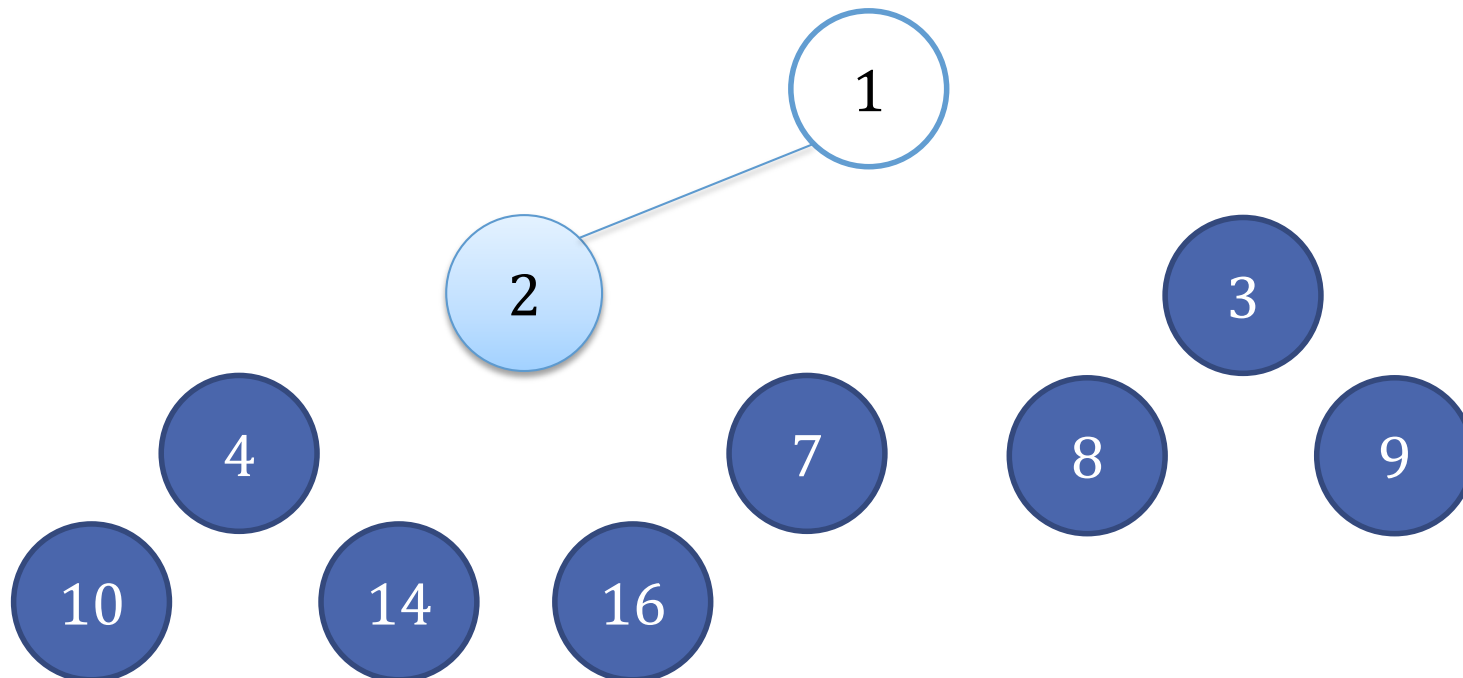
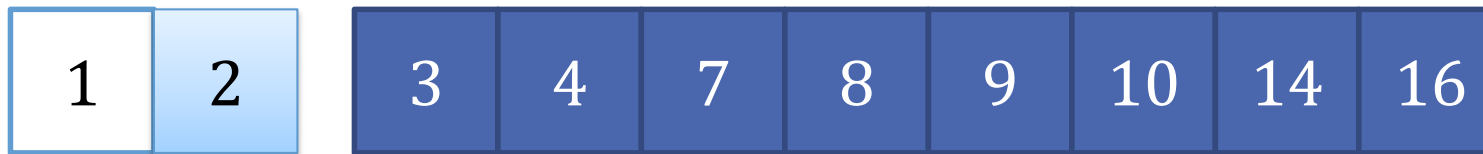
Heap sort Algorithm

Exchange



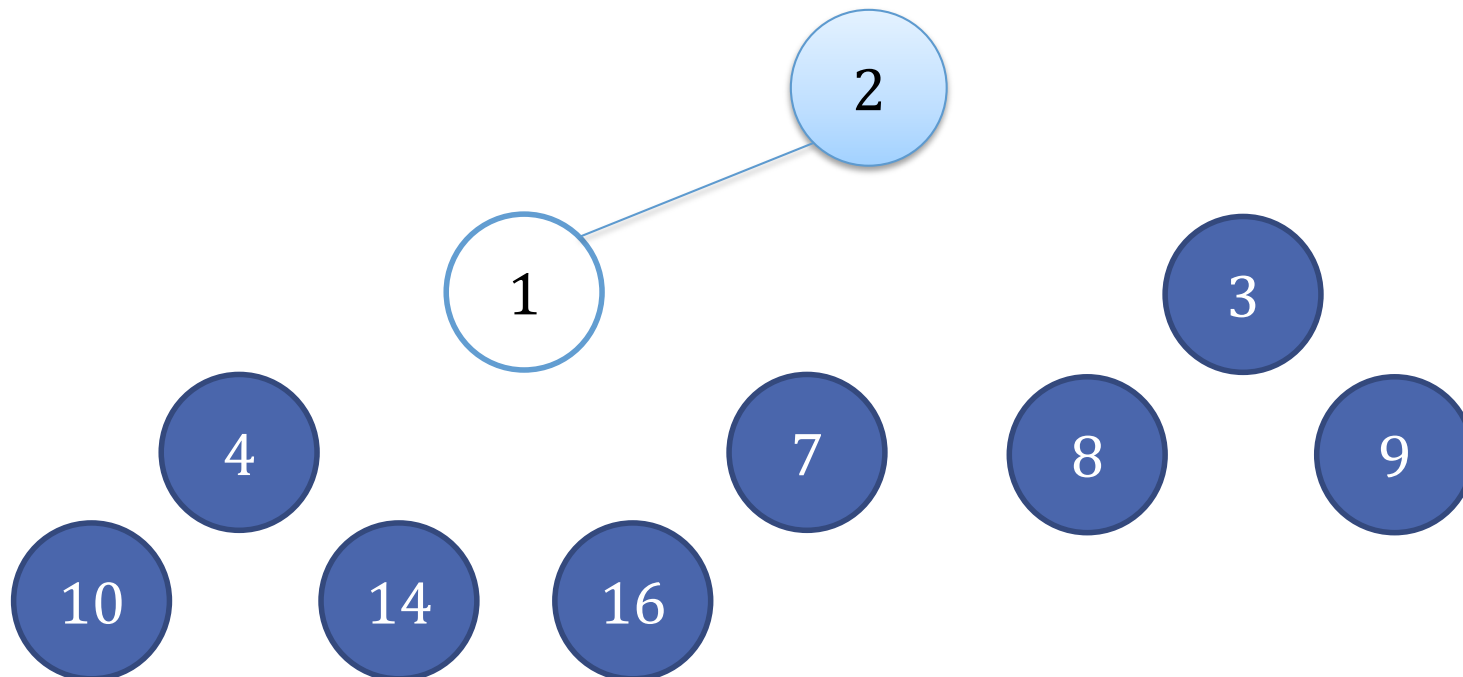
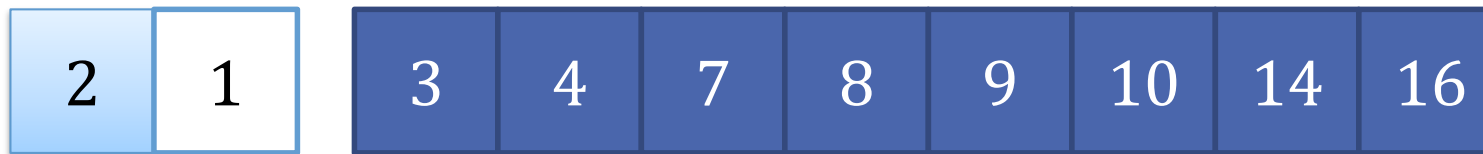
Heap sort Algorithm

❑ Discard the last node



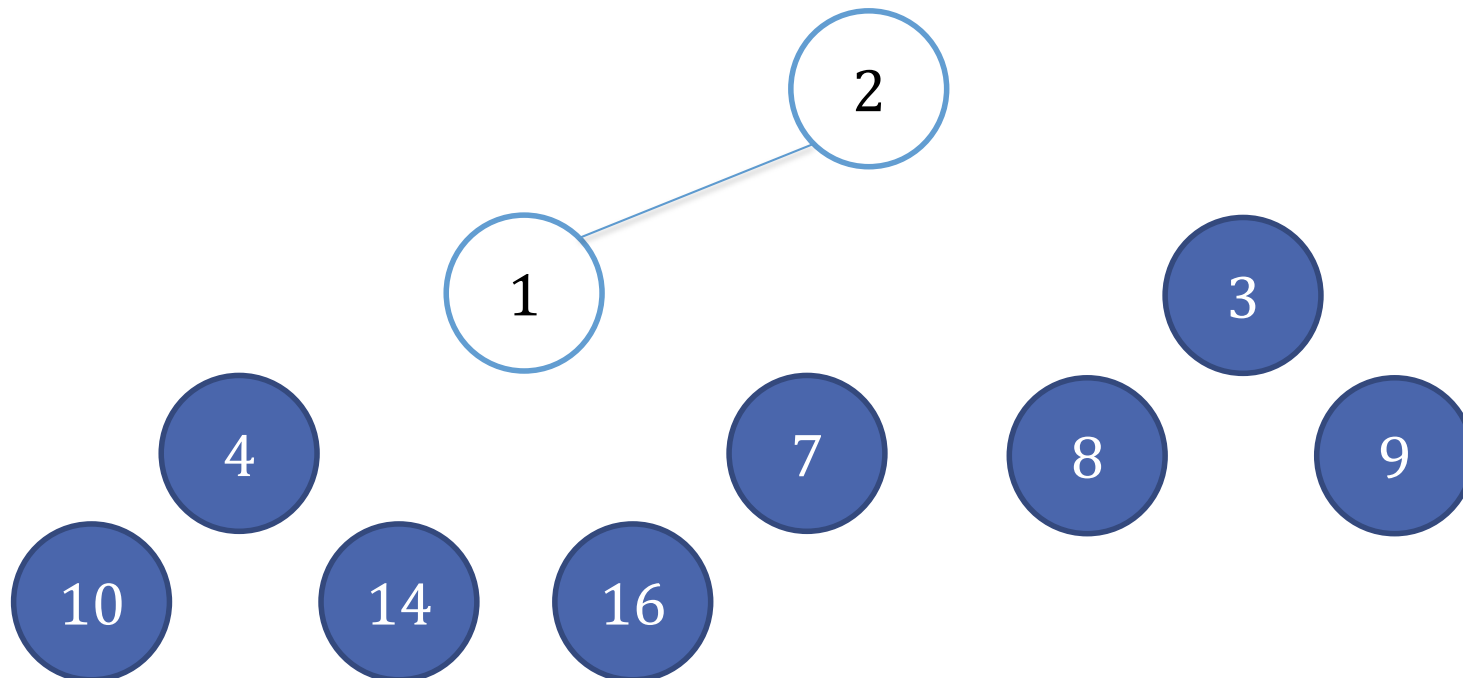
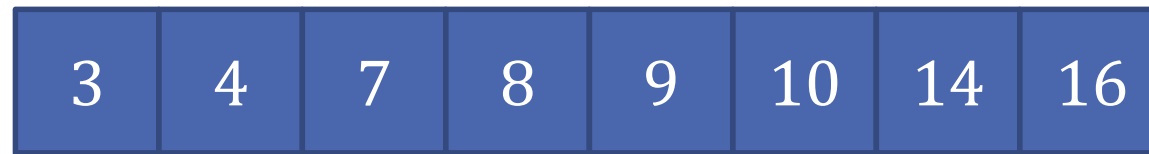
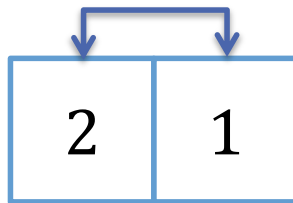
Heap sort Algorithm

□ MAX-HEAPIFY (A,0)



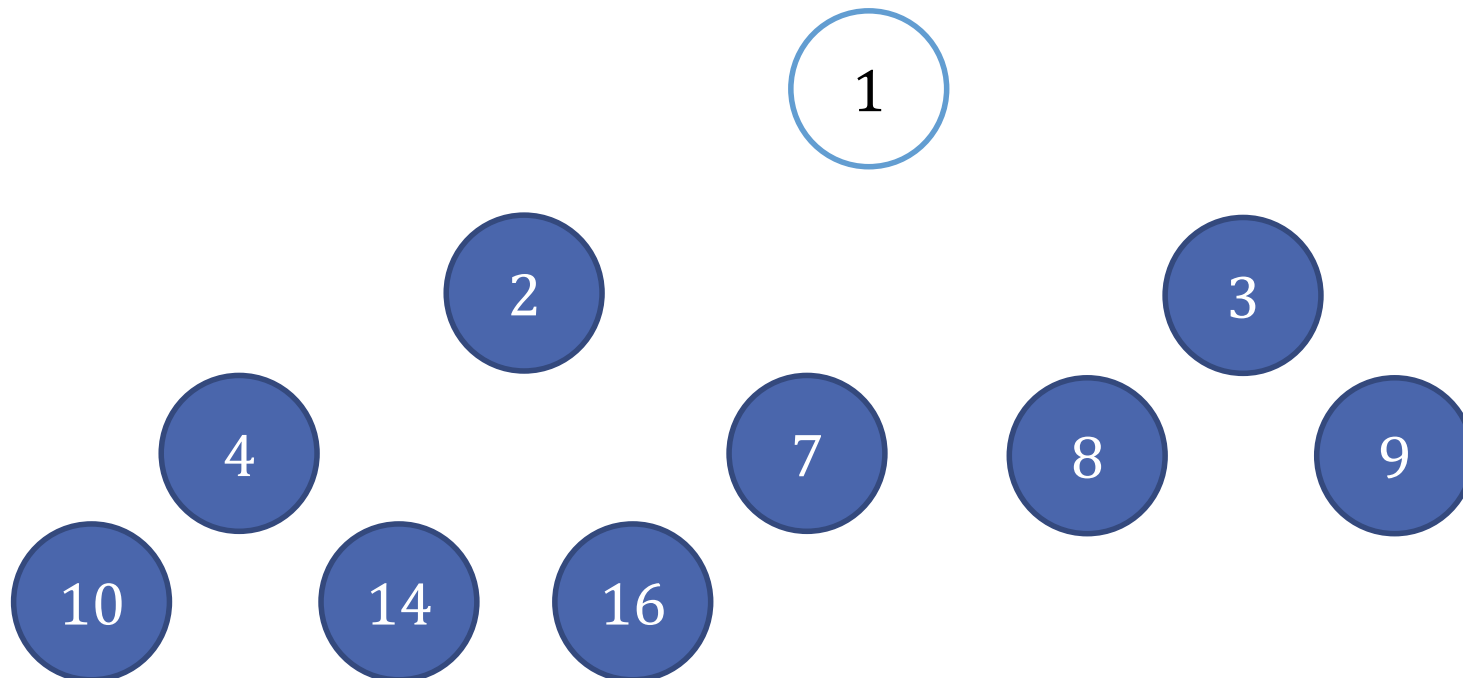
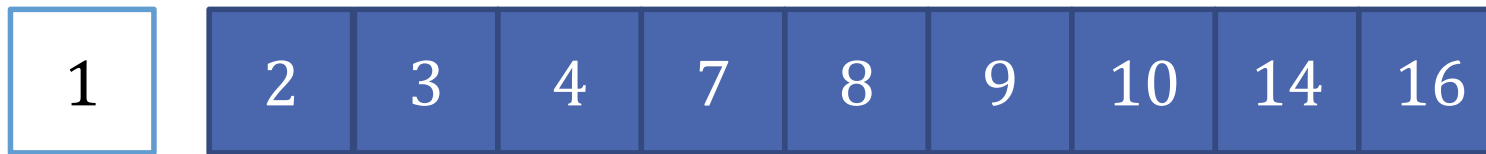
Heap sort Algorithm

Exchange



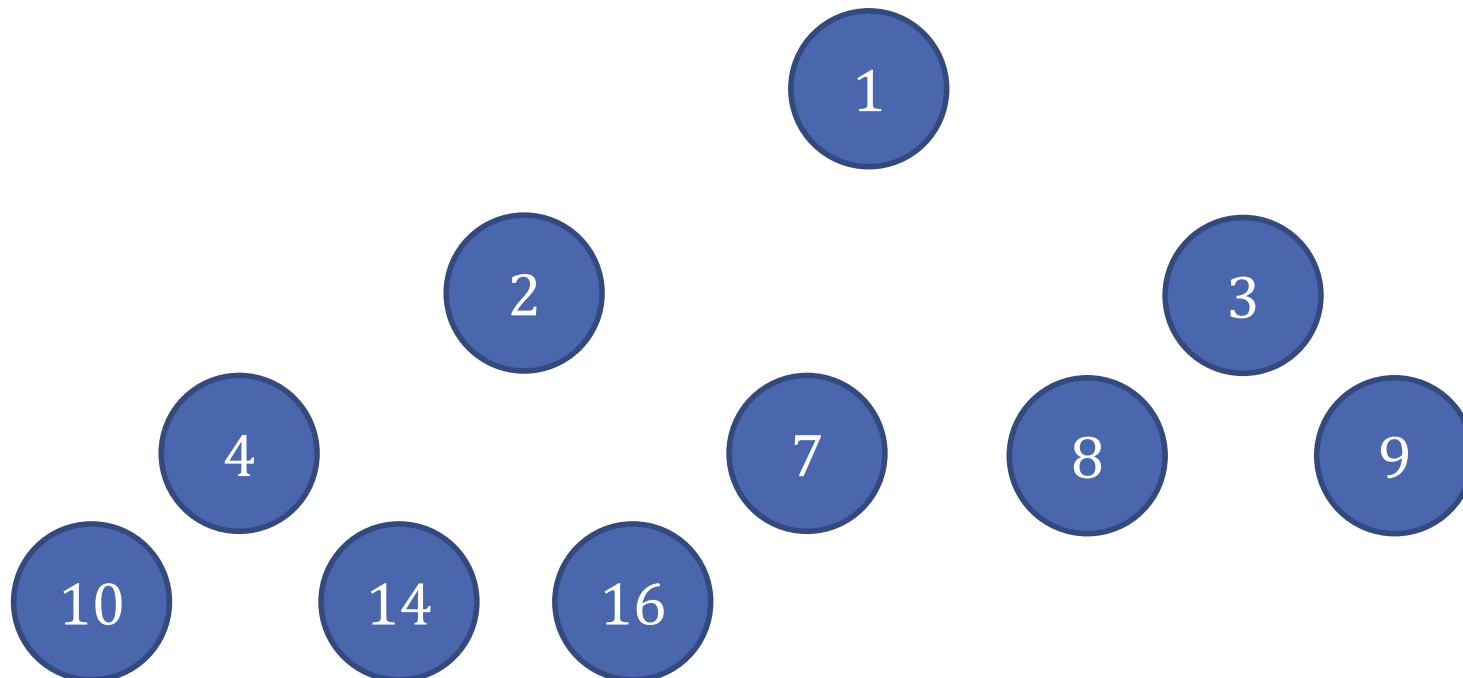
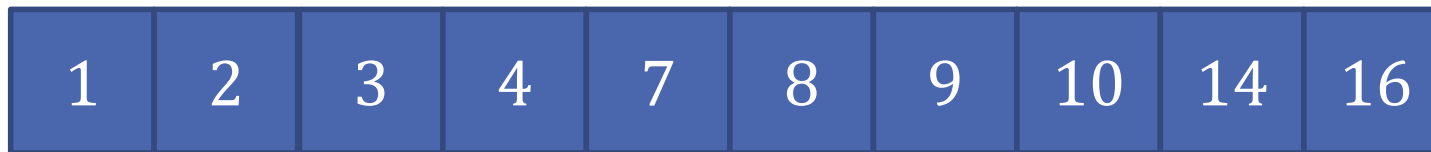
Heap sort Algorithm

❑ Discard the last node



Heap sort Algorithm

□ Finish!



Building a Heap

BUILD-MAX-HEAP(A,n) //Build a max heap from the original array A //Input: Array A[0..n – 1] //Output: Array A[0..n – 1] as a max-heap	Cost times
1 for $i \leftarrow n/2$ to 0 do 2 MAX-HEAPIFY(A,i)	$C(n)$

Maintaining the heap property

MAX-HEAPIFY(A,i) //Input: Array A[0..n – 1] and index i //Output: Array A[0..n – 1] as a max-heap	Cost times
<pre> 1 l ← LEFT(i) //l ← 2i+1 2 r ← RIGHT(i) //r ← 2i+2 3 if l ≤ A.heapsize and A[l] > A[i] 4 largest ← l 5 else largest ← i 6 if r ≤ A.heapsize and A[r] > A[largest] 7 largest ← r 8 if largest ≠ i 9 swap A[i] with A[largest] 10 MAX-HEAPIFY(A,largest) </pre>	$O(\log_2 n)$



The Heapsort Algorithm

1. **BUILD-MAX-HEAP** on the input array $A[0 \dots n-1]$, where $n = \text{length}[A]$
2. **SORTING:**
 - Exchange the root $A[0]$ with $A[n-1]$
 - Discard node $n-1$ from the heap (by decrementing $\text{heap-size}[A]$)
 - Restore the max-heap property using MAX-HEAPIFY($A, 0$)
 - Repeats this process for the max-heap of size $n-1$ to a heap of size 2.



The Heapsort Algorithm

HEAPSORT(A,n) //Input: Array A[0..n – 1] //Output: Array A[0..n – 1] sorted increasingly	Cost times
1 BUILD-MAX-HEAP(A,n) 2 for i $\leftarrow$ n-1 to 1 do 3 swap A[i] with A[0] 4 A.heapsize $\leftarrow$ A.heapsize - 1 5 MAX-HEAPIFY(A,0)	$O(n\log_2 n)$ $O(\log_2 n)$

Heapsort Analysis

1. The heap construction stage: $O(n \log_2 n)$
2. The second stage: the number of key comparisons $C(n)$ needed for eliminating the root keys from the heaps of diminishing sizes from n to 2, we have:

$$\begin{aligned}
 C(n) &\leq 2\lfloor \log_2(n-1) \rfloor + 2\lfloor \log_2(n-2) \rfloor + \cdots + 2\lfloor \log_2 1 \rfloor \leq 2 \sum_{i=1}^{n-1} \log_2 i \\
 &\leq 2 \sum_{i=1}^{n-1} \log_2(n-1) = 2(n-1) \log_2(n-1) \leq 2n \log_2 n. \in O(n \log_2 n)
 \end{aligned}$$

3. For both stages, we get: $O(n \log_2 n) + O(n \log_2 n) =$
 $O(n \log_2 n)$



Heapsort – Pros and Cons

□ Pros:

- The performance is guaranteed (unaffected by original ordering of the input)
 - Competitive running time with Mergesort
- In-place sort (does not require extra storage)

□ Cons:

- Practical experiments show that heapsort runs more slowly than quicksort



Quiz

- Classify the following sorting algorithms to groups of
 - a) Stable/non-stable sorts
 - b) In-place/not in-place sorts
 - c) Adaptable/not adaptable sorts
- 1. Selection Sort
- 2. Bubble Sort
- 3. Insertion Sort
- 4. Interchange Sort
- 5. Comb Sort
- 6. Cocktail Sort
- 7. Binary Insertion Sort
- 8. Shell Sort
- 9. Mergesort
- 10. Quicksort
- 11. Heapsort



What's next?

□ After today:

- Read textbook 1 – Chapter 2 (page 223~)
- Read textbook 3 – 3.1, 4.1, 5.1, 5.2, 6.4
- Review Lecture Sorting in the previous course

□ Next Week:

- Individual Assignment 1
- Lecture 2: Sorting Algorithms (part 2)
- Lecture 3: Searching Algorithms



Q&A